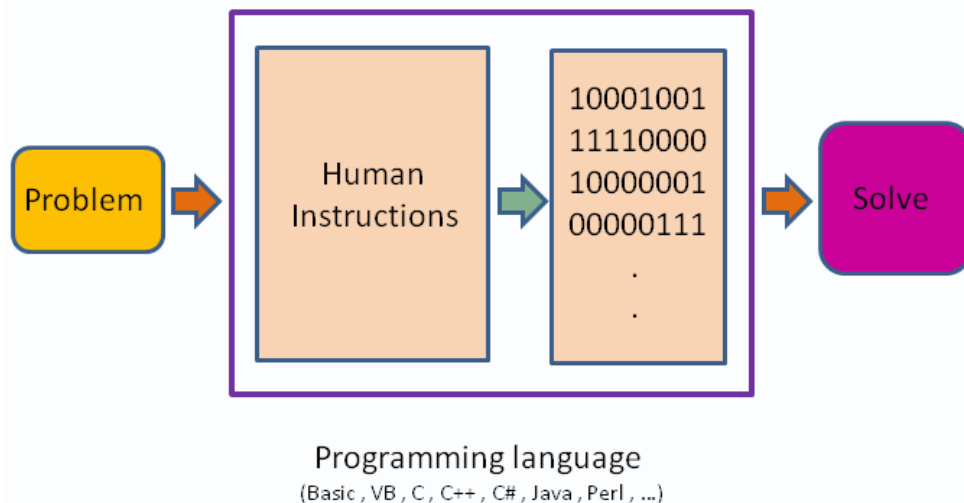


Poglavlje 0: Predgovor

0.1 Pojam programiranja

- *Program* je sekvenca (niz) instrukcija koje daju uputstvo računar kako da uradi neki zadatak; kada on sledi te instrukcije, kaže se da *izvršava* program; davanje tih instrukcija računar naziva se *programiranje*
- Ovo u svakodnevnom životu podseća na situaciju u kojoj domaćica sledi recept kako da napravi princes-krofne: ona ima ulogu računara, spremanje princes-krofni je zadatak, a recept daje instrukcije kako da se zadatak obavi
- Ali, za razliku od domaćice koja će uvideti eventualnu grešku ili nelogičnost u receptu, računar sve shvata bukvalno: njemu se instrukcije moraju dati bez grešaka ili nejasnoća; u slučaju da se pojavi i najmanja, zadatak neće biti odrađen (ili će se odraditi pogrešno)
- Sve što računar radi je implementirano u mašinskom kodu; on je zasnovan na binarnom sistemu, koji se sastoji od nula (odsustvo napona) i jedinica (prisustvo); ali za čoveka je potpuno nemoguće da se spusti na taj nivo; zato on koristi programske jezike, i u njima piše programe, koji će se na neki način konvertovati u mašinski kod



- Drugi razlog zašto je pisanje (programiranje) u mašinskom kodu gotovo nemoguće zadatak je što svaka računarska platforma (HW+OS) zahteva specifičan mašinski kod, što znači da bi za svaku mašinu morao da se piše različit program
- Savremeni programski jezici, osim što su bliži čoveku jer su mnogo jednostavniji da se pišu, omogućavaju i bolji interfejs između čoveka i računara; to se obavlja tako što se u nekom programskom jeziku piše univerzalni kod, koji se kasnije prevodi u mašinski kod za specifičnu platformu
- Da bi se to prevođenje ostvarilo, računar mora da bude u stanju da kompajlira ili da interpretira program koga je napisao čovek, kako bi se on konvertovao u mašinski kod koji može da se izvršava
- *Kompajliranje* je procedura u kojoj se napisani program, uz pomoć kompajlera, konvertuje u mašinski kod unapred, to jest pre nego što počne njegovo izvršavanje; taj mašinski kod se posle toga permanentno čuva; on, kao što je pomenuto, zavisi od računarske platforme; to znači da se kompajliranje obavlja za konkretnu platformu
- *Interpretiranje* je procedura u kojoj interpreter redom, liniju po liniju napisanog koda, prevodi u mašinski kod koji se odmah zatim izvršava; kao posledica toga, ne stvara se i ne čuva nikakva verzija celog programa u mašinskom kodu; kada se jedna instrukcija mašinskog koda izvrši, ona se odmah briše iz memorije
- Programski jezici se prema nivou dele na niže (bliži mašinskom kodu), i više (bliži ljudskom jeziku)
- Jezik najnižeg nivoa naziva se *assembler* ("assembly"): on koristi binarne instrukcije, pa se svaka linija assemblera konvertuje u jednu liniju mašinskog koda; svaka procesorska arhitektura ima svoj assembler; vrlo se retko koristi

- Od jezika koje programeri koriste, C se nalazi na najnižem nivou, iznad njega su C++ i Java, a Python je najviši; programer nekad sam bira koji jezik će koristiti; ne postoji jezik koji dominira nad ostalima, već svaki ima prednosti i mane

0.2 Brojni sistemi

- Čovek svakodnevno koristi decimalni brojni sistem, i u njemu (i nijednom drugom) obavlja izračunavanja; to je sistem čija je osnova 10; cifre u ovom sistemu su od 0 do 9

- Tako na primer, broj dana u godini zapisan u decimalnom brojnem sistemu je 365, jer (u decimalnom brojnem sistemu) važi jednakost:

$$365 = 3 \cdot 100 + 6 \cdot 10 + 5 \cdot 1 = \underline{3} \cdot 10^2 + \underline{6} \cdot 10^1 + \underline{5} \cdot 10^0$$

- Kao što se primećuje, prva cifra zdesna u zapisu broja množi osnovu (u ovom slučaju 10) na stepen 0, druga zdesna osnovu na stepen 1, i tako redom nalevo

- Računari koriste binarni brojni sistem; to je sistem čija je osnova 2; cifre u ovom sistemu su 0 i 1; postojanje samo dve cifre je pogodno za računar, jer on "zna" samo za dva nivoa signala: nema napona (logička 0) i ima napona (logička 1)

- Tako na primer, broj dana u godini zapisan u binarnom brojnem sistemu je 101101101, jer (u decimalnom brojnem sistemu) važi jednakost:

$$365 = 1 \cdot 256 + 0 \cdot 128 + 1 \cdot 64 + 1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = \underline{1} \cdot 2^8 + \underline{0} \cdot 2^7 + \underline{1} \cdot 2^6 + \underline{1} \cdot 2^5 + \underline{0} \cdot 2^4 + \underline{1} \cdot 2^3 + \underline{1} \cdot 2^2 + \underline{0} \cdot 2^1 + \underline{1} \cdot 2^0$$

- Računarski procesor obavlja izračunavanja u binarnom brojnem sistemu, koristeći logička kola; tako je na primer u binarnom brojnem sistemu $1+1=10$; podaci se u računarskoj

memoriji predstavljaju takođe u binarnom brojnom sistemu; svaka elementarna ćelija memorije je jedan nano-kondenzator: on može biti pun (1) ili prazan (0)

- Još jedan brojni sistem koji se koristi u nekim računarskim domenima je heksadecimalni brojni sistem; to je sistem čija je osnova 16; cifre u ovom sistemu su od 0 do 9, i dalje A (množi 16 na odgovarajući eksponent faktorom 10), B (faktorom 11), C (faktorom 12), D (faktorom 13), E (faktorom 14) i F (faktorom 15)

- Tako na primer, broj dana u godini zapisan u heksadecimalnom brojnom sistemu je 16D, jer (u decimalnom brojnom sistemu) važi jednakost:

$$365 = 1 \cdot 256 + 6 \cdot 16 + 13 \cdot 1 = \underline{1} \cdot 16^2 + \underline{6} \cdot 16^1 + \underline{13} \cdot 16^0$$

- Konverzija iz nekog ne-decimalnog brojnog sistema u decimalni se obavlja kako je već prikazano u gornjim primerima; još dva primera:

- 10100001 binarno jednako je $1 \cdot 2^7 + 1 \cdot 2^5 + 1 \cdot 2^0 = 161$ decimalno (izostavljene su nule kao činioci)

- A1 heksadecimalno jednako je $10 \cdot 16^1 + 1 \cdot 16^0 = 161$ decimalno

- Obrnuto, konverzija iz decimalnog brojnog sistema u sistem sa osnovom N (različit od 10) se obavlja na sledeći način:

- 1) broj se deli sa N; piše se ostatak pri deljenju

- 2) celobrojni deo količnika iz prethodnog deljenja se deli sa N; piše se ostatak pri deljenju

- tačka 2) se ponavlja sve dok celobrojni deo količnika ne postane 0

- ostaci zapisani u obrnutom redosledu od onih kojim su dobijani daju traženi broj

- Primer 1: konvertovati decimalni broj 161 u binarni sistem

161/2: ostatak 1; 80/2: ostatak 0; 40/2: ostatak 0; 20/2: ostatak 0; 10/2: ostatak 0; 5/2: ostatak 1; 2/2: ostatak 0; 1/2: ostatak 1, što daje binarni broj 10100001

- Primer 2: konvertovati decimalni broj 161 u heksadecimalni sistem

161/16: ostatak 1; 10/16: ostatak 10, to jest A, što daje heksadecimalni broj A1

0.3 O predmetu Uvod u programiranje

- Predmet Uvod u programiranje (šifra: UPR) izučava se na primeru modernog programskog jezika Python; biće obrađivan samo njegov ne-OO (objektno-orijentisani) deo; OO programiranje, koje se zasniva na konceptu koji se naziva *klasa*, će se izučavati u drugom semestru, na predmetu Objektno orijentisano programiranje

- Predmet je obavezan u prvom semestru za studente IT i nosi 9 ESP bodova

- Predmet po poglavljima:

1. Uvod
2. Tipovi, promenljive i jednostavni ulaz-izlaz
3. Grananje, while-petlja, planiranje programa
4. For petlja, stringovi i n-torke
5. Liste i rečnici
6. Funkcije
7. Fajlovi i izuzeci

- Literatura za predmet je knjiga: Michael Dawson "Python -- Uvod u programiranje" (Mikroknjiga); predmet UPR redom pokriva prvih 7 poglavlja iz te knjige; materija u knjizi je izložena na primeru igara, kako bi se na zabavan način prihvatili složeni koncepti



- Na drugim školama se kao prvi jezik često izučava C; međutim taj programski jezik gubi značaj jer su mnogi njegovi koncepti zastareli
- Predavanja: Prof. Dr Miroslav Marković; 2 časa nedeljno; na stranici predmeta nalazi se skripta sa predavanja u obliku 7 pdf dokumenata, za svako poglavlje po jedan
- Zašto skripta ako postoji knjiga? Zato što je prevod knjige na srpski jezik često veoma loš; u skripti su na nekoliko mesta u odnosu na knjigu dodate važne pojedinosti; smatraće se zato da je **skripta glavna literatura**, a da je knjiga (čija se kupovina svakako preporučuje) pomoćna, koja skoro sasvim prati glavnu
- Studenti treba da tokom predavanja imaju ispred sebe materijal: skripta štampana ili skripta na ekranu ili knjiga, radi lakšeg praćenja
- Vežbe: 2 časa nedeljno u računarskoj učionici, po grupama; na stranici predmeta se nalazi skripta sa vežbi (zbirka rešenih zadataka), u obliku 7 pdf dokumenata, za svako poglavlje po jedan, kao i programski kodovi koji prate primere
- Uz predavanja idu i dve arhive: `py3e_software.zip` i `py3e_source.zip`, koje se preuzimaju na adresi:

<http://www.delmarlearning.com/companions/content/1435455002/downloads/index.asp?isbn=1435455002>

a takođe su postavljene i na Stranicu predmeta

- Semestar traje 15 sedmica, od kojih će se nastava održavati tokom 12; tri sedmice su kolokvijumske -- tokom njih nema ni predavanja ni vežbi
- Prisustvo na predavanjima i vežbama nije obavezno; sav materijal potreban za polaganje ispita nalazi se na stranici predmeta tako da studenti ništa formalno ne propuštaju ako ne dolaze na nastavu

Predispitne obaveze

- Prvi kolokvijum se održava u toku sedmice 8 (u terminu predavanja), i pokriva gradivo iz poglavlja 1, 2, 3 i 4. Drugi kolokvijum se održava u toku sedmice 14 (u terminu predavanja), i pokriva gradivo iz poglavlja 5, 6 i 7.
- Na kolokvijumima se radi po nekoliko teorijskih pitanja i zadataka, na papiru. Svaki od dva kolokvijuma nosi po 30 poena. To daje ukupno **60** poena sa predispitnih obaveza. Uslov za izlazak na ispit (to jest dobijanje potpisa) je po 15 poena i više na oba kolokvijuma.
- U toku sedmice 15 (u terminu predavanja), organizuje se Popravni kolokvijum. Na njega mogu izaći studenti koji nisu izašli na neki od dva kolokvijuma, ili koji nisu zadovoljni brojem poena. Na Popravnom kolokvijumu student može izaći samo na jedan deo.

Ispit

- Ispit se radi na računaru i nosi **40** poena.

Ocenjivanje

- Ocena O se prema broju poena P daje na ustaljeni način:

$51 \leq P \leq 60$: O=6

$61 \leq P \leq 70$: O=7

$71 \leq P \leq 80$: O=8

$81 \leq P \leq 90$: O=9

$91 \leq P \leq 100$: O=10

- Solidno znanje engleskog jezika se podrazumeva; ukoliko neki pojam nije jasan, prevesti ga pomoću rečnika

Poglavlje 1: Uvod

- Programiranje se sastoji u tome da računar uradi ono što mu se zada
- Za učenje programiranja, programski jezik Python je optimalan izbor -- razlozi za ovu tvrdnju biće objašnjeni kasnije
- U poglavlju će biti pokazano:
 - šta je Python i po čemu je izuzetan
 - kako se instalira
 - kako se tekst ispisuje na ekranu
 - kako se upisuju komentari
 - kako se programi pišu, menjaju, izvršavaju i snimaju na disk

1.1 Program Game Over

- Slika 1.1 prikazuje prozor konzole ("console window") programa Game Over



- U prozoru konzole se može prikazivati samo tekst u primitivnoj formi; ovi prozori nisu dopadljivi kao oni u grafičkom korisničkom interfejsu ("graphical user interface", GUI), ali se aplikacije u njima lakše pišu i dobri su za početnike
- Program Game Over je sasvim jednostavan, ali zahteva kompletnu proceduru koja je neophodna da bi se programiralo: instaliranje Pythona, pisanje ("writing") programa, snimanje ("saving") na disk, i izvršenje ("running")

- Uobičajeno je da se ovaj najjednostavniji program naziva "Hello World", ali pošto će ovde sve biti postavljeno u kontekst igara, tekst poruke je promenjen

1.2 O programskom jeziku Python

- Programski jezik Python je moćan ali jednostavan za korišćenje; kreirao ga je Guido van Rossum; prva verzija je objavljena 1991. (v2.0 2000, v3.0 2008, aktuelna v3.13)



- Python omogućava da se brzo napiše jednostavan projekat (kompletan program koji se instalira i obavlja određenu funkciju); ali istovremeno omogućava da se napišu složene komercijalne aplikacije
- Naziv je dobio prema komičarskoj grupi Monti Pajton; zvanična maskota jezika je zmija piton; ceo jezik nosi sa sobom i određenu programersku podkulturu ("Lepo je bolje nego ružno", a van Rossum je proglašen za "dobrovoljnog doživotnog diktatora" cele Python zajednice)
- Python ima i svoje "zen principe":

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.
- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one- and preferably only one -obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.
- Although never is often better than **right** now.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.
- Namespaces are one honking great idea -- let's do more of those!

- Prednosti Pythona u odnosu na druge jezike navedene su u narednim tačkama

Python se lako koristi

- Najvažniji zadatak svakog programskog jezika: premostiti jaz između programera (koji funkcioniše na ljudskom jeziku) i računara (koji funkcioniše na mašinskom jeziku); većina popularnih jezika (C#, Java) su visokog nivoa (to jest bliski umu), ali Python je i od njih viši, to jest još bliži ljudskom jeziku
- Pisanje programa na Pythonu, sa njegovim jasnim i prostim pravilima, je toliko jednostavno da je nazvano "programiranje brzinom razmišljanja"
- Kao posledica navedenog, programerska produktivnost je povećana; napisani programi su kraći i brže se pišu

Python je moćan

- Python nudi sve što se očekuje od moćnog programskog jezika; koriste ga najveće svetske kompanije (Google, IBM, Microsoft, NASA), i najbolji programeri

Python je objektno orijentisan (OO)

- OO programiranje (OOP) je moderan pristup programiranju -- ono na intuitivan način organizuje podatke i akcije koje program koristi i preduzima; ovo nije jedini način pisanja programa, ali je često najbolji (posebno za velike projekte)
- Python je OO jezik, kao i C# i Java, ali:
 - u C# i Javi, OOP je obavezno; zbog toga su kratki programi nepotrebno složeni, pa programer treba da ima dosta znanja da bi započeo svoj rad
 - u Pythonu, OOP je opciono, to jest koristi se po želji/potrebi; jednostavan projekat -- bez OOP; složen projekat -- sa OOP

Python je "lepljiv" jezik

- Python se može integrisati sa drugim jezicima (C, C#, Java) -- Python programer može iskoristiti rezultate programa koji su napisani u drugim jezicima; takođe, on može koristiti prednosti drugih jezika (brzina u C, C++)

Python radi svuda

- Programi napisani u Pythonu se izvršavaju na svim platformama, od smart uređaja do superkompjuteru, na svim operativnim sistemima

- Python programi su nezavisni od platforme ("platform-independent") -- nezavisno od toga pod kojim OS (Windows, Linux, MacOS...) se kreira Python program, on će moći da se izvršava na svakom drugom računaru na kom je instaliran Python

Python ima ogromnu zajednicu korisnika

- Većina programskih jezika ima svoje newsgroup, ali Python ima nešto što se zove Python Tutor mejling lista, na kojoj na manje formalan način početnici (i ne samo oni) mogu postavljati pitanja, na koja iskusnije kolege odgovaraju:
<http://mail.python.org/mailman/listinfo/tutor>

Python je besplatan i otvorenog koda

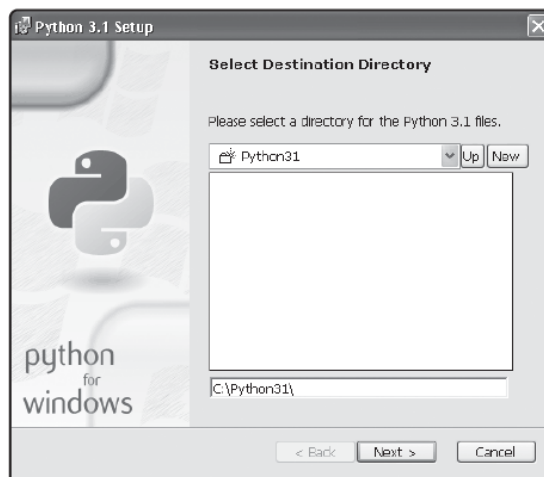
- Python je sasvim besplatan; ali njegova licenca pruža još mnogo toga -- korisnici ga mogu kopirati i modifikovati

Python je jezik mašinskog učenja

- Python je najpopularniji programski jezik za mašinsko učenje (Machine Learning, jednu posebnu oblast veštačke inteligencije) zbog velikog broja raspoloživih biblioteka namenjenih toj oblasti (Scikit-learn, Theano, TensorFlow, Keras, PyTorch)

1.3 Instaliranje Pythona na Windows

- Preuzme se Python Windows instaler iz arhive `py3e_software.zip`, folder "python":
`python-3.1.1.msi`
- Izvrši se instaler (Slika 1.2), prihvati se default konfiguracija



1.4 Instaliranje Pythona na druge OS

(ovo se neće raditi)

1.5 Uvod u IDLE

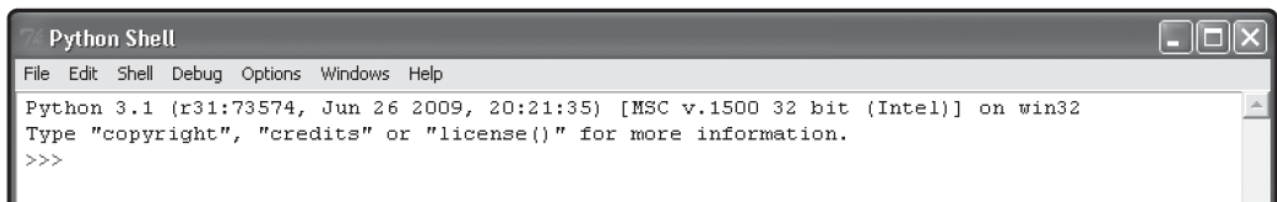
- Python sadrži i integrisano razvojno okruženje ("Integrated DeveLopment Environment" = IDLE; na engleskom idle=besposlen) -- radi se o skupu alata koje olakšavaju pisanje programa (kao program za obradu teksta, samo ne služi za tekstove nego za programe)
- Pored toga što se u IDLE programi pišu, snimaju i menjaju (edituju), u njemu postoje i dva radna režima (moda): interaktivni režim i skript režim

Programiranje u interaktivnom režimu

- Ovaj režim služi da se brzo počne sa pisanjem programa -- korisnik naredi Pythonu šta da uradi, i ovaj trenutno reaguje; zato se zove interaktivni režim -- javlja se interakcija između korisnika i Pythona, komandu po komandu

Pisanje prvog programa

- Pokretanje: Start >> All Programs >> Python 3.1 >> IDLE, čime se otvara novi prozor Python Shell



- Komandna linija je označena sa >>>; ako se upiše print("Game Over") i pritisne Enter, na ekranu se pojavi tekst Game Over

Upotreba funkcije print

- Kao što se da zaključiti, funkcija print() ispisuje tekst koji se (između zagrada) zada pod navodnicima; ako između zagrada ne stoji ništa, ispisaće se prazan red
- Važno je sledeće: Python pravi razliku između malih i velikih slova ("case-sensitive"); po pravilu, imena funkcija (kao što je print) se pišu svim malim slovima; ako napišemo Print ili PRINT, rezultata neće biti

Učenje žargona

- Tokom predavanja, postepeno će se uvoditi razni stručni termini koji čine "žargon" jezika Python; na primer, *funkcija* je u stvari mini-program koji se izvršava u pozadini (njegov kod možemo i da ne vidimo) i koji obavlja određeni zadatak; primer: zadatak koga obavlja funkcija print() je da na ekranu prikaže neku *vrednost* ("value")
- Žargon vezan za funkcije:
 - funkcija se *poziva* ("call") pomoću njenog imena posle koga slede zagrade
 - prilikom poziva nekih funkcija, njima se mogu *zadati* ("pass") neke ulazne vrednosti koje su toj funkciji potrebne za izvršenje

- te ulazne vrednosti nazivamo *argumenti* ("arguments") date funkcije, i pišemo ih u zagradama, iza imena funkcije

- U prethodnom primeru, funkciji `print()` je zadat argument "Game Over" u obliku tekst stringa, koga je funkcija ispisala na ekranu; drugim rečima, funkcija `print()` obavlja zadatak ispisivanja na ekranu argumenta koji joj je zadat

- Neke funkcije mogu i da vrate neku vrednost (ili više vrednosti) onom programu koji ih je pozvao; ove vrednosti se nazivaju *povratne vrednosti* ("return values"); primer -- ako napišemo funkciju `saberi()` za sabiranje dva broja, argumenti su dva sabirka a povratna vrednost je zbir

- U navedenom primeru, pomenuto je da je vrednost "Game Over", koja je zadata funkciji `print()`, *string* ("string") -- koristi se i pojam *znakovni niz*; to je niz (povorka) određenog broja alfanumeričkih znakova (karaktera) u nizu, i po pravilu se stavlja između zagrada

- Ceo tekst koji je unet na komandnoj liniji naziva se *naredba* ("statement") -- on predstavlja kompletnu instrukciju koja obavlja neki konkretni zadatak; svaki program se sastoji od određenog broja naredbi koje se nalaze jedna ispod druge i izvršavaju jedna za drugom

- Konačno, programski *kod* ("code") je skup naredbi, to jest precizniji naziv za ispisani program (pod programom se smatra i nešto što je na primer kupljeno); takođe se koristi glagol *kodirati* koji znači ispisivati programski kod, što je preciznije od *programirati* (programirati se može i na primer pomoću GUI)

Greške u kodu

- Računar sve razume doslovno; ako se umesto print otkuca print, izaći će sledeća poruka koja kaže da pozivamo funkciju print koju nismo definisali:

```
Traceback (most recent call last):  
  File "<pyshell#0>", line 1, in <module>  
    print("Game Over")  
NameError: name 'print' is not defined
```

- Ovo je primer programske greške, koja se u žargonu naziva *bag* ("bug") -- ova je jednostavna i lako se otkloni, dok druge ne moraju da budu

Isticanje sintakse

- Tokom programiranja može se primetiti da se delovi koda prikazuju na ekranu različitim bojama; ovo svojstvo Pythona se naziva *isticanje sintakse* ("syntax highlighting"); ovo se radi u cilju bolje preglednosti

- Pravila:

specijalne reči koje su deo Pythona (kao što su funkcije) se označavaju ljubičastom bojom

stringovi (kao što je "Game Over") se označavaju zelenom

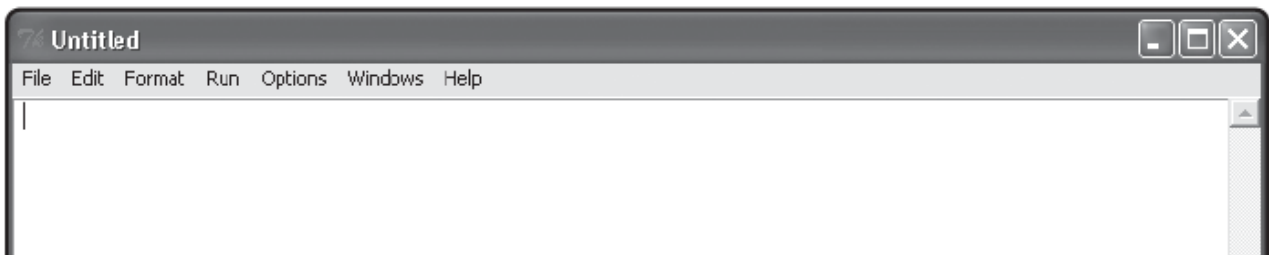
izlaz (rezultat onoga što je zadato) označava se plavom

Programiranje u skript režimu

- Pri radu u interaktivnom režimu, odziv je trenutni: zada se naredba i odmah vidi njen rezultat; ali programi se uvek pišu tako da se moraju sačuvati i koristiti kasnije; za to služi skript režim, u kome se programi mogu pisati, editovati, učitavati i snimati; to je pravi tekst procesor za kod, koji sadrži i dodatne funkcije tipa Find, Replace, Cut, Paste

Pisanje prvog programa (ponovo)

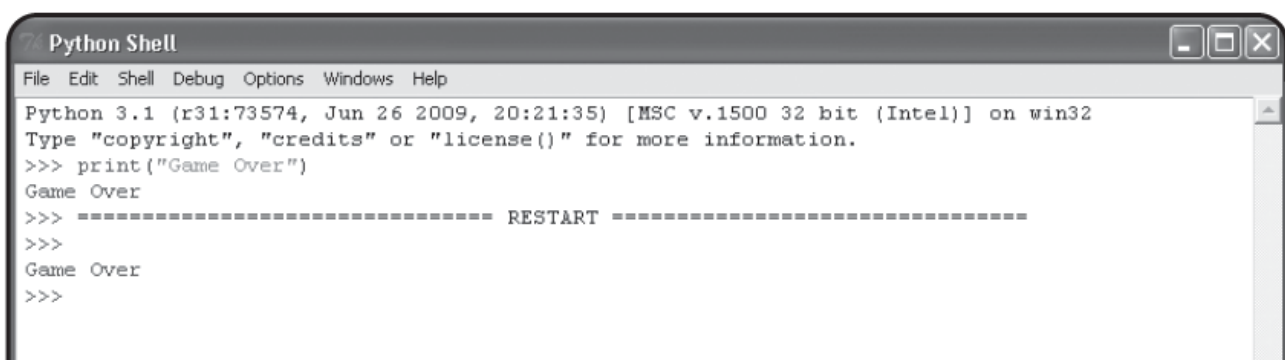
- Skript režim se pokreće iz prozora u interaktivnom režimu: File >> New Window, čime se otvara novi blanko prozor (Untitled) -- ovaj prozor sadrži poznate stavke u meniju: File, Edit, Format ali i nešto novo tipa Run



- U ovom režimu, piše se lista (sekvenca) naredbi jedna ispod druge, koje će se kasnije izvršavati jedna posle druge; da bi se to uradilo, program prvo treba da se snimi/sačuva ("Save") kao fajl na disk, a zatim izvrši ("Run")

Snimanje i izvršavanje programa

- Pošto se ceo program ispiše, bira se File >> Save As; zadaje se ime game_over.py (svi Python programi moraju imati ekstenziju py); zatim se pokreće izvršenje: Run >> Run Module
- Prilikom izvršavanja programa, u interaktivnom prozoru (Python Shell) se prikazuju rezultati



- Primećuje se da u prozoru ostaje i stari tekst (iz interaktivnog moda); zatim se vidi tekst RESTART koji je nastao odmah pri pokretanju programa, i zatim se vidi rezultat programa

- Preporuka je da se radi na sledeći način:
 - za pisanje programa koristiti skript režim
 - interaktivni prozor držati otvorenim: on pomaže da se proba neka ideja, ili proveriti da li je neka funkcija ispravno pozvana (to jest: u skriptu se pravi finalni proizvod; interaktivni služi za isprobavanje i testiranje)

1.6 Povratak na program Game Over

- U prethodnoj sekciji, program Game Over je izvršavan kroz IDLE; u fazi kreiranja (pisanja, razvijanja) programa, IDLE je od velike koristi
- Međutim, cilj programera je da se njegov program izvrši na isti način kao što se izvršavaju drugi programi, to jest duplim klikom miša na ikonu; ako se duplo klikne na ikonu našeg programa, sve se odvija toliko brzo da se i ne primeti: otvori se prozor konzole, ispiše se tekst, i odmah se prozor zatvori
- Da bi prozor konzole ostao otvoren posle ispisivanja teksta, program se mora korigovati tako da odradi sledeće: otvori se prozor, ispiše tekst, posle koga se ispiše poruka "Press the enter key to exit" (pritisni taster enter da bi izašao), pa tek kada korisnik pritisne Enter, prozor se zatvori

- Modifikovani kod je:

```
# Game Over
# Demonstrates the print function

print("Game Over")

input("\n\nPress the enter key to exit.")
```

Komentari

- Linije koda koji počinju sa # nisu naredbe -- njih računar kompletno ignoriše; zovu se komentari, i namenjene su ljudima koji kod pišu ili čitaju; korisni su drugima (koji sa programom treba da rade, na primer da ga modifikuju) ali i samom autoru da se podseti (jer i sam može vrlo brzo da zaboravi šta radi koja linija)
- Formalno., sve što se nalazi posle znaka # smatra se komentarom; u IDLE se prikazuju crvenom bojom da bi se lakše uočili
- Dobra programerska praksa je da:
 - svaki kod počne sa nekoliko linija komentara; on treba da sadrži naslov, namenu programa, ime programera i datum
 - svaka linija koda sadrži komentar

Prazni redovi

- Prazni redovi služe da program bude lakše čitljiv/pregledan; tako se na primer odvajaju nezavisni blokovi unutar koda (na primer, jedna petlja)

Prikazivanje stringa

- String se prikazuje pomoću funkcije print() kao što je već pokazano

Čekanje na odgovor korisnika

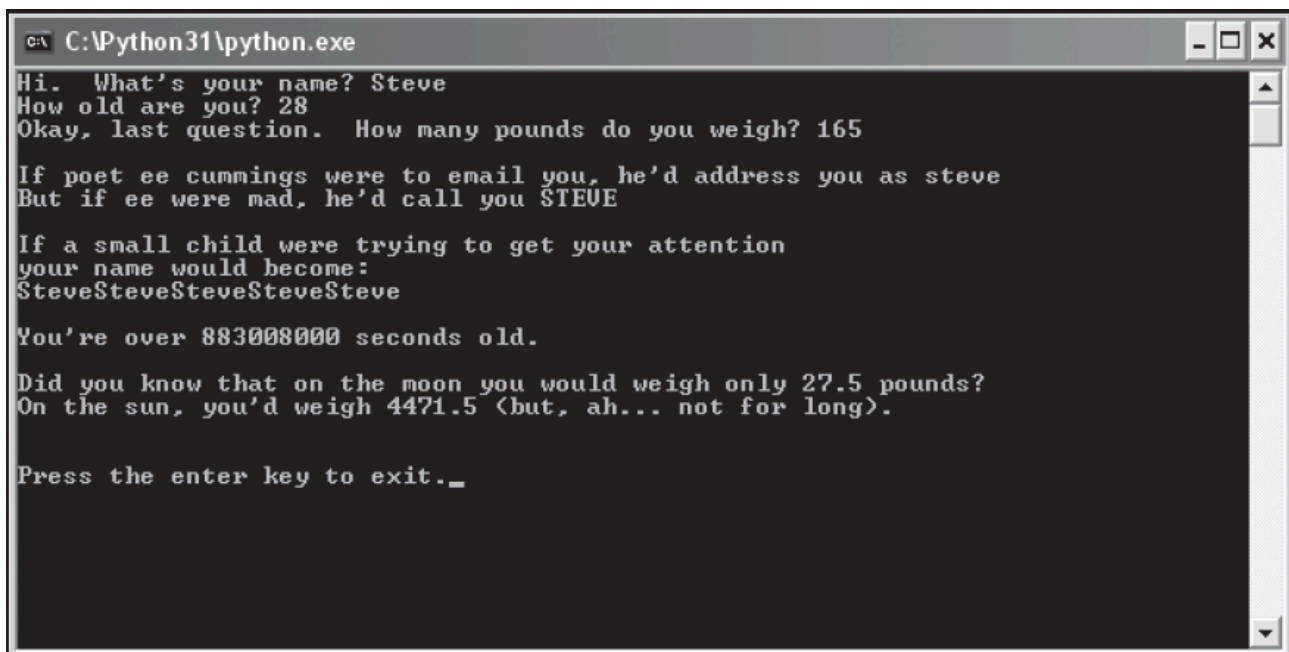
- Poslednja linija u kodu prikazuje poruku za korisnika i čeka da on pritisne Enter, čime se program završava i konzolni prozor zatvara; funkcija input() biće objašnjena u narednim poglavljima

Poglavlje 2: Tipovi, promenljive i jednostavni ulaz-izlaz

- Ovo poglavlje predstavlja razne načine pomoću kojih računar može da kategorizuje i čuva podatke, i kako se ti podaci koriste u programima; takođe, predstaviće i kako od korisnika dobiti/preuzeti podatke tako da program postane interaktivan (to jest da korisnik zadaje neke ulazne vrednosti za program)
- U poglavlju će biti pokazano:
 - kako koristiti stringove sa trostrukim navodnicima da se poveća kontrola nad tekstem
 - kako da program izvršava matematička izračunavanja
 - kako da program čuva podatke u memoriji
 - kako koristiti varijable (promenljive) da se vrši pristup i obrada podataka
 - kako preuzeti podatke od korisnika tako da se kreira interaktivni program

2.1 Program Useless Trivia

- Kombinovanjem znanja stečenih u ovom poglavlju, na njegovom kraju biće prikazan program Useless Trivia (Beskorisne sitnice), Slika 2-1



```
C:\Python31\python.exe
Hi. What's your name? Steve
How old are you? 28
Okay, last question. How many pounds do you weigh? 165

If poet ee cummings were to email you, he'd address you as steve
But if ee were mad, he'd call you STEVE

If a small child were trying to get your attention
your name would become:
SteveSteveSteveSteveSteve

You're over 883008000 seconds old.

Did you know that on the moon you would weigh only 27.5 pounds?
On the sun, you'd weigh 4471.5 (but, ah... not for long).

Press the enter key to exit._
```

- Program traži od korisnika: ime, starost i težinu (u pounds, oko 0.5 kg); na osnovu tih podataka on daje neke (trivijalne) činjenice o korisniku

2.2 Upotreba navodnika sa stringovima

- Već je pokazano šta je string i kako se označava navodnicima; dat je "Game Over" kao primer; često postoji potreba da stringovi budu komplikovaniji nego u ovom primeru; mogu sadržati više pasusa teksta; za označavanje stringova koriste se, kao što je prikazano, navodnici ("..."), ali mogu i polunavodnici ('...')
- Engleski nazivi: polunavodnik = single quote, navodnik = double quote; uvek uz korišćenje množine: navodnici = quotes

Program Game Over 2.0

- Verzija 2.0 programa poboljšava prethodnu verziju u smislu da izbacuje istu poruku samo napisanu ogromnim slovima (Slika 2-2)



- Programski kod game_over2.py:

```

# Game Over – Version 2
# Demonstrates the use of quotes in strings

print("Program 'Game Over' 2.0")

print("Same", "message", "as before")

print("Just",
      "a bit",
      "bigger")

print("Here", end=" ")
print("it is...")

print(
    """
    GAME
    OVER
    """
)

input("\n\nPress the enter key to exit.")

```

Upotreba navodnika u stringovima

- Već je pokazano kako se pravi string uz korišćenje navodnika; kao što je pomenuto: string se može označiti (formirati) pomoću para polunavodnika ('tekst') ili para navodnika ("tekst") -- računaru je svejedno; znači moglo je biti napisano i 'Game Over'
- Takođe, sasvim je logično da sami (polu)navodnici pomoću kojih se string označava (formira) ne pripadaju samom stringu, već samo označavaju (služe kao "graničnici") gde mu je kraj a gde početak; drugim rečima, karakter koji sledi iza prvog "graničnika" je prvi karakter stringa; karakter koji prethodi drugom "graničniku" je poslednji

- Tako, kada se izvrši linija programa:

```
print("Program 'Game Over' 2.0")
```

na ekranu se ispiše tekst:

Program 'Game Over' 2.0

naravno bez navodnika (jer oni nisu deo stringa), a sa polunavodnicima (koji jesu, kao na primer i slovo G)

- Važno je sledeće: kada string označimo parom navodnika, unutar tog stringa može se nalaziti neograničeno mnogo polunavodnika; takođe važi i suprotno

- Sa druge strane, kada string označimo parom navodnika, (druge) navodnike ne možemo uključiti u sam string; to je sasvim logično, jer kada računar naiđe na drugi navodnik po redu, on će smatrati da je njime označen kraj stringa (to jest da je stigao do drugog "graničnika"); isto važi i za polunavodnike

- Primer: da je linija programa

```
print("Program "Game Over" 2.0")
```

računar bi smatrao da je string "Program ", i pošto mu neće biti jasno šta znači Game Over, poslaće poruku o grešci

- Na ovom predmetu, odsad će se po konvenciji za označavanje stringova koristiti navodnici (i samim tim stringovi ne mogu sadržati druge navodnike)

Prikazivanje više stringova pomoću jedne naredbe

- Pomoću funkcije print() može se istovremeno prikazati na ekranu više stringova (ili nekih drugih vrednosti) -- potrebno ih je samo razdvojiti zarezima ("comma")

- Primer:

```
print("Same", "message", "as before")
```

prikazuje tri stringa jedan za drugim, uz obavezno ubacivanje po jednog razmaka posle svakog stringa:

Same message as before

- Pri zadavanju liste stringova, može se otpočeti novi red posle svakog zareza -- rezultat će biti isti, samo što čitljivost/preglednost koda može biti poboljšana; primer:

```
print("Just",  
      "a bit",  
      "bigger")
```

izbacuje tekst:

Just a bit bigger

Zadavanje završnog karaktera u stringu

- Kao što je već pokazano, funkcija print() prikazuje na ekranu ceo zadati string, to jest sve njegove karaktere u nizu; ova funkcija međutim (bez da joj se to zada), posle svih karaktera stringa prikazuje na ekranu još jedan karakter, koji se naziva završni ("end character")
- Po difoltu (podrazumevano, sve dok se eksplicitno ne zada drugačije = "default"), funkcija print() "podrazumeva" da je taj završni karakter jednak newline karakteru
- Newline je "karakter" (navodnici označavaju da se ne radi o karakteru u užem smislu, kao što su cifre, brojevi i specijalni znaci) koji simbolizuje novu liniju -- kada se taj karakter "ispiše" na ekranu, naredno ispisivanje počinje u novoj liniji na ekranu
- Ovo jednostavno znači da će, prilikom narednog poziva funkcije print(), ono što se u okviru nje ispisuje biti ispisano u novoj liniji, i tako redom
- Newline karakter je jedan od specijalnih karaktera (onih koji nisu karakteri u užem smislu); ti karakteri se označavaju pomoću znaka \ ("beksleš"), posle koga ide određeno slovo: u ovom slučaju, newline karakter se označava sa \n; više detalja u nastavku poglavlja

- Primer:

```
print ("First line \n Second line")
```

na ekranu ispisuje:

First line

Second line

(obratiti pažnju na razmak koji ide ispred S)

- Može se zamisliti i ovako: print() funkcija kao svoj prvi argument vidi zadati string, zatim ide zarez, pa onda kao drugi argument vidi string koga čini završni karakter; kao što je prikazano ranije, kada su dva argumenta (stringa) odvojena zarezom oni se prikazuju jedan za drugim, uz razmak posle prvog

- Tako na primer, linija koda

```
print("Program 'Game Over' 2.0")
```

će ispisati ceo zadati string (karakter po karakter, od P do 0); zatim će dodati razmak (podrazumevajući nenavedeni zarez), pa će ispisati završni string (to jest prebaciti u novu liniju)

- Ovaj difolt prelazak na novu liniju prilikom ispisivanja na ekranu je obično poželjan, ali u slučaju potrebe programer može da promeni završni karakter, da više nema difolt vrednost (newline karakter)

- Na primer, programer može da zada da print() funkcija kao završni karakter ispisuje razmak; to znači da sledeća print() funkcija počinje prikazivanje teksta odmah posle tog dodatog razmaka (umesto u novom redu)

- Ovo se zadaje na primer ovako:

```
print("Here", end=" ")  
print("it is...")
```

čime se ispisuje tekst u jednoj liniji:

Here it is...

- U ovom primeru:
 - prvo se ispisuje tekst **Here**
 - zatim se ispisuje razmak (zbog zareza)
 - zatim se računaru, pomoću koda `end=" "`, stavlja do znanja da završni karakter (umesto newline) postaje razmak ("space")
 - napomena: ovaj razmak se neće duplirati jer već postoji zbog zareza
 - zatim se posle tog (jednog) razmaka nastavlja string od naredne `print()` funkcije, **it is...**
- `end` je parametar funkcije `print()`; šta parametar tačno znači, biće pokazano u Poglavlju 6

Stringovi sa trostrukim navodnicima `"""`

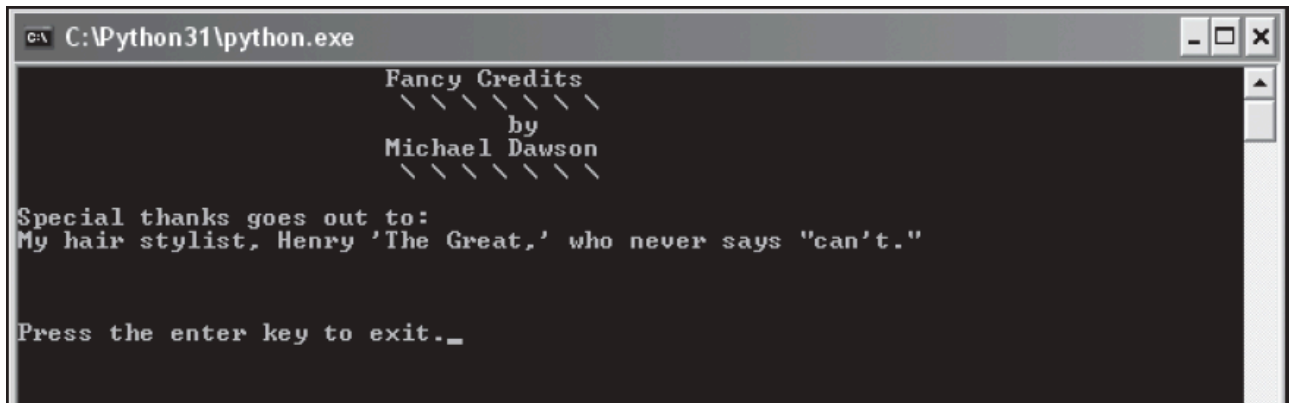
- U navedenom primeru, ispisivanje velikih slova je ostvareno pomoću linije `print("""...""")`
-- ovo se naziva *string sa trostrukim navodnicima* ("triple-quoted string"); to je string ograničen sa obe strane sa `"""` (mogu se koristiti i trostruki polunavodnici ali nije preporučljivo)
- Ovi stringovi mogu da zauzimaju više linija; njihova karakteristika je da na ekranu prikazuju tačno onako kako su prikazani u kodu

2.3 Upotreba izlaznih sekvenci u stringovima

- Izlazne sekvence ("escape sequences") omogućavaju umetanje specijalnih karaktera u stringove, i time omogućavaju veću kontrolu i fleksibilnost pri ispisivanju teksta
- Izlaznu sekvencu čine dva karaktera: beksleš ("backslash", obrnuta kosa crta, `\`) za kojim sledi drugi karakter

Program Fancy Credits

- Objašnjenje: "credits" znači zahvaljivanje nekome na početku ili kraju knjige ili igre, odnosno navodi imena osobe koje su zaslužne za njen nastanak; ovaj program će, osim obaveštenja da je igra završena, navesti i imena ljudi kojima se autor zahvaljuje (frizeru) -- to se ostvaruje pomoću izlaznih sekvenci (Slika 2-3)



- Kod programa fancy_credits.py:

```
# Fancy Credits
# Demonstrates escape sequences

print("\t\t\tFancy Credits")

print("\t\t\t \ \ \ \ \ \ \ \ \")
print("\t\t\t\tby")
print("\t\t\tMichael Dawson")
print("\t\t\t \ \ \ \ \ \ \ \ \")

print("\nSpecial thanks goes out to:")
print("My hair stylist, Henry \'The Great,\' who never says \"can\'t.\"")

# sound the system bell
print("\a")

input("\n\nPress the enter key to exit.")
```

Pomeranje teksta pomoću tabulatora

- Da bi se početak ispisivanja stringa (teksta) pomerio (uvukao) od leve margine, koristi se izlazna sekvenca koja predstavlja tabulator (karakter): \t; to je urađeno u liniji:

```
print("\t\t\tFancy Credits")
```

- Ovde je to urađeno tri puta uzastopno; "ispisivanje" sekvence (karaktera) `\t` znači pomeranje početka ispisivanja teksta za neko rastojanje na ekranu; ovim se napravilo pomeranje za tri takva rastojanja, posle čega je ispisan tekst `Fancy Credits`

Štampanje beksleša

- Postavlja se pitanje: kako na ekranu, u okviru nekog stringa, prikazati beksleš, kada će računar uvek interpretirati beksleš u stringu kao početni karakter izlazne sekvence? Rešenje je vrlo jednostavno: pišu se dva beksleša jedan za drugim: `\\`

- Znači, izlazna sekvenca `\\` predstavlja beksleš karakter; tako linija

```
print("\t\t\t \\ \\ \\ \\ \\ \\ \\")
```

prikazuje tri taba za kojima sledi 7 bekslešova (kao rezultat 7 sekvenci `\\` odvojenih razmacima)

Prelazak u novu liniju (red)

- Da bi se ispisivanje teksta prebacilo u novi red (liniju), koristi se (kao što je već pokazano) izlazna sekvenca `\n`; kada se ona nađe u stringu, automatski se prelazi u naredni red; ako se stavi kao prvi karakter u stringu, ispisuje se prazan red

- Navedeno je urađeno pomoću linije:

```
print("\nSpecial thanks goes out to:")
```

Znači, `print()` funkcija je, samim svojim početkom izvršenja, započela ispisivanje u novom redu (u odnosu na prethodni), a pri nailasku na `\n`, pomerila je ispisivanje za još jedan red, čime je napravila prazan red

Umetanje navodnika

- Umetanje (polu)navodnika u string, kada je sam string zadat bilo pomoću navodnika bilo pomoću polunavodnika, je sasvim jednostavno: koristi se izlazna sekvenca \' za polunavodnik, i \" za navodnik

- Ovo znači: "ovde stavi navodnik" (ili polunavodnik); zato ne postoji nikakav rizik da se napravi zabuna, to jest da računar pomeša sam karakter u stringu sa "graničnikom" za string

- Ovo je napravljeno u liniji:

```
print("My hair stylist, Henry \'The Great,\' who never says \"can\'t.\\\"")
```

koja proizvodi tekst:

My hair stylist, Henry 'The Great,' who never says "can't."

Aktiviranje zvučnog signala

- Da bi se aktivirao zvučni signal (sistemske zvonce na računaru), funkcija print() treba da "ispiše" karakter ("system bell character") koji odgovara izlaznoj sekvenci \a; to radi linija:

```
print("\a")
```

s tim što je moguće i ponavljanje ove sekvence da bi se kreiralo nekoliko uzastopnih signala

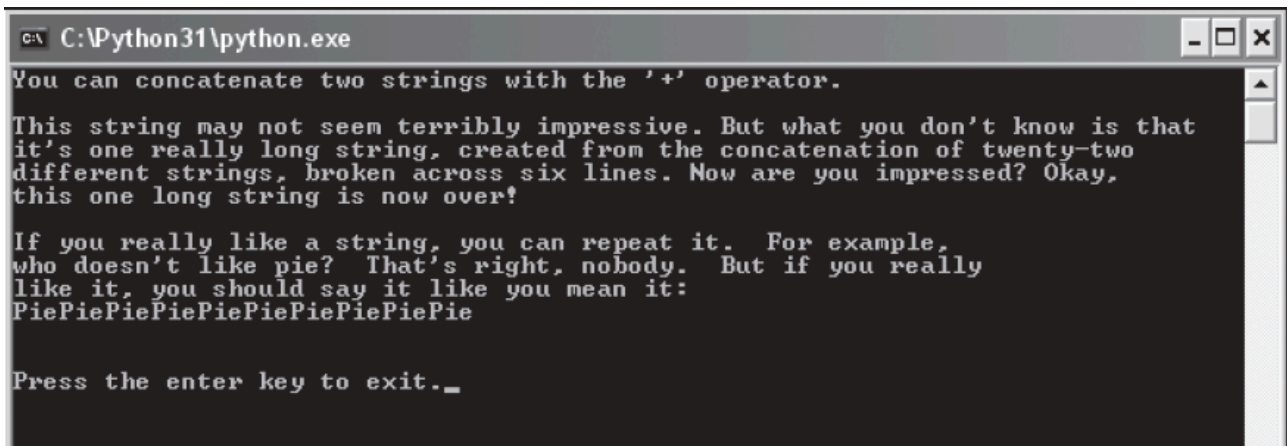
- Treba napomenuti da neke izlazne sekvence (kao što su one sistemske, na primer pomenuta \a), rade ako se startuju iz operativnog sistema (duplim klikom na ikonu fajla), a ne rade ako se startuju iz IDLE

2.4 Spajanje i ponavljanje stringova

- Pokazano je kako se u string umeću razni karakteri; ova sekcija pokazuje šta se može raditi sa samim stringovima -- kako se njih više može kombinovati u jedan, i kako se jedan može ponavljati više puta

Program Silly Strings

- Program Silly Strings daje sledeći rezultat na ekranu (Slika 2-4):



```

C:\Python31\python.exe
You can concatenate two strings with the '+' operator.

This string may not seem terribly impressive. But what you don't know is that
it's one really long string, created from the concatenation of twenty-two
different strings, broken across six lines. Now are you impressed? Okay,
this one long string is now over!

If you really like a string, you can repeat it. For example,
who doesn't like pie? That's right, nobody. But if you really
like it, you should say it like you mean it:
PiePiePiePiePiePiePiePiePiePiePiePie

Press the enter key to exit._

```

- Odgovarajući kod silly_strings.py je:

```

# Silly Strings
# Demonstrates string concatenation and repetition

print("You can concatenate two " + "strings with the '+' operator.")

print("\nThis string " + "may not " + "seem terr" + "ibly impressive. " \
      + "But what " + "you don't know" + " is that\n" + "it's one real" \
      + "l" + "y" + " long string, created from the concatenation " \
      + "of " + "twenty-two\n" + "different strings, broken across " \
      + "six lines." + " Now are you" + " impressed? " + "Okay,\n" \
      + "this " + "one " + "long" + " string is now over!")

print("\nIf you really like a string, you can repeat it. For example,")
print("who doesn't like pie? That's right, nobody. But if you really")
print("like it, you should say it like you mean it:")
print("Pie" * 10)

input("\n\nPress the enter key to exit.")

```

Spajanje stringova

- Spajanje ili nadovezivanje ili nastavljjanje stringova jedan na drugi ("concatenation") se vrši pomoću operatora +:

```
print("You can concatenate two " + "strings with the '+' operator.")
```

pri spajanju, računar ne umeće nikakve nove karaktere (zato posle "two" stoji razmak)

- Naredna dugačka linija prikazuje primer nadovezivanja većeg broja stringova

Karakter za nastavljjanje reda

- U opštem slučaju, u programskom kodu se piše jedna naredba u liniji (redu) na ekranu; ako je naredba predugačka (kao u prethodnom slučaju), može se "razvući" u više linija; da bi se to označilo, koristi se *karakter za nastavljjanje reda* ("line continuation character") \, to jest beksleš

- Ovaj karakter se stavlja na mesto gde bi programer normalno u kodu stavio razmak (ali naravno ne unutar stringa) -- računar će smatrati da se radi o jednoj jedinstvenoj liniji koda; ovo se često koristi kako bi se povećala preglednost programa

Ponavljjanje stringova

- Da bi se dati string više puta ispisao na ekranu, koristi se *operator ponavljanja* * (znak za množenje); tako na primer linija:

```
print("Pie" * 10)
```

znači da će se string "Pie" ponoviti 10 puta, što će ispisati:

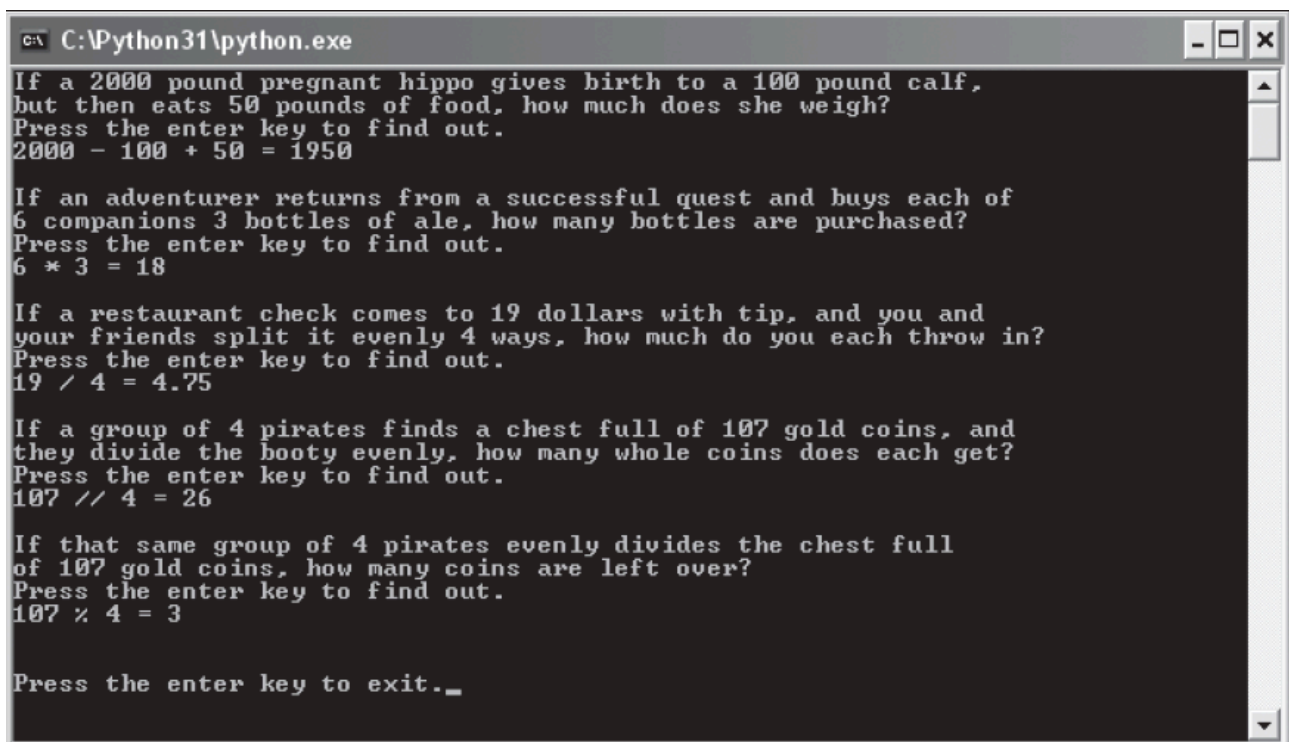
PiePiePiePiePiePiePiePiePiePie

2.5 Rad sa brojevima

- Dosad je pokazan rad sa stringovima koji reprezentuju tekst; podaci se osim tekstualno mogu predstavljati i kao brojevi; brojevi se koriste skoro u svakom programu; Python nudi više tipova brojnih podataka koji se koriste zavisno od potrebe

Program Word Problems

- Program Word Problems obavlja jednostavna izračunavanja pomoću računskih operacija sabiranja, oduzimanja, množenja i deljenja (Slika 2-5)



```

C:\Python31\python.exe
If a 2000 pound pregnant hippo gives birth to a 100 pound calf,
but then eats 50 pounds of food, how much does she weigh?
Press the enter key to find out.
2000 - 100 + 50 = 1950

If an adventurer returns from a successful quest and buys each of
6 companions 3 bottles of ale, how many bottles are purchased?
Press the enter key to find out.
6 * 3 = 18

If a restaurant check comes to 19 dollars with tip, and you and
your friends split it evenly 4 ways, how much do you each throw in?
Press the enter key to find out.
19 / 4 = 4.75

If a group of 4 pirates finds a chest full of 107 gold coins, and
they divide the booty evenly, how many whole coins does each get?
Press the enter key to find out.
107 // 4 = 26

If that same group of 4 pirates evenly divides the chest full
of 107 gold coins, how many coins are left over?
Press the enter key to find out.
107 % 4 = 3

Press the enter key to exit._

```

- Odgovarajući kod word_problems.py je:

```

# Word Problems
# Demonstrates numbers and math

print("If a 2000 pound pregnant hippo gives birth to a 100 pound calf,")
print("but then eats 50 pounds of food, how much does she weigh?")
input("Press the enter key to find out.")
print("2000 - 100 + 50 =", 2000 - 100 + 50)

print("\nIf an adventurer returns from a successful quest and buys each of")
print("6 companions 3 bottles of ale, how many bottles are purchased?")
input("Press the enter key to find out.")
print("6 * 3 =", 6 * 3)

```

```

print("\nIf a restaurant check comes to 19 dollars with tip, and you and")
print("your friends split it evenly 4 ways, how much do you each throw in?")
input("Press the enter key to find out.")
print("19 / 4 =", 19 / 4)

print("\nIf a group of 4 pirates finds a chest full of 107 gold coins, and")
print("they divide the booty evenly, how many whole coins does each get?")
input("Press the enter key to find out.")
print("107 // 4 =", 107 // 4)

print("\nIf that same group of 4 pirates evenly divides the chest full")
print("of 107 gold coins, how many coins are left over?")
input("Press the enter key to find out.")
print("107 % 4 =", 107 % 4)

input("\n\nPress the enter key to exit.")

```

Numerički tipovi podataka

- Prethodni program radi sa brojevima; on koristi dva najčešće korišćena tipa numeričkih (brojnih) podataka koji se koriste u Pythonu: celi brojevi ("integers") i realni brojevi sa pokretnim zarezom ("floating-point real numbers" = "floats")
- Iz matematike je poznato da celi brojevi nemaju decimalni deo (onaj posle decimalnog zareza), i pišu se bez decimalnog zareza, na primer 10; realni brojevi imaju i decimalni deo, na primer 10.688; i jedni i drugi mogu biti pozitivni i negativni

Upotreba matematičkih operatora

- Računar ume da obavlja navedene četiri računske operacije: +, -, * i /; na ove se dodaje još dve:
 - celobrojno deljenje (operator //) je deljenje pri kome se deo količnika posle decimalnog zareza zanemaruje: $7//3=2$
 - deljenje po modulu (operator %) je deljenje pri kome se određuje samo celobrojni ostatak: $7\%3=1$

- Još treba napomenuti da Python radi sa definisanim brojem decimala: tako je rezultat pri deljenju 7 sa 3 jednak 2.3333333333333335 (?)

2.6 Promenljive (variable)

- Uz pomoć promenljivih/varijabli ("variable" -- koriste se i srpski i engleski termin) mogu se čuvati i obrađivati informacije, koje su suštinski aspekt programiranja

Program Greeter

- Rezultat programa Greeter je prikazan na Slici 2-6



- Odgovarajući kod greeter.py je:

```
# Greeter
# Demonstrates the use of a variable

name = "Larry"
print(name)
print("Hi,", name)
input("\n\nPress the enter key to exit.")
```

Definisanje promenljivih

- Promenljiva omogućava da se neki podatak označi i da mu se kasnije pristupa radi obrade; programer ne mora da zna gde se u memoriji računara nalazi traženi podatak; umesto toga, on koristi promenljivu koju je definisao

- Linija:

```
name = "Larry"
```

predstavlja naredbu dodele ("assignment statement") -- datoj promenljivoj (name) se dodeljuje data vrednost (string "Larry"), to jest ona dobija tu vrednost

- Ako je pre dodele:

- promenljiva postojala (čime je morala i da ima određenu vrednost), naredbom dodele joj se dodeljuje nova vrednost

- promenljiva nije postojala, naredbom dodele ona se automatski kreira (definiše) i dodeljuje joj se navedena vrednost (kao inicijalna)

- U jeziku Python promenljive se ne deklarishu unapred; u nekim drugim jezicima, promenljiva se prvo u jednoj liniji mora deklarirati (definiše joj se tip), pa joj se tek onda može dodeliti vrednost

- U navedenom primeru, promenljivoj name se dodaje vrednost "Larry", koja predstavlja string, pa ona tako postane string promenljiva; da se radilo o celom broju (name=66), ona bi postala integer promenljiva

Upotreba promenljivih

- Za promenljivu se kaže da *referencira* onu vrednost koja joj je dodeljena; zato se promenljiva može koristiti na isti način kao i vrednost koju referencira

- U prikazanom primeru, pošto se izvrši dodela, naredba:

```
print(name)
```

obavlja istu funkciju kao i

```
print("Larry")
```

Imena promenljivih

- Programer je slobodan da po želji odabere imena kojima će nazvati svoje promenljive; potrebno je samo da ispoštuje sledeća dva pravila (da se Python ne bi bunio):

- 1) ime može sadržati samo cifre, slova i donje crte

- 2) ime promenljive ne može početi cifrom

- Sa druge strane, dobra programerska praksa daje nekoliko preporuka koja je pametno ispoštovati, kao što sledi

- Preporuka 1: birati što opisnija imena -- tako će kasnije čitanje programa biti jasnije i za autora, i za druge (čak i kad nema komentara): mnogo je bolje `zbir_godina` nego `zbg`

- Preporuka 2: biti dosledan -- držati se jednom definisanih zakonitosti; po preporuci, bolje je `broj_poena` (znači sve sa donjom crtom) nego `brojPoena` ili `brojpoena`

- Preporuka 3: imena promenljivih treba uvek da počinju malim slovom, i da nikada ne počinju podvučenom crtom (za takva imena čuva se posebno značenje)

- Preporuka 4: ne davati predugačka imena, jer može da se pogreši u kucanju, i teško se čita; preporuka: dužina do 15 karaktera

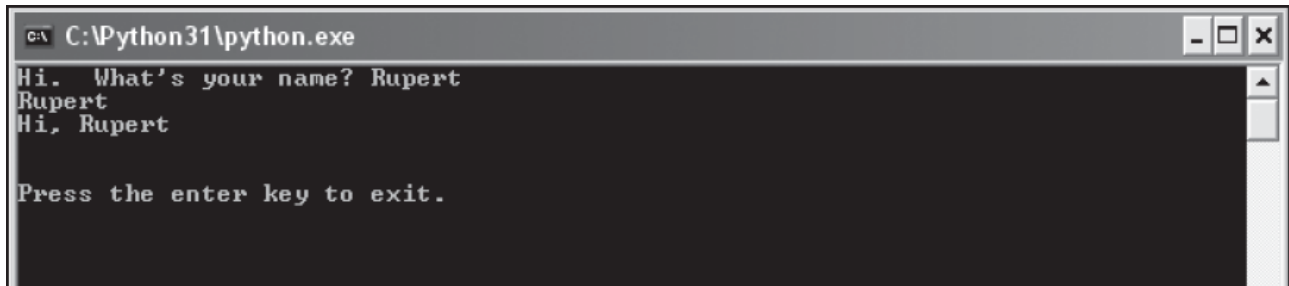
- Sve navedeno uglavnom važi i za biranje imena py fajlova

2.7 Uzimanje podataka od korisnika

- Promenljive dobijene od korisnika su neophodne za svaki ozbiljniji program -- bez njih nema nikakve obrade podataka

Program Personal Greeter

- Ovaj program uvodi novi element -- mogućnost da korisnik unese podatak (sa tastature); on unese svoje ime, koje se zatim pojavi u pozdravnoj poruci (Slika 2.7)



- Odgovarajući kod `personal_greeter.py` je:

```
# Personal Greeter
# Demonstrates getting user input

name = input("Hi. What's your name? ")
print(name)
print("Hi,", name)
input("\n\nPress the enter key to exit.")
```

Upotreba funkcije input()

- Jedina promena u odnosu na prethodni kod je naredba dodele:

```
name = input("Hi. What's your name? ")
```

Umesto da promenljivoj `name` vrednost dodeli programer, poziva se funkcija `input()` koja traži od korisnika ulazni podatak (njegovo ime), tako što mu postavi pitanje i čeka na odgovor

- Argument koji je zadat funkciji `input()` je string "Hi. What's your name? "; kada korisnik otukuca nešto na tastaturi i pritisne Enter, ova funkcija vraća ("return") kao *povratnu vrednost* ("return value") ono što je korisnik otkucao u obliku stringa

- Konačno, string (tekst) koga je korisnik uneo postaje ona vrednost koja se dodeljuje promenljivoj name (isto kao da smo otkucali name="tekst koga je korisnik otkucao"); u naredne dve linije koda, promenljiva name se normalno koristi -- računaru je svejedno na koji način je promenljivoj name dodeljena vrednost

- U poslednjoj liniji koda:

```
input("\n\nPress the enter key to exit.")
```

cilj je da se sačeka da korisnik pritisne Enter (ili šta god bilo pa Enter, nebitno); pošto je sasvim nebitno šta će korisnik da pritisne, jer je jedino bitno da se na njegovu komandu konzola zatvori pošto se ova linija koda izvrši, u ovoj liniji nema nikakve dodele (drugim rečima, dobija se povratna vrednost ali se sa njom ne radi ništa)

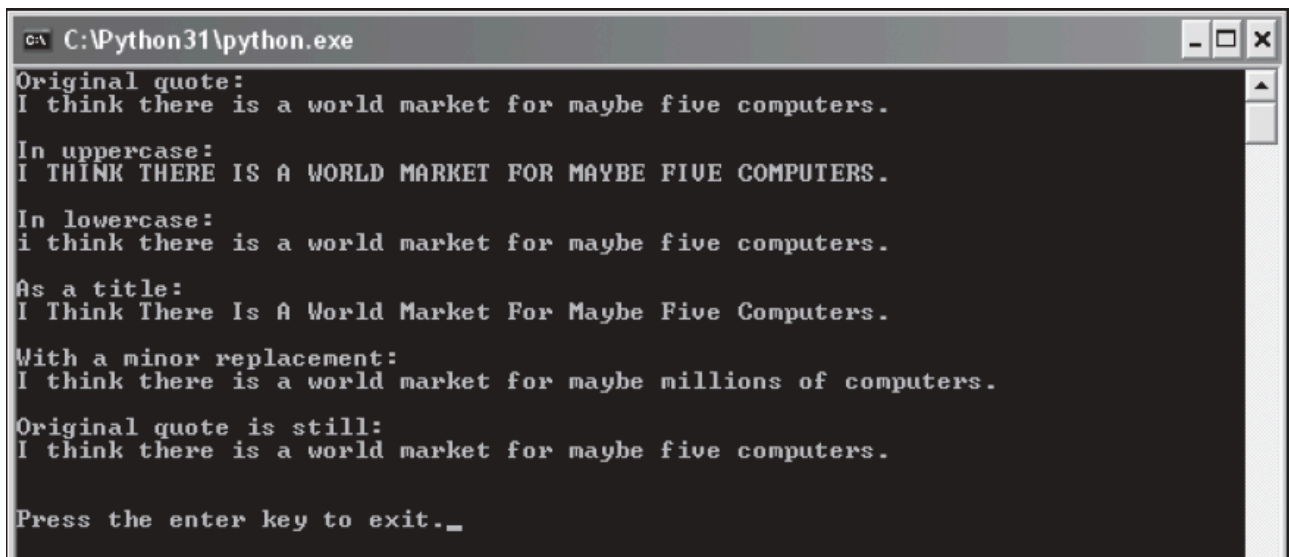
2.8 Metode za obradu stringova (string metode)

- Pre svega: pojam "metoda" je vezan za OO programiranje i neće biti objašnjavan; ovde je dovoljno prihvatiti da je to neki programski alat koji obavlja određeni zadatak, nalik na funkciju

- Pomoću metoda za obradu stringova (kraće: string metoda) mogu se obavljati određeni zadaci, vezani za stringove, sa kojima se programeri i korisnici često sreću; primer: pretvaranje svih slova u nekom tekstu u velika slova

Program Quotation Manipulation

- Ovaj program (koji služi za manipulisanje citatima) uzima određeni tekst (u ovom slučaju jedan zanimljiv citat iz 1943.) i transformiše ga na nekoliko načina -- pravi sve velika slova, pravi sve mala slova, pravi naslov na američki način (početna slova svih reči veliko), i pravi promenu jedne reči (Slika 2-8)



```
C:\Python31\python.exe
Original quote:
I think there is a world market for maybe five computers.
In uppercase:
I THINK THERE IS A WORLD MARKET FOR MAYBE FIVE COMPUTERS.
In lowercase:
i think there is a world market for maybe five computers.
As a title:
I Think There Is A World Market For Maybe Five Computers.
With a minor replacement:
I think there is a world market for maybe millions of computers.
Original quote is still:
I think there is a world market for maybe five computers.
Press the enter key to exit._
```

- Odgovarajući kod quotation_manipulation.py je:

```
# Quotation Manipulation
# Demonstrates string methods

# quote from IBM Chairman, Thomas Watson, in 1943
quote = "I think there is a world market for maybe five computers."

print("Original quote:")
print(quote)

print("\nIn uppercase:")
print(quote.upper())

print("\nIn lowercase:")
print(quote.lower())

print("\nAs a title:")
print(quote.title())

print("\nWith a minor replacement:")
print(quote.replace("five", "millions of"))

print("\nOriginal quote is still:")
print(quote)

input("\n\nPress the enter key to exit.")
```

Pravljenje novih stringova

- U prethodnom kodu je sve intuitivno jasno; na primer, linija:

```
print(quote.upper())
```

šampa string quote tako da su sva slova velika ("uppercase letters"); ovo se postiže pozivanjem string metode upper()

- String metodom se obavlja određena operacija (manipulacija/obrada) nad stringom -- u ovom slučaju, od stringa quote kreira se nov string, njegova verzija sa svim velikim slovima, pomoću string metode upper() koja se stringu quote pridružuje

- Kaže se da string metoda vraća ("return" -- isti termin koji se koristio i za funkcije) novi string koga je kreirala, i u ovom slučaju on se ispisuje

- Pored imena string metode nalazi se par zagrada, što podseća na funkcije; međutim, funkcija se može pozvati samostalno, kao što je pokazano u slučaju print(); sa druge strane, string metoda mora biti pozvana "u paketu" sa odgovarajućim stringom

- String metoda se *poziva* ("invoke" -- nema odgovarajuće srpske reči pa se koristi ista kao za "call" kod funkcija) tako što se posle imena stringa doda tačka pa onda ime string metode, posle koje stoji i par zagrada:

ime_stringa.ime_metode()

- U slučaju string metode upper(), između zagrada se ne nalazi ništa, što znači da ova metoda ne zahteva nikakve argumente; neke druge metode ih zahtevaju

- Sledeće dve metode, lower() i title() su vrlo slične i ne zahtevaju posebno objašnjenje

- Linija:

```
print(quote.replace("five", "millions of"))
```

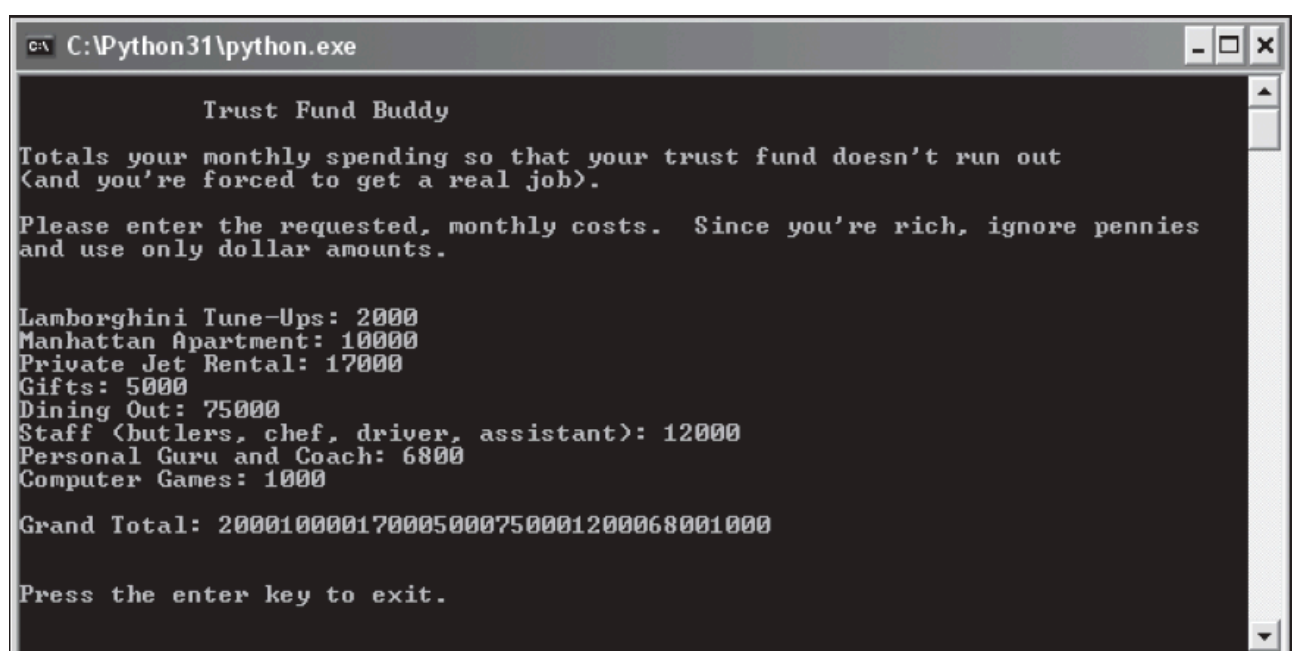
postiže to da se tekst "five" zameni tekstem "millions of"; ovo se radi pomoću string metode replace()

- Metoda `replace()` zahteva dva argumenta, razdvojena zarezom: stari string (koji se menja) i novi string (kojim se menja) -- vrši se zamena na svim mestima gde se stari tekst pojavljuje, izuzev ako se kao treći argument doda celobrojna vrednost koja zadaje najveći dozvoljeni broj zamena
- Na kraju koda nalazi se linija koja pokazuje da se string quote nije promenio; naime string metode kreiraju (i vraćaju) nove stringove ali ne menjaju onaj originalni
- Još tri string metode koje mogu biti korisne su: `swapcase()` -- velika slova menja malima i obrnuto, `capitalize()` -- samo prvo slovo stringa veliko, `strip()` -- svi tabovi i razmaci na početku i kraju stringa se brišu

2.9 Upotreba ispravnih tipova podataka

- U dosadašnjem tekstu prikazani su sledeći tipovi podataka: stringovi, celi brojevi i realni brojevi; u ovoj sekciji se uvode i objašnjavaju još neki

Program Trust Fund Buddy (loša verzija)



```

C:\Python31\python.exe

Trust Fund Buddy

Totals your monthly spending so that your trust fund doesn't run out
(and you're forced to get a real job).

Please enter the requested, monthly costs. Since you're rich, ignore pennies
and use only dollar amounts.

Lamborghini Tune-Ups: 2000
Manhattan Apartment: 10000
Private Jet Rental: 17000
Gifts: 5000
Dining Out: 75000
Staff (butlers, chef, driver, assistant): 12000
Personal Guru and Coach: 6800
Computer Games: 1000

Grand Total: 200010000170005000750001200068001000

Press the enter key to exit.

```

- Program ima za cilj da izračuna ukupne mesečne troškove nekog bogataša, tako što korisnik unosi pojedinačne troškove; prvo će biti predstavljena loša verzija (Slika 2-9)

- Očigledno, dobijeni rezultat nije tačan; on ima bag, ali ne onaj koji uzrokuje da se program blokira; takav tip greške (baga) naziva se *logička greška* ("logical error"): prema obliku rezultata, očigledno je da je program samo nastavio/nadovezao stringove jedan na drugi, umesto da izračuna zbir

- Odgovarajući kod trust_fund_bad.py je:

```
# Trust Fund Buddy - Bad
# Demonstrates a logical error

print(
    """
        Trust Fund Buddy

Totals your monthly spending so that your trust fund doesn't run out
(and you're forced to get a real job).

Please enter the requested, monthly costs.  Since you're rich, ignore pennies
and use only dollar amounts.

    """)

car = input("Lamborghini Tune-Ups: ")
rent = input("Manhattan Apartment: ")
jet = input("Private Jet Rental: ")
gifts = input("Gifts: ")
food = input("Dining Out: ")
staff = input("Staff (butlers, chef, driver, assistant): ")
guru = input("Personal Guru and Coach: ")
games = input("Computer Games: ")

total = car + rent + jet + gifts + food + staff + guru + games

print("\nGrand Total:", total)

input("\n\nPress the enter key to exit.")
```

Otkrivanje logičkih grešaka

- Logičke greške često mogu biti vrlo teške za otkrivanje: pošto se program ne blokira ali ne odradi zadatak kako treba, nema poruke o grešci koja bi mogla da pomogne; treba pregledati ceo kod

- U ovom slučaju, pogled na rezultat otkriva grešku: svaki "broj" koga je korisnik uneo je tretiran kao string; tako da linija

```
total = car + rent + jet + gifts + food + staff + guru + games
```

nadovezuje stringove (kao što je pokazano, pomoću operatora +), umesto da sabere brojeve

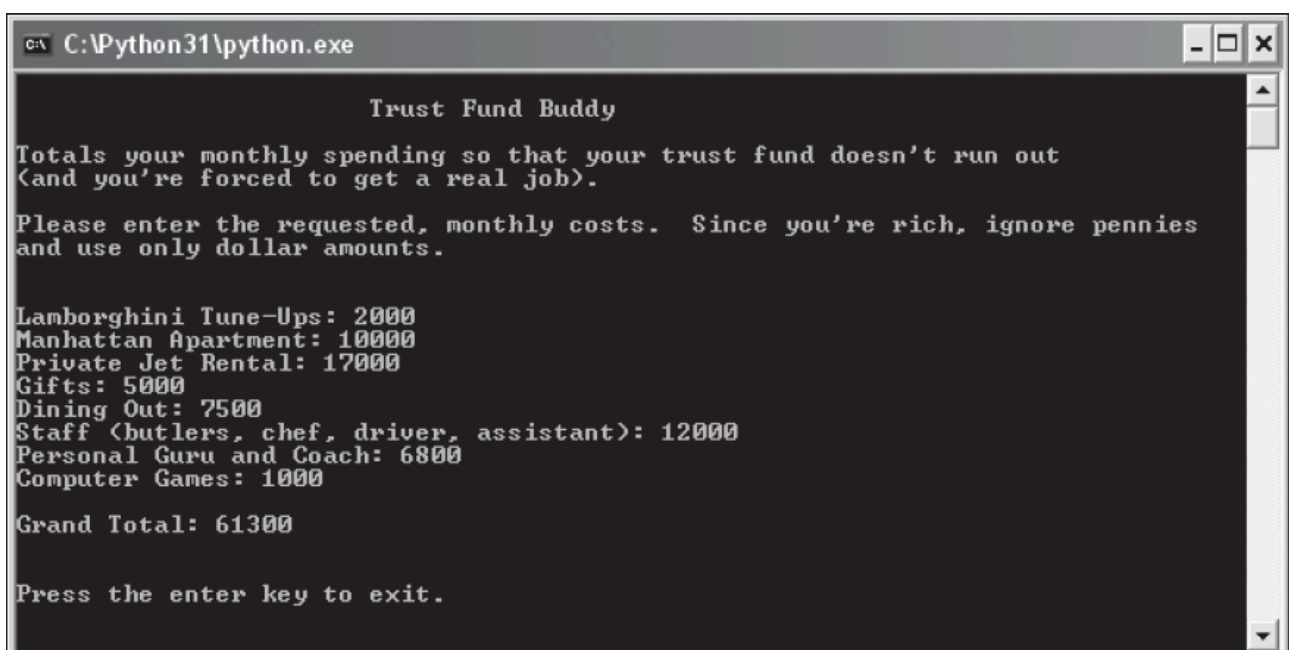
- Ova greška se otklanja tako što unete string vrednosti treba nekako konvertovati u brojeve

2.10 Konverzija (pretvaranje) vrednosti

- Znači, string vrednosti koje je vratila funkcija input() treba konvertovati (pretvoriti) u brojne vrednosti; pošto se (prema uslovu) ne unose decimalni iznosi, svaki string treba konvertovati u celi broj (celobrojnu vrednost)

Program Trust Fund Buddy (dobra verzija)

- Korigovana verzija programa daje tačan rezultat (Slika 2-10)



```
C:\Python31\python.exe

Trust Fund Buddy

Totals your monthly spending so that your trust fund doesn't run out
(And you're forced to get a real job).

Please enter the requested, monthly costs. Since you're rich, ignore pennies
and use only dollar amounts.

Lamborghini Tune-Ups: 2000
Manhattan Apartment: 10000
Private Jet Rental: 17000
Gifts: 5000
Dining Out: 7500
Staff (butlers, chef, driver, assistant): 12000
Personal Guru and Coach: 6800
Computer Games: 1000

Grand Total: 61300

Press the enter key to exit.
```

- Odgovarajući kod `trust_fund_good.py` je:

```
# Trust Fund Buddy - Good
# Demonstrates type conversion

print(
    """
                Trust Fund Buddy

Totals your monthly spending so that your trust fund doesn't run out
(and you're forced to get a real job).

Please enter the requested, monthly costs.  Since you're rich, ignore pennies
and use only dollar amounts.

    """
)

car = input("Lamborghini Tune-Ups: ")
car = int(car)

rent = int(input("Manhattan Apartment: "))
jet = int(input("Private Jet Rental: "))
gifts = int(input("Gifts: "))
food = int(input("Dining Out: "))
staff = int(input("Staff (butlers, chef, driver, assistant): "))
guru = int(input("Personal Guru and Coach: ") )
games = int(input("Computer Games: "))

total = car + rent + jet + gifts + food + staff + guru + games

print("\nGrand Total:", total)

input("\n\nPress the enter key to exit.")
```

Konverzija stringova u cele brojeve

- Funkcija `int()` konvertuje string u ceo broj: promenljiva `car` je uneta kao string, ali u narednoj liniji se obavlja konverzija, i postaje ceo broj ("integer"); naredne linije u kodu obavljaju isto to, samo u jednoj liniji
- U navedenim linijama koristi se *ugnježdživanje* ("nesting") funkcija (u ovom slučaju, `int()` i `input()`) -- stavljanje "jedne u drugu"; ovo se vrlo često koristi, s tim što naravno treba voditi računa da povratna vrednost "unutrašnje" funkcije može da bude argument "spoljašnje" funkcije; ovde `input()` vraća string, koga `int()` prihvata kao argument

- Osim `int(x)`, mogu se pomenuti još dve funkcije za konverziju:
 - `float(x)` koja vraća realni broj tako što konvertuje string `x`
`float("10.0")=10.0`
 - `str(x)` koja radi obrnuto od `int(x)`: ona konvertuje celi broj u string
`str(10)="10"`

Upotreba inkrementnih operatora za dodelu

- *Inkrementni operatori za dodelu* ("augmented assignment operator") predstavljaju jednostavni koncept, koji je ilustrovan na sledeći način -- umesto na primer:

```
food = food * 52
```

može se jednostavnije pisati:

```
food *= 52
```

- Operator `*=` je jedan od pomenutih operatora; oni obezbeđuju skraćeni zapis koda; ostali koji se koriste (dati pomoću primera) su:

`x /= 5` je `x = x / 5`

`x %= 5` je `x = x % 5`

`x += 5` je `x = x + 5`

`x -= 5` je `x = x - 5`

2.11 Povratak na program Useless Trivia

- Pošto je predstavljeno sve što treba, može biti predstavljen ceo kod za program Useless Trivia (`useless_trivia.py`)

```

# Useless Trivia
#
# Gets personal information from the user and then
# prints true but useless information about him or her

name = input("Hi. What's your name? ")

age = input("How old are you? ")
age = int(age)

weight = int(input("Okay, last question. How many pounds do you weigh? "))

print("\nIf poet ee cummings were to email you, he'd address you as",
      name.lower())
print("But if ee were mad, he'd call you", name.upper())

called = name * 5
print("\nIf a small child were trying to get your attention",)
print("your name would become:")
print(called)

seconds = age * 365 * 24 * 60 * 60
print("\nYou're over", seconds, "seconds old.")

moon_weight = weight / 6
print("\nDid you know that on the moon you would weigh only",
      moon_weight, "pounds?")

sun_weight = weight * 27.1
print("On the sun, you'd weigh", sun_weight, "(but, ah... not for long).")

input("\n\nPress the enter key to exit.")

```

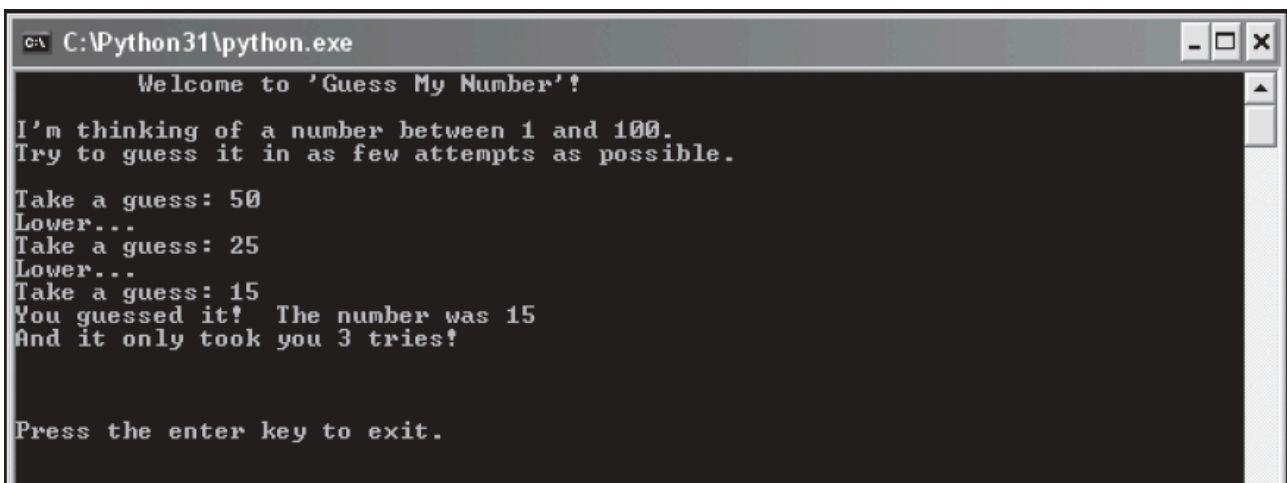
- Nekoliko dodatnih napomena i podsećanja:
 - blokovi (sekcije) koda su odvojeni praznim linijama
 - iskusni programeri koriste uvodni komentar i da navedu koje su modifikacije napravili, i kog datuma -- tako se vidi istorija promena
 - promenljive age i weight se konvertuju na dva načina -- u praksi treba odabrati jedan zbog konzistentnosti
 - called = name * 5 je ponavljanje stringa 5 puta
 - kada se ceo broj weight pomnoži realnim brojem 27.1, nastaje realan broj sun_weight

Poglavlje 3: grananje, while-petlja, planiranje programa

- Dosadašnji programi su imali jednostavan, sekvencijalni tok: svaka naredba je bila izvršavana tačno jedanput, po redu pojavljivanja; ovo poglavlje će pokazati kako se naredbe selektivno izvršavaju, i kako se ponavljaju delovi koda
- U poglavlju će biti pokazano:
 - kako se generišu slučajni brojevi pomoću `randint()` i `randrange()`
 - kako se koriste `if`, `else`, `elif` naredbe da se kod izvrši prema nekom uslovu
 - kako se koristi `while`-petlja da se ponavljaju delovi programa
 - kako se planira program pomoću pseudokoda

3.1 Igra Guess My Number

- Program koji će biti kreiran na kraju ovog poglavlja je klasična igra pogađanja broja: računar bira slučajni broj od 1 do 100, a igrač hoće da pogodi iz što manje pokušaja; prilikom pogađanja, računar će igrača obavestavati da li je traženi broj veći, manji ili jednak onome koga je igrač odigrao (Slika 3-1); kad pogodi, obavestiće ga o broju pokušaja



```
C:\Python31\python.exe

Welcome to 'Guess My Number'!

I'm thinking of a number between 1 and 100.
Try to guess it in as few attempts as possible.

Take a guess: 50
Lower...
Take a guess: 25
Lower...
Take a guess: 15
You guessed it! The number was 15
And it only took you 3 tries!

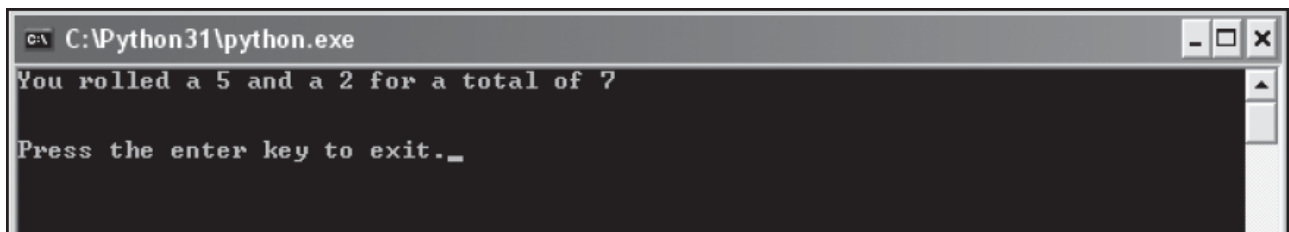
Press the enter key to exit.
```

3.2 Generisanje slučajnih brojeva

- U programima je često potrebno imati slučajne (nasumično odabrane) brojeve (primer: ponašanje protivnika u nekoj igri) -- Python pruža ovu mogućnost
- U stvari, generisanje tih brojeva nije u računarima sasvim slučajno, već se bazira na određenim matematičkim formulama (pa se ti brojevi zato nazivaju pseudoslučajni); ipak, dovoljno su "slučajni" za svakodnevne programerske potrebe

Program Craps Roller

- Ovaj program simulira istovremeno bacanje dve kocke -- prikazuje brojeve koji su pali na obe, i zbir te dve vrednosti; program koristi generisanje slučajnih brojeva (Slika 3-2)



- Odgovarajući kod craps_roller.py je:

```
# Craps Roller
# Demonstrates random number generation

import random

# generate random numbers 1 - 6
die1 = random.randint(1, 6)
die2 = random.randrange(6) + 1

total = die1 + die2

print("You rolled a", die1, "and a", die2, "for a total of", total)

input("\n\nPress the enter key to exit.")
```

Učitavanje modula random

- Prva linija koda sadrži naredbu import -- ova naredba služi da se u memoriju učitaju (uvezu) programski moduli; u ovom slučaju to je modul pod imenom random

- *Modul* ("module") je fajl koji sadrži neki kod, koji je namenjen da se po potrebi koristi u drugim programima; svaki modul obično sadrži nekoliko funkcija, koje obavljaju zadatke vezane za određeni domen; tako modul *random* sadrži nekoliko funkcija koje generišu slučajne brojeve ili proizvode neke slične slučajne rezultate
- Moduli se mogu posmatrati kao posebni i uvek spremni alati (preciznije: skupovi funkcija), koje je napisao neko drugi, i koji programeru stoje na raspolaganju "u fioci" da može da ih koristi prema svojim potrebama
- Program svaku od tih funkcija poziva posredno, uz pomoć njenog modula, na sledeći način: ime modula pa tačka pa ime funkcije;, uz zagradu u kojoj se mogu nalaziti neki argumenti; pomoću ove notacije (notacija sa tačkom, "dot notation") stavlja se do znanja da funkcija pripada modulu
- Usput pomenimo, radi ilustracije, još neke važne module i po jednu njihovu funkciju:
 - *time* (sve u vezi sa računanjem vremena); *clock()* koja vraća aktuelno CPU vreme, kao realan broj u sekundama
 - *math* (matematičke funkcije); *log(x)* računa prirodni logaritam
 - *array* (rad sa nizovima); *append(x)* dodaje *x* na kraj niza
 - *os* (rad sa fajlovima i folderima); *getcwd()* vraća aktivni folder

Upotreba funkcije *randint()*

- Modul *random* sadrži funkciju *randint()* koja generiše slučajni ceo broj; u ovom primeru, ona se koristi na sledeći način:

```
random.randint(1, 6)
```

- Funkcija `randint()` kao argumente zahteva dva cela broja `c1` i `c2`, a vraća slučajni ceo broj `x` koji ispunjava uslov $c1 \leq x \leq c2$; u slučaju kocke, argumenti su 1 i 6

Upotreba funkcije `randrange()`

- Modul `random` sadrži i funkciju `randrange()`; ona se može koristiti na više načina, ali onaj koji je prikazan u primeru je najjednostavniji -- funkcija kao argument zahteva jedan ceo broj `c`, a vraća slučajni ceo broj `x` koji ispunjava uslov $0 \leq x < c$
- Za kocku, generiše se broj od 0 do 5, pa se zatim dodaje 1:

```
random.randrange(6) + 1
```

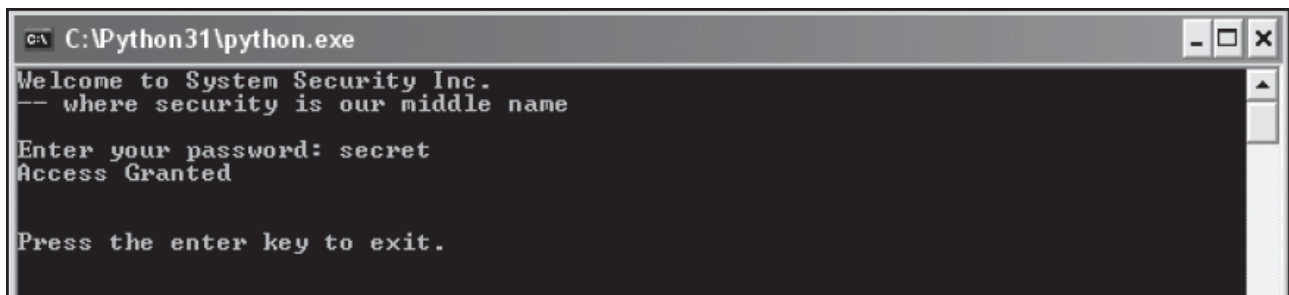
(naravno oba načina generisanja su ravnopravni)

3.3 Upotreba naredbe `if`

- *Grananje* ("branching") je fundamentalni koncept u programiranju; on označava da se donosi odluka da li da se krene jednim ili drugim "putem"; naredba `if` (=ako), omogućava izbor između dve opcije koje jedna drugu isključuju, tako što se:
 - nešto uradi (to jest izvrši određeni blok u kodu) ako je ispunjen neki uslov koji se proverava, i:
 - uradi nešto drugo (to jest izvrši neki drugi blok) ako nije

Program Password

- Ovaj jednostavni program traži od korisnika da unese lozinku (password), i obaveštava ga o tome da li je lozinka ispravna ili ne (Slika 3-4 pokazuje situaciju kada jeste; ako nije, odmah završi program); lozinka koju treba uneti je "secret"

A screenshot of a Windows command prompt window titled "C:\Python31\python.exe". The window contains the following text: "Welcome to System Security Inc.", "-- where security is our middle name", "Enter your password: secret", "Access Granted", and "Press the enter key to exit.". The text is displayed in a monospaced font on a dark background.

```
C:\Python31\python.exe
Welcome to System Security Inc.
-- where security is our middle name

Enter your password: secret
Access Granted

Press the enter key to exit.
```

- Odgovarajući kod password.py je:

```
# Password
# Demonstrates the if statement

print("Welcome to System Security Inc.")
print("-- where security is our middle name\n")

password = input("Enter your password: ")

if password == "secret":
    print("Access Granted")

input("\n\nPress the enter key to exit.")
```

Naredba if

- U prethodnom programu, ključna naredba je *if* -- ona je logički veoma prosta, jer samo proverava uslov: ako je uneti string jednak stringu "secret", poruka se štampa na ekranu i ide se na poslednju liniju; ako nije jednak, odmah se prelazi na poslednju liniju

Zadavanje uslova

- Svaka if naredba sadrži neki *uslov* ("condition"): uslov je izraz koji je ili tačan/istinit ("true") ili netačan/neistinit ("false"); takvi izrazi su sasvim uobičajeni u svakodnevnom životu, kao na primer "napolju je 23 stepena" -- ili je tačno ili nije (nema treće)
- Python ima svoje sopstvene vrednosti koje reprezentuju istinitost i neistinitost: True (= tačno) i False (=netačno); neki uslov može/mora imati tačno jednu od te dve vrednosti

- U prethodnom primeru, uslov je:

```
password == "secret"
```

i dobija vrednost ili True ili False, zavisno od vrednosti promenljive password

Operatori za poređenje

- Uslovi nastaju tako što se porede neke vrednosti; to poređenje se obavlja pomoću *operatora za poređenje* ("comparison operators"); u primeru je već pokazan operator za jednakost: ==

- Znači, za proveru jednakosti koristi se dupli znak jednakosti; običan znak jednakosti se koristi za dodelu, kao što je ranije pokazano (primer: password = "secret")

- Pored ovog operatora, postoje i sledeći: != nije jednako, > veće, < manje, >= veće ili jednako, <= manje ili jednako

- Pomoću navedenih operatora, porede se celi brojevi sa celima, realni sa realnima, kao i celi sa realnima; mogu se "porediti" i dva stringa -- "manji" je onaj koji je pre po abecednom redu (pa tako uslov "jabuka" < "mandarina" ima vrednost True); stringovi i brojne vrednosti se ne mogu porediti

Kreiranje blokova pomoću indentacije

- U prethodnom kodu, linija koda posle naredbe if je uvučena:

```
if password == "secret":  
    print("Access Granted")
```

- Uvlačenje koda (pomeranje njegovog početka od leve margine), koje se obavlja pomoću Tab dugmeta, naziva se *indentacija* ("indentation"), a za liniju se kaže da je indentirana (uvučena)

- Pomoću indentacije se formiraju blokovi u okviru koda: *blok* ("block") je skup od jedne ili više uzastopnih linija koda koje su indentirane za isti razmak (to jest jednako pomerene od leve margine)

- Osim što su vizuelne, blokovi (što je još važnije) predstavljaju logičke celine: blok koda predstavlja jednu celinu koja se izvršava; blok je znači logički pojam

- U ovom primeru, blok se koristi kao deo naredbe `if`: njega čini jedna naredba (može ih biti i više), koja se izvršava ako uslov ima vrednost `True`; u slučaju da je vrednost `False`, ceo blok se preskače, i ide se na sledeću liniju (prvu koja ne pripada bloku)

- Primer za više naredbi u bloku je:

```
if password == "secret":  
    print("Access Granted")  
    print("Welcome!")
```

```
input("\n\nPress the enter key to exit.")
```

znači: u slučaju vrednosti `True` ispisuju se dva teksta; u slučaju vrednosti `False`, ne ispisuje se ništa; zatim se prelazi na prvu sledeću liniju (`input`)

- Indentacija je posebno svojstvo jezika Python, koja služi boljoj preglednosti i jednostavnosti: nema potrebe da se označava dokle ide petlja `if`, već se samo prati blok; što se tiče pravljenja indentacije, osim `Tab` može se koristiti i razmak (to jest nekoliko njih) ali to je manje preporučljivo; nivoa indentacije može biti koliko god je to potrebno

Sintaksa naredbe `if`

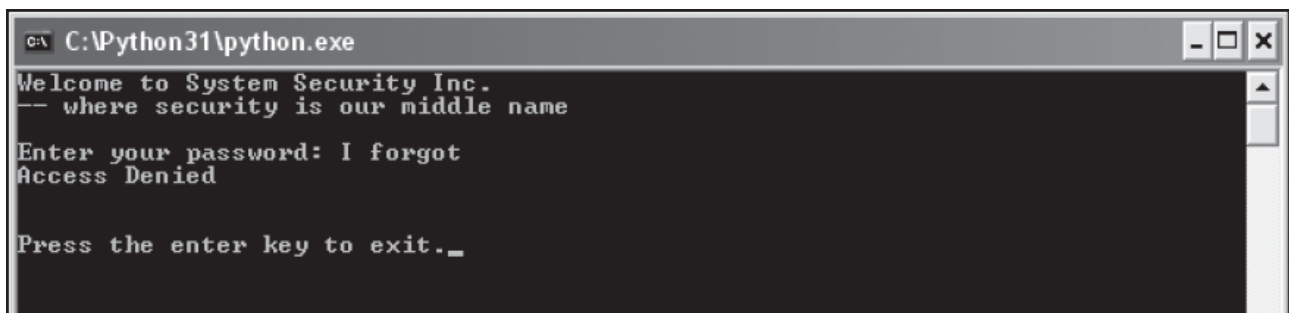
- Znači, naredba `if` se koristi na sledeći način: `if`, pa uslov, pa dve tačke, pa blok (od jedne ili više naredbi) -- ako je vrednost uslova `True`, blok se izvršava; ako je `False`, blok se preskače i izvršavanje se nastavlja na prvoj liniji posle bloka

3.4 Upotreba odredbe else

- U nekim situacijama postoji potreba da se u slučaju True uradi jedna stvar, a u slučaju False druga: za to se koristi odredba ("clause") *else* (=inače); *else* se naziva odredba jer nije samostalna, već se koristi uz naredbu *if*

Program Granted or Denied

- Program Password je obavestavao korisnika da je uneo ispravnu lozinku, ali nije davao nikakvo obavještenje u slučaju da je ona neispravna; program Granted or Denied (dozvoljen ili odbijen pristup) obrađuje ovaj drugi slučaj, uz upotrebu odredbe *else* (Slika 3-6; u slučaju da je lozinka ispravna dešava se kao i ranije, Slika 3-4)



- Odgovarajući kod `granted_or_denied.py` je:

```
# Granted or Denied
# Demonstrates an else clause

print("Welcome to System Security Inc.")
print("-- where security is our middle name\n")

password = input("Enter your password: ")

if password == "secret":
    print("Access Granted")
else:
    print("Access Denied")

input("\n\nPress the enter key to exit.")
```

Analiza odredbe else

- U odnosu na prethodni program, napravljena je samo jedna izmena:

```
if password == "secret":  
    print("Access Granted")  
else:  
    print("Access Denied")
```

zahvaljujući odredbi else, izvršava se drugi blok i ispisuje se obaveštenje "Access Denied"

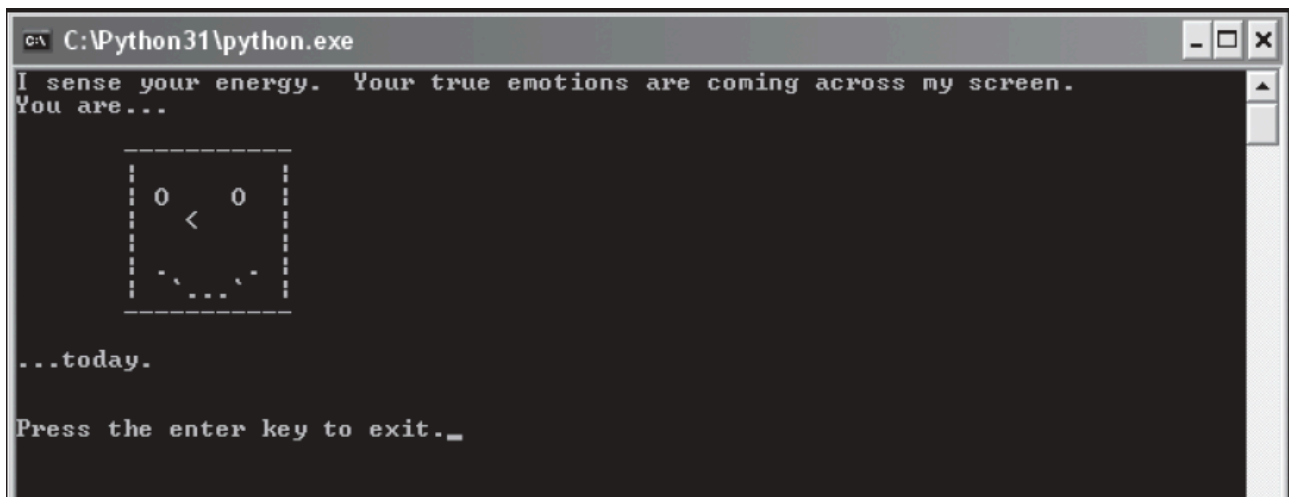
- Zaključak: u slučaju konstrukcije if-else (koju čine if naredba i else odredba), izvršiće se tačno jedan blok koda, i to:
 - ako je uslov True, izvršiće se onaj blok koji sledi posle linije if
 - ako je uslov False, izvršiće se onaj blok koji sledi posle linije else
- Znači, odredba else se koristi na sledeći način: posle bloka koji odgovara if naredbi, ide odredba else, pa dve tačke, pa odgovarajući blok (od jedne ili više naredbi)
- Važno je naglasiti da odredba else mora biti u istom bloku kao odgovarajuća naredba if (njen "parnjak"); drugim rečima, if i else moraju biti indentirani za isto rastojanje, to jest za isti broj tabova, to jest moraju se nalaziti na istoj "vertikali"

3.5 Upotreba odredbe elif

- Ako postoji više mogućnosti (opcija) u smislu da se jedna promenjiva poredi sa više različitih vrednosti, koristi se odredba *elif* (= else if)

Program Mood Computer

- Program generiše slučajni broj od 1 do 3, i na osnovu njega bira da na ekranu prikazuje jedan od tri jednostavna crteža lica (Slika 3-7)



- Odgovarajući kod mood_computer.py je (skraćena verzija, bez crteža lica -- umesto njih stoje oznake XXX, YYY, ZZZ):

```
# Mood Computer
# Demonstrates the elif clause

import random

print("I sense your energy. Your true emotions are coming across my screen.")
print("You are...")

mood = random.randint(1, 3)

if mood == 1:
    # happy
    print(XXX)
elif mood == 2:
    # neutral
    print(YYY)
elif mood == 3:
    # sad
    print(ZZZ)
else:
    print("Illegal mood value! (You must be in a really bad mood).")

print("...today.")

input("\n\nPress the enter key to exit.")
```

Analiza odredbe elif

- Naredba if upotrebljena sa odredbom elif sadrži sekvencu od nekoliko uslova koji se redom proveravaju (evaluiraju); u prethodnom kodu, proverava se redom da li je vrednost

mood jednaka 1, 2 ili 3; u sekvenci uvek uz prvi uslov ide if, a uz svaki naredni (koliko god ih bilo) ide elif

- U primeru se vrednost promenljive mood proverava u odnosu na tri različite vrednosti, pa u skladu sa tim:

- ako je vrednost 1: štampa se jedan crtež, i izlazi iz if-bloka

- ako nije 1, proverava se da li je vrednost 2, pa ako jeste: štampa se drugi crtež, i izlazi

- ako nije ni 2, proverava se da li je 3, pa ako jeste: štampa se treći crtež, i izlazi

- Važno je ponovo naglasiti sledeće: kada se nađe na uslov koji je ispunjen (True), izvršava se odgovarajući blok i zatim izlazi iz celog if-bloka, to jest ostali uslovi se više ne proveravaju, iako možda još neki može biti tačan

- U navedenom primeru ovo ne stvara nikakvu promenu, jer uslovi isključuju jedan drugog (može biti tačan samo jedan); ali u nekim slučajevima nekoliko uslova mogu biti tačni, i važno je voditi računa da će se izvršiti samo onaj blok koji odgovara prvom od tih uslova

- Na kraju, ukoliko vrednost promenljive mood nije jednaka ni 3 (to jest nije ni 1 ni 2 ni 3), izvršava se blok koji odgovara završnoj else odredbi (štampanje poruke); u ovom konkretnom slučaju, do ovog scenarija nikad neće doći jer će se sigurno generisati jedan od brojeva 1, 2, 3, ali je stavljeno da služi kao ilustracija

- Upotreba završne else odredbe je opcionalna, to jest ako nema potrebe za njom ne mora se ni pisati; ipak, njena upotreba se preporučuje u svakom slučaju, makar pokrivala i teorijski nemoguć slučaj (kao što je urađeno u primeru)

- Rezime je prikazan na sledeći način:

1) if

```
if <condition>:      if statement. If <condition> is true, <block> is executed; otherwise it's skipped.
    <block>
```

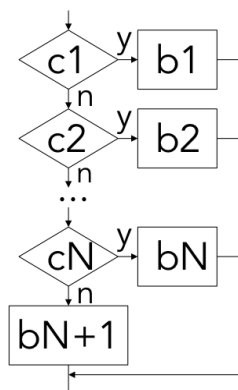
2) if-else

if <condition>: <block 1> else: <block 2>	if statement with else clause. If <condition> is true, <block1> is executed; otherwise <block2> is executed.
--	---

3) if-elif-else

if <condition 1>: <block 1> elif <condition 2>: <block 2> . . . elif <condition N>: <block N> else: <block N+1>	if statement with elif clauses and optional final else clause. The block of the first true condition is executed. If no condition is true, the optional else clause's block is executed.
---	---

- Ilustracija slučaja 3) pomoću blok-dijagrama:

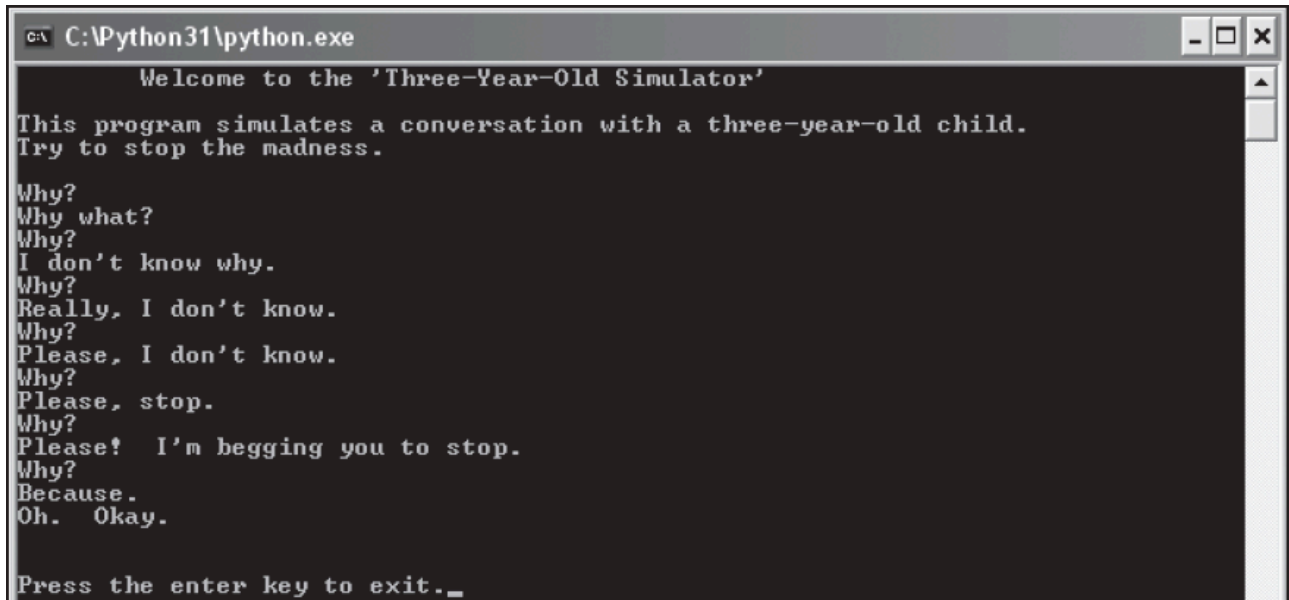


3.6 While petlja

- U programiranju često postoji potreba da se nešto ponavlja sve dok je ispunjen neki uslov (primer: programer pravi kviz -- sve dok ima pitanja koja nisu postavljena, kviz traje); while (= dok, sve dok) petlja omogućava da se ovo odradi

Program Three-Year-Old Simulator

- Ovaj jednostavni program simulira razgovor sa detetom od tri godine, i to uz korišćenje while petlje: program neprestano pita Why? (Zašto?) sve dok se ne odgovori sa Because. (Zato.); preciznije, program neprestano postavlja pitanje sve dok je ispunjen sledeći uslov: odgovor koji se daje NIJE (to jest različit je od) Because. (Slika 3-8)



```
C:\Python31\python.exe
Welcome to the 'Three-Year-Old Simulator'
This program simulates a conversation with a three-year-old child.
Try to stop the madness.
Why?
Why what?
Why?
I don't know why.
Why?
Really, I don't know.
Why?
Please, I don't know.
Why?
Please, stop.
Why?
Please! I'm begging you to stop.
Why?
Because.
Oh. Okay.
Press the enter key to exit._
```

- Odgovarajući kod three_year-old.py je:

```
# Three Year-Old Simulator
# Demonstrates the while loop

print("\tWelcome to the 'Three-Year-Old Simulator'\n")
print("This program simulates a conversation with a three-year-old child.")
print("Try to stop the madness.\n")

response = ""
while response != "Because.":
    response = input("Why?\n")

print("Oh. Okay.")

input("\n\nPress the enter key to exit.")
```

Analiza while petlje

- U navedenom primeru, petlja se sastoji iz dve linije:

```
while response != "Because.":
    response = input("Why?\n")
```

- While petlja liči na if naredbu; kao i kod if, i kod while petlje blok (koji se može nazvati i *telo petlje* -- "loop body") se izvršava kada je zadati uslov ispunjen
- Međutim, razlika je sledeća: kod while, računar konstantno (umesto samo jedanput, kao kod if) proverava uslov i izvršava blok, i tako sve dok uslov ne postane neispunjen, kada se iz petlje izlazi -- zato se ovo zove petlja ("loop")
- Znači, sekvenca je sledeća:
 - proverava uslova: ako je True, odradi blok
 - proverava uslova: ako je True, odradi blok
 - ...
 - proverava uslova: (više) nije True nego False, preskoči blok i nastavi dalje
- Očigledno je da postoji mogućnost da se u poslednju tačku (False vrednost) nikada i ne stigne -- tada se kaže da se program vrti u *beskonačnoj petlji* ("infinite loop"), što znači da se blokirao, što treba izbeći; takođe, može se desiti da je uslov False u startu -- tada se blok ne izvrši nijednom
- Takođe, jasno je da ono šta se dešava prilikom izvršenja bloka mora da ima nekakav uticaj na uslov, inače sam program ne bi imao smisla (i upao bi u beskonačnu petlju) -- u navedenom primeru, vrednost promenljive response se u okviru bloka redovno "osvežava" tako što je korisnik unosi sa tastature, i nova vrednost se pojavljuje u uslovu (to jest poredi sa Because.)
- U navedenom primeru, telo petlje se sastoji od samo jedne naredbe (preuzimanje odgovora korisnika pomoću funkcije input), koja će se izvršavati sve dok on ne unese tekst Because.

- Kada (to jest ako) se unese tekst "Because.", uslov će postati False (jer promenljiva response više nije različita od Because.), i petlja će se okončati, posle čega će se izvršiti sledeća linija koda (print)

Inicijalizacija kontrolne promenljive

- While petlja se kontroliše pomoću *kontrolne promenljive* ("sentry variable", sentry = stražar) -- ta promenljiva se koristi u uslovu i poredi se sa nekom drugom vrednošću; u navedenom primeru, kontrolna promenljiva je response: ona se u uslovu poredi sa stringom "Because.", svaki put pre potencijalnog izvršenja bloka
- Kontrolna promenljiva se mora inicijalizovati; najbolje je da se to uradi neposredno pre same petlje; ako se inicijalizacija ne uradi, program će generisati grešku
- U navedenom primeru, inicijalizacija je:

```
response = ""
```

to jest promenljivoj response je dodeljen prazan string (ne sadrži nijedan karakter); često je praktično da inicijalna vrednost bude jednaka nekakvoj praznoj, ili nultoj, vrednosti; osim praznog stringa, bilo koja druga vrednost za inicijalnu vrednost bi takođe bila odgovarajuća (naravno, izuzev vrednosti "Because.")

Provera kontrolne promenljive

- U principu, programer treba da osigura da uslov while petlje bar jedanput bude True; u suprotnom, blok se nikada neće izvršiti; primer:

```
response = "Because."  
while response != "Because.":  
    response = input("Why?\n")
```

Pošto je response inicijalno jednako vrednosti kojom se proverava petlja, blok se neće nijednom izvršiti (ovo je, naravno, greška programera -- bag)

Ažuriranje kontrolne promenljive

- Pošto je definisan uslov, inicijalizovana kontrolna promenljiva, i pošto je osigurano da se blok (telo petlje) bar jednom izvrši, petlja je operativna; dalje treba osigurati da se petlja jednom završi, to jest da neće biti beskonačna

- Primer:

```
counter = 0
while counter <= 10
    print(counter)
```

rezultat je da se beskonačno ispisuje 0; programer je zaboravio da menja vrednost kontrolne promenljive counter unutar tela petlje (verovatno je hteo da kaže counter = counter + 1)

- Znači, vrednosti u uslovu moraju da se menjaju kao rezultat koda koji se nalazi u telu petlje; ako se ne menjaju, porediće se stalno jedne te iste vrednosti i petlja se neće nikad završiti, to jest postaće beskonačna

3.7 Izbegavanje beskonačnih petlji

- Trivijalan slučaj nastanka beskonačne petlje nastupa kada se kontrolna promenljiva ne ažurira u telu petlje, kao što je pokazano u prethodnom primeru; međutim postoje i neki manje trivijalni slučajevi; naredni primer će pokazati program u kome se ažuriranje obavlja, ali se petlja ipak nikad ne završava

Program Losing Battle

- Program Losing Battle (izgubljena bitka) prati heroja koji se bori protiv zlih trolova -- odgovarajući kod losing_battle-bad.py je:

```
# Losing Battle
# Demonstrates the dreaded infinite loop

print("Your lone hero is surrounded by a massive army of trolls.")
print("Their decaying green bodies stretch out, melting into the horizon.")
print("Your hero unsheathes his sword for the last fight of his life.\n")

health = 10
trolls = 0
damage = 3

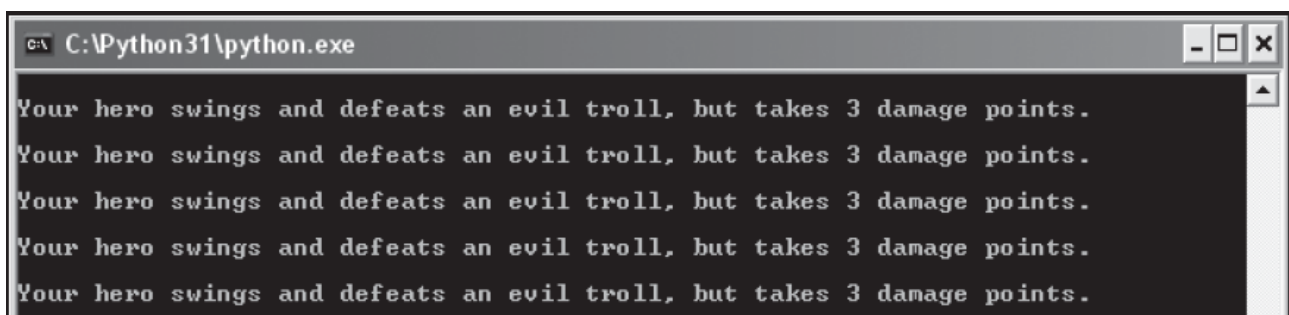
while health != 0:
    trolls += 1
    health -= damage

    print("Your hero swings and defeats an evil troll, " \
          "but takes", damage, "damage points.\n")

print("Your hero fought valiantly and defeated", trolls, "trolls.")
print("But alas, your hero is no more.")

input("\n\nPress the enter key to exit.")
```

- Kada se program pokrene, dobija se beskonačna petlja (Slika 3-9): ona se zaustavlja sa Ctrl+C



Praćenje izvršenja programa korak po korak

- Program očigledno sadrži logičku grešku: najbolji način da se ona pronađe je da se prati izvršenje korak po korak; praćenje ("tracing") znači da programer ručno radi po redu (simulira) isto ono što bi radio računar -- tako će on shvatiti šta se dešava u kojoj naredbi, i naći će grešku
- Najbolji način da se praćenje odradi je na papiru -- pišu se promenljive po kolonama (promenljiva damage se ne menja pa je ne treba ni pisati), kao i uslov:

health	trolls	health != 0
10	0	True
7	1	True
4	2	True
1	3	True
-2	4	True
-5	5	True ...

- Problem je očigledno sa promenljivom health -- ona preskače vrednost 0 (jer se menja po koracima od po 3) i dobija sve negativnije vrednosti, ali uslov i dalje čuva vrednost True -- ona nikada neće postati False, znači stvorena je beskonačna petlja

Zadavanje novog uslova

- Programer mora uvek da osigura da u jednom trenutku uslov dobije vrednost False, kako bi se izašlo iz petlje; u datom primeru, treba odraditi sledeću korekciju u uslovu:

```
while health > 0:
```

- Novi kod je losing_battle-good.py, i daje rezultat kao na Slici 3-10

```
C:\Python31\python.exe
Your lone hero is surrounded by a massive army of trolls.
Their decaying green bodies stretch out, melting into the horizon.
Your hero unsheathes his sword for the last fight of his life.

Your hero swings and defeats an evil troll, but takes 3 damage points.
Your hero swings and defeats an evil troll, but takes 3 damage points.
Your hero swings and defeats an evil troll, but takes 3 damage points.
Your hero swings and defeats an evil troll, but takes 3 damage points.

Your hero fought valiantly and defeated 4 trolls.
But alas, your hero is no more.

Press the enter key to exit._
```

- Praćenje toka programa će dati:

health	trolls	health > 0
10	0	True
7	1	True
4	2	True

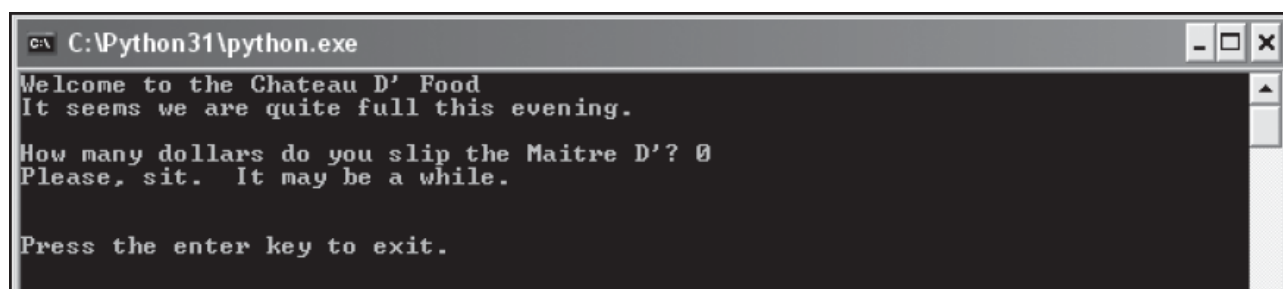
1	3	True
-2	4	False

3.8 Tretiranje vrednosti kao uslova

- Python ima sledeću specifičnost: bilo koja vrednost promenljive bilo kog tipa može biti tretirana/interpretirana kao True ili False; znači, vrednosti 2749, 8.6, "banana" mogu biti interpretirane kao True ili False -- ovo omogućava jednostavno kreiranje uslova

Program Maitre D

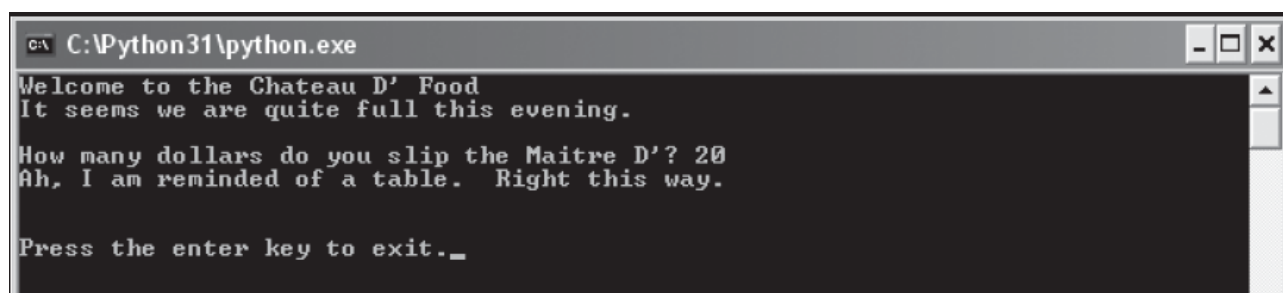
- Program radi sledeće: da bi korisnik dobio mesto u skupom restoranu, treba da podmiti menadžera; korisnik dobije pitanje na ekranu, koliko je mito u USD spreman da da -- ako je mito nula, za mesto će se načekati (Slika 3-11); ako da koji god drugi iznos, dobija mesto (Slika 3-12)



```
C:\Python31\python.exe
Welcome to the Chateau D' Food
It seems we are quite full this evening.

How many dollars do you slip the Maitre D'? 0
Please, sit. It may be a while.

Press the enter key to exit.
```



```
C:\Python31\python.exe
Welcome to the Chateau D' Food
It seems we are quite full this evening.

How many dollars do you slip the Maitre D'? 20
Ah, I am reminded of a table. Right this way.

Press the enter key to exit._
```

- Zanimljivost u vezi sa ovim programom je što se u njemu ne koristi operator poređenja; umesto toga, sama vrednost (iznos u USD) se koristi kao uslov; odgovarajući kod maitre_d.py je:

```
# Maitre D'
# Demonstrates treating a value as a condition

print("Welcome to the Chateau D' Food")
print("It seems we are quite full this evening.\n")

money = int(input("How many dollars do you slip the Maitre D'? "))

if money:
    print("Ah, I am reminded of a table. Right this way.")
else:
    print("Please, sit. It may be a while.")

input("\n\nPress the enter key to exit.")
```

Interpretiranje neke vrednosti kao True ili False

- Objašnjeni koncept je ilustrovan u liniji:

```
if money:
```

u kojoj se promenljiva money ne poredi sa nekom drugom vrednošću, već je ona sama uslov: kada se neki broj koristi kao uslov, vrednost 0 daće False, a svaka druga daće True

- Drugi način da se odradi ovo isto bi bio:

```
if money != 0:
```

ali je gore prikazani način jednostavniji i intuitivniji ("ako ima para")

- Opšte pravilo za određivanje True ili False je jednostavno:

- sve što je prazno ili nula daje False (u slučaju stringa: "")
- sve što je neprazno ili ne-nula daje True

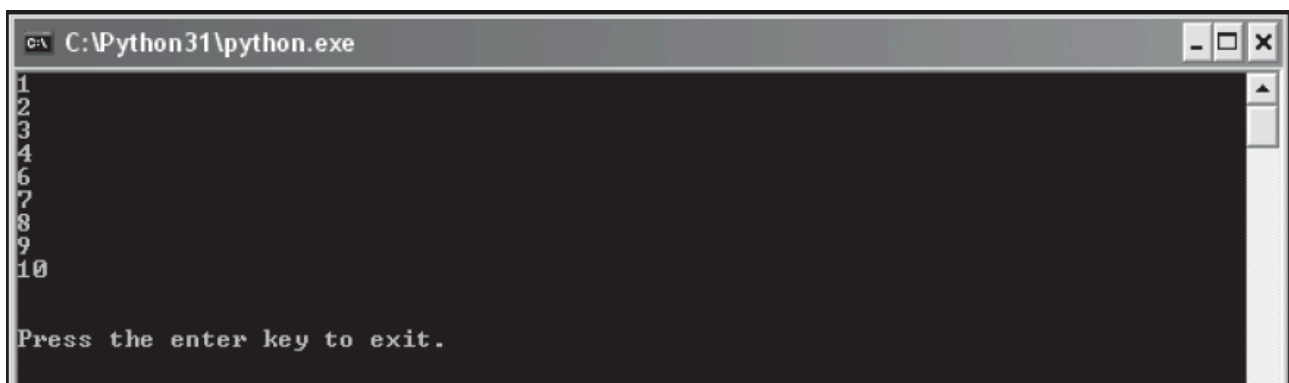
- U slučaju podmitljivog menadžera, ovo znači i da ako korisnik ponudi negativni iznos u USD, ponovo će dobiti mesto (negativni broj je ne-nula)

3.9 Namerno formiranje beskonačnih petlji

- U prethodnom tekstu je pokazano zašto beskonačne petlje treba izbegavati, i to ostaje važeće; ali nekada se koriste *namerne beskonačne petlje* ("intentional infinite loop"); ove petlje u svom telu sadrže uslov za izlaz; zato, formalno gledano, one i nisu beskonačne u bukvalnom smislu, jer se ipak završavaju (ne blokira se računar)

Program Finicky Counter

- Ovaj program broji od 1 do 10 uz pomoć namerne beskonačne petlje; ali on ne voli broj 5 pa ga preskače (finicky = izbirljiv), kao što pokazuje slika 3-13



- Odgovarajući kod finicky_counter.py je:

```
# Finicky Counter
# Demonstrates the break and continue statements

count = 0
while True:
    count += 1
    # end loop if count greater than 10
    if count > 10:
        break
    # skip 5
    if count == 5:
        continue
    print(count)

input("\n\nPress the enter key to exit.")
```

Upotreba naredbe break za izlazak iz petlje

- Petlja je definisana sa:

```
while True:
```

što znači da je ona predviđena da bude beskonačna, izuzev ako je u tome nešto drugo ne spreči

- Ono što će sprečiti da petlja zaista postane beskonačna je sledeći uslov:

```
if count > 10:  
    break
```

count se inkrementira svaki put kada počinje telo petlje, što znači da će na kraju dostići vrednost 11 (>10); kada se to desi, naredba break (= prekini, to jest prekini izvršenje tela petlje) se izvršava i iz petlje se izlazi (ona se završava)

Upotreba naredbe continue za povratak na početak petlje

- Pre nego što se count prikaže na ekranu (pomoću print), nalazi se linija:

```
if count == 5:  
    continue
```

- Naredba continue znači "vрати se na početak petlje", to jest nalaže računaru da se vrati na prvu liniju petlje (prevremeno, bez da odradi linije koje su u telu petlje preostale) -- prva linije je while, znači uslov se proverava i, ukoliko je vrednost True, petlja se ponovo izvršava
- U pomenutom primeru, kada count ima vrednost 5, računar ne izvršava naredbu print, već se vraća na while tako da se vrednost 5 na ekranu uopšte ne ispiše

Određivanje kada koristiti break i continue

- Naredbe break i continue se mogu koristiti u svakoj while petlji, to jest ne samo u namerno beskonačnim; ali one moraju biti korišćene u slučaju prave potrebe -- one zasigurno komplikuju program; zapravo, one formalno nisu ni potrebne -- svaka petlja se može uraditi i bez njih

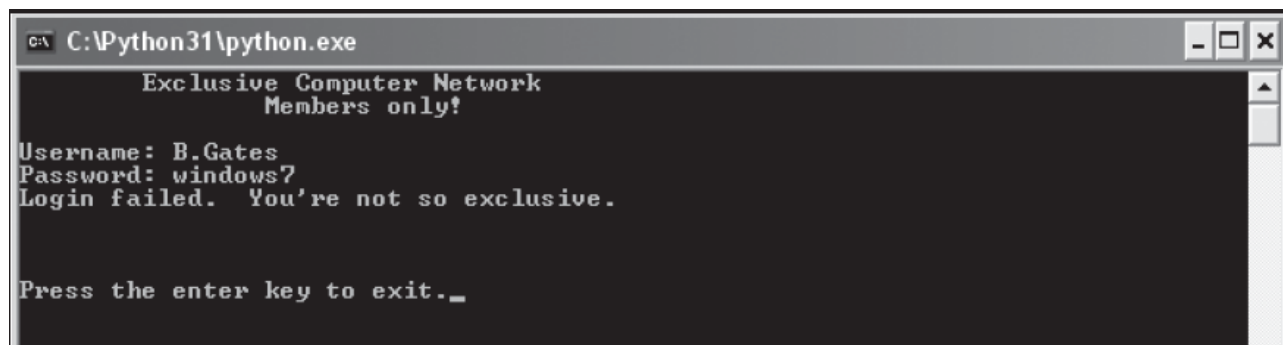
- U principu, uvek treba pokušati sa običnom while petljom; ako to baš nije moguće (ili je suviše složeno), treba koristiti namerno beskonačnu petlju

3.10 Upotreba kombinovanih uslova

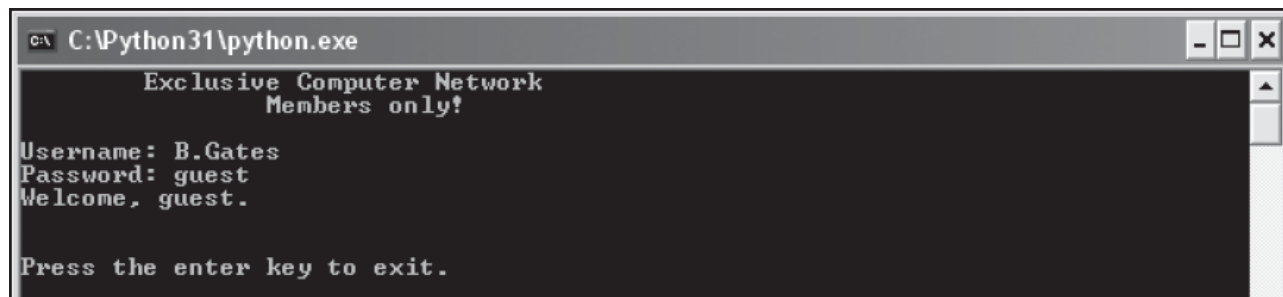
- U dosadašnjim primerima pokazano je kako se obavljaju poređenja između dve vrednosti; ti uslovi se nazivaju *prosti uslovi* ("simple condition"); oni se, pomoću logičkih operatora, mogu kombinovati; tako nastaju *kombinovani uslovi* ("compound condition"), pomoću kojih se može obaviti više poređenja

Program Exclusive Network

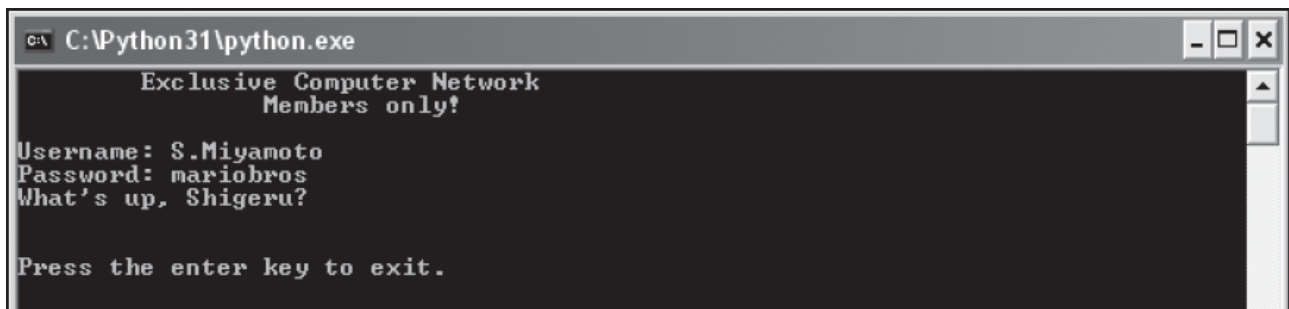
- Program omogućava pristup članovima jednog ekskluzivnog kluba; svako od njih ima svoje posebno korisničko ime i lozinku, pomoću kojih se uloguje; pored toga, pristup, i to samo ograničen -- kao gosti, imaju i osobe koje nisu članovi, ako kao korisničko ime ili kao lozinku otkucaju "guest"; ostalima je pristup uskraćen; takođe, svako ko se uloguje dobija neki nivo sigurnosti ("security level") -- Slike 3-14, 3-15 i 3-16



```
C:\Python31\python.exe
Exclusive Computer Network
Members only!
Username: B.Gates
Password: windows?
Login failed. You're not so exclusive.
Press the enter key to exit._
```



```
C:\Python31\python.exe
Exclusive Computer Network
Members only!
Username: B.Gates
Password: guest
Welcome, guest.
Press the enter key to exit.
```



```
C:\Python31\python.exe
Exclusive Computer Network
Members only!
Username: S.Miyamoto
Password: mariobros
What's up, Shigeru?
Press the enter key to exit.
```

- Odgovarajući kod `exclusive_network.py` je:

```
# Exclusive Network
# Demonstrates logical operators and compound conditions

print("\tExclusive Computer Network")
print("\t\tMembers only!\n")

security = 0

username = ""
while not username:
    username = input("Username: ")

password = ""
while not password:
    password = input("Password: ")

if username == "M.Dawson" and password == "secret":
    print("Hi, Mike.")
    security = 5
elif username == "S.Meier" and password == "civilization":
    print("Hey, Sid.")
    security = 3
elif username == "S.Miyamoto" and password == "mariobros":
    print("What's up, Shigeru?")
    security = 3
elif username == "W.Wright" and password == "thesims":
    print("How goes it, Will?")
    security = 3
elif username == "guest" or password == "guest":
    print("Welcome, guest.")
    security = 1
else:
    print("Login failed. You're not so exclusive.\n")

input("\n\nPress the enter key to exit.")
```

- U realnosti, u slučaju potrebe za sličnim programom, programer nikada ne bi pisao korisnička imena i lozinke direktno u kod; za te potrebe on bi koristio neki tip DBMS (database management system) koji omogućava organizaciju i pristup sličnim podacima

Logički operator not

- Kako bi programer osigurao da će korisnik da unese neki tekst za korisničko ime i lozinku, to jest da neće pritisnuti samo Enter i tako uneti prazan string, kreirao je while petlju koja se izvršava sve dok korisnik ne unese bilo kakav tekst:

```
username = ""  
while not username:  
    username = input("Username: ")
```

(analogno je i za lozinku)

- Pored naredbe while, upotrebljen je logički operator not, koji označava "ne", to jest obrnuto od navedenog; preciznije, not ispred nekog uslova znači novi uslov koji izračunava obrnutu/kontra vrednost od navedenog uslova; u ovom slučaju: not username je True kada je username False, i obrnuto; petlja znači bukvalno "sve dok nema korisničkog imena..."

- Ovo je ilustrovano na sledeći način:

username	not username
True	False
False	True

- Username je inicijalizovan kao prazan string (""), to znači da startuje kao False, što znači da je not username True, tako da je uslov petlje ispunjen i ona se prvi put izvrši, i čeka se na odgovor korisnika:

- ako on pritisne samo Enter, username ostaje prazan string i petlja se ponovi izvrši
- tek kad/ako on unese neki neprazan string (i naravno posle toga Enter), username postaje True, znači not username postaje False, i petlja se završava

Logički operator and

- Da bi se član ekskluzivnog kluba ulogovao, treba da unese ispravne i korisničko ime i lozinku (ako bar jedno nije ispravno uneto, pristup je uskraćen); na primer, program

proverava da li je korisnik Sid Meier otkucao ispravno i svoje korisničko ime i svoju lozinku pomoću sledeće linije:

```
elif username == "S.Meier" and password == "civilization":
```

- Prethodna linija očigledno sadrži kombinovani uslov napravljen od dva prosta; oni su iskombinovani pomoću logičkog operatora and (logičko "i") -- dobijeni kombinovani uslov je takođe uslov, znači može biti ili True ili False

- Navedeni kombinovani uslov je True ako i samo ako (akko) su oba prosta uslova takođe True; u svim ostalim slučajevima (kojih ima 3), kombinovani uslov je False; ovo je poznato iz matematičke logike

- Ovo je ilustrovano na sledeći način:

username == "S.Meier"	password == "civilization"	username == "S.Meier" and password == "civilization"
True	True	True
True	False	False
False	True	False
False	False	False

- Znači, tek kad pomenuti član unese ispravne i korisničko ime i lozinku, on dobija pozdravnu poruku i nivo sigurnosti

- Ovo radi i za sve ostale članove kluba (o gostima u nastavku teksta) -- pomoću if-elif-else, program proverava 4 postojeća člana kluba; ako korisnik ne otkuca ispravne korisničko ime ili lozinku, na ekranu izlazi poruka "Login failed"

Logički operator or

- Kao što je pomenuto, pristup je dozvoljen i gostima; da bi se neko prihvatio kao gost, potrebno je da unese tekst guest kao korisničko ime ili kao lozinku -- za to je zadužena linija:

```
elif username == "guest" or password == "guest":
```

- U ovom slučaju je upotrebljen logički operator or (logičko "ili"): kombinovani uslov formiran pomoću tog operatora je True kada je bar jedan od prostih uslova True (mogu naravno biti i oba); on je False jedino kada su oba False; i ovo je poznato iz logike

- Ovo je ilustrovano na sledeći način:

<code>username == "guest"</code>	<code>password == "guest"</code>	<code>username == "guest" or password == "guest"</code>
True	True	True
True	False	True
False	True	True
False	False	False

- Konačno, navedeni logički operatori (not, and, or) se mogu proizvoljno kombinovati u cilju generisanja složenijih kombinovanih uslova

3.11 Planiranje programa

- Svi dosadašnji programi su bili vrlo jednostavni; ozbiljniji programi moraju se planirati, i to formalno, na papiru; bez plana programer može očekivati samo probleme; postoji posebno polje softverskog inženjerstva koje se zove *planiranje programa* ("program planning") koje se time bavi

Pisanje algoritama pomoću pseudokoda

- Algoritam je konačna lista jasnih i jednostavnih instrukcija, datih korak po korak, koje služe da se obavi neki zadatak; on se kreira pre pisanja samog programa; pošto se on završi, programski kod se piše prema njemu
- Primer: algoritam kako postati milioner

```
ako možeš da osmisliš neki novi i korisan proizvod
    onda je to tvoj proizvod
inače
    prepakuj neki postojeći proizvod tako da postane tvoj proizvod
napravi reklamu za proizvod
prikaži reklamu na tv
proizvod naplaćuj 100 EUR
prodaj 10000 komada proizvoda
```

- Algoritmi se najčešće pišu u takozvanom pseudokodu; on se nalazi negde između ljudskog jezika i programskog jezika -- on je razumljiv svakome, ali izgleda pomalo i kao program; na primer, prve četiri linije iz prethodnog primera podsećaju na if-else konstrukciju

Rafiniranje algoritma

- Kao i svaki plan, nijedan algoritam ne može biti gotov (spreman za implementaciju, to jest kodiranje) posle prvog pisanja; najbolje je početi u grubim crtama, pa onda u svakom od narednih koraka taj algoritam rafinirati, u smislu da se doda još preciznih detalja; tako se na kraju dobije niz jednostavnih koraka, koji se mogu direktno (linija u liniju) prevesti u programski kod

- Primer: korak iz prethodnog algoritma

```
napravi reklamu za proizvod
```

je suviše neodređen -- nije precizirano kako se tačno pravi reklama

- Prethodni korak se rafinira na sledeći način:

```
napiši scenario za reklamu za proizvod
iznajmi tv studio na jedan dan
iznajmi ekipu za snimanje reklama
iznajmi publiku koja će se oduševljavati
snimi reklamu
```

ako je potrebno, svaki od ovih koraka se može dalje rafinisati

3.12 Povratak na igru Guess My Number

- Igra Guess My Number kombinuje sve što je predstavljeno u ovom poglavlju, i predstavlja njegovu rekapitulaciju

Planiranje programa

- Pseudokod je:

```
izaberi slučajan broj
sve dok igrač ne pogodi broj
    pusti igrača neka pogađa
čestitaj igraču
```

ovo je gruba verzija koja predstavlja osnovne ideje; nedostaje sledeće: program treba igraču da kaže da li je broj koji je odigrao veći ili manji od traženog; program treba da broji pokušaje da bi igrača na kraju obavestio iz kog pokušaja je pogodio

- Rafinirani pseudokod je:

```
poželi igraču dobrodošlicu i objasni igru
izaberi slučajni broj između 1 i 100
traži od igrača da pogađa
postavi broj pokušaja na 1
    sve dok igračev broj nije jednak traženom broju
        ako je igračev broj veći od traženog broja
            kaži igraču da bira manji broj
        inače
            kaži igraču da bira veći broj
        traži od igrača da pogađa
        povećaj broj pokušaja za 1
čestitaj igraču na pogotku
kaži igraču koliko pokušaja mu je trebalo
```

- Zatim se pseudokod pretvara (implementira) u Python kod; odgovarajući kod guess_my_number.py je:

```

# Guess My Number
#
# The computer picks a random number between 1 and 100
# The player tries to guess it and the computer lets
# the player know if the guess is too high, too low
# or right on the money

import random

print("\tWelcome to 'Guess My Number'!")
print("\nI'm thinking of a number between 1 and 100.")
print("Try to guess it in as few attempts as possible.\n")

# set the initial values
the_number = random.randint(1, 100)
guess = int(input("Take a guess: "))
tries = 1

# guessing loop
while guess != the_number:
    if guess > the_number:
        print("Lower...")
    else:
        print("Higher...")

    guess = int(input("Take a guess: "))
    tries += 1

print("You guessed it! The number was", the_number)
print("And it only took you", tries, "tries!\n")

input("\n\nPress the enter key to exit.")

```

- Program počinje komentarima (#); zatim učitava random modul (import); zatim na ekranu ispisuje poruke (print)
- Inicijalne vrednosti:
 - the_number je traženi broj (koji se pogađa), slučajno odabran
 - zatim se traži igračev prvi (inicijalni) broj; int() pretvara uneti string u ceo broj, i to postaje promenljiva guess
 - promenljiva tries, koja označava broj pokušaja do tog trenutka, je postavljena na 1
- Petlja se "vrti" sve dok igrač ne pogodi broj: igračev broj se poredi sa traženim, pa se u skladu sa tim on obaveštava da li treba da poveća ili da smanji svoj broj; on unosi novi broj, a broj pokušaja se inkrementira

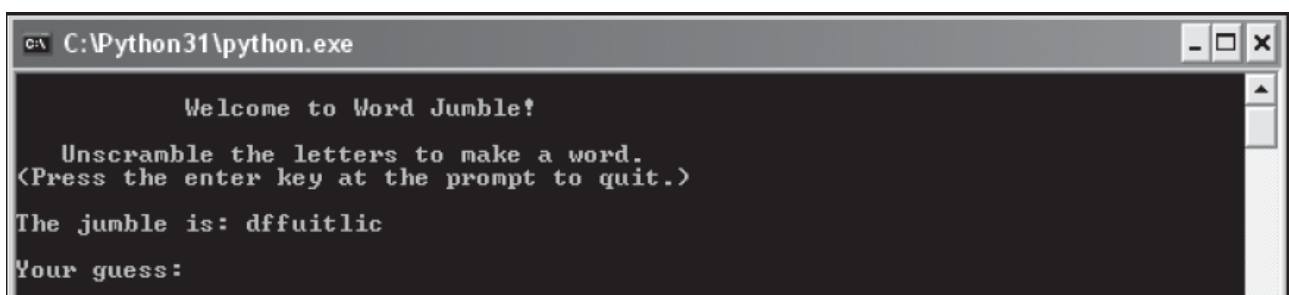
- Kada igrač pogodi, to jest kada je njegov broj jednak traženom, uslov petlje postaje False i ona se završava; ispisuje se poruka i broj pokušaja; na kraju se čeka Enter da se igra završi

Poglavlje 4: for petlja, stringovi i n-torke

- U prethodnim poglavljima je prikazano kako se promenljive koriste za pristup i obradu podataka; međutim, kako se nekom programu povećavaju dužina i složenost, broj promenljivih se takođe povećava, i voditi računa o svima njima postaje sve složeniji posao
- Ovo poglavlje će uvesti sekvence podataka, i zatim uvesti jedan poseban tip sekvence, zvani n-torka, koji omogućava da se podacima pristupa u okviru uređenih grupa; pokazaće se kako string, tip koji je već ranije objašnjen, može biti interpretiran kao sekvenca; biće uvedena posebna vrsta petlje koja je namenjena za rad sa sekvencama
- U poglavlju će biti pokazano:
 - petlja kojom se "kreće" kroz sekvencu
 - funkcija kojom se kreira sekvenca
 - tretiranje stringova kao sekvenci
 - korišćenje n-torki da se iskoriste pogodnosti sekvenci
 - korišćenje funkcija i operatora za rad sa sekvencama
 - indeksiranje i isecanje sekvenci

4.1 Igra Word Jumble

- Igra Word Jumble koristiće većinu materije koja će biti prikazana u ovom poglavlju --
Slika 4.1



```
C:\Python31\python.exe

Welcome to Word Jumble!

Unscramble the letters to make a word.
(Press the enter key at the prompt to quit.)
The jumble is: dffuitlic
Your guess:
```

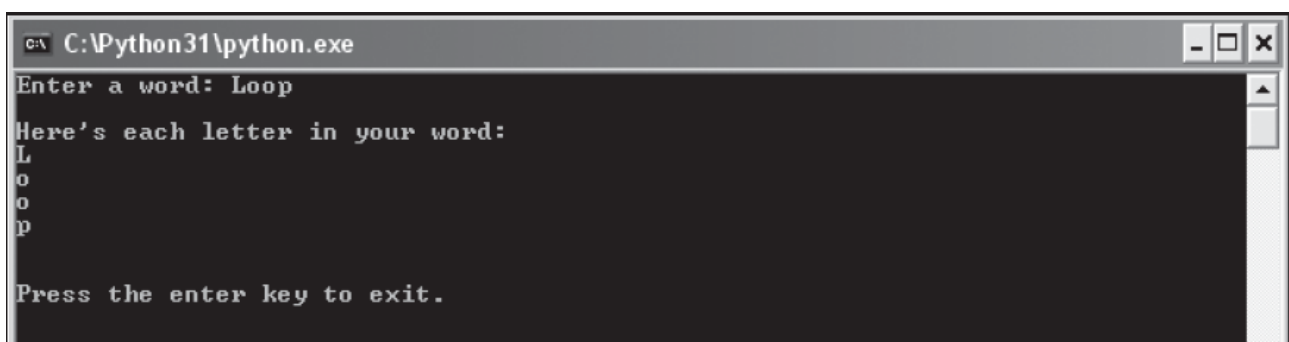
- Računar bira slučajnu reč iz grupe reči, i "ispretura" joj redosled slova tako da se nađu u slučajnom poretku -- korisnik treba da pogodi o kojoj se reči radi (primer: fteelno = telefon)

4.2 For petlja

- U prethodnom poglavlju predstavljena je while petlja: ona omogućava ponavljanje dela nekog koda (koji se naziva telo petlje) u zavisnosti od nekog uslova; sve dok je uslov ispunjen, telo se ponavlja
- Sada uvodimo for petlju: ona omogućava ponavljanje dela koda, ali ne na osnovu uslova (kao while), već na osnovu sekvence ("sequence"), to jest grupe (skupa) nekih elemenata u nekom poretku ($e_1, e_2, \dots, e_N$); sekvenca je na primer spisak nečijih 10 omiljenih filmova
- For petlja ponavlja ("vrti") odgovarajuće telo petlje redom za svaki element sekvence; kada stigne na kraj sekvence, petlja se završava; u prethodnom primeru sa filmovima, petlja bi išla kroz sekvencu (spisak) filmova, redom jedan po jedan, i na primer ispisivala svaki od njih

Program Loopy String

- Program Loopy String uzima reč koju je korisnik uneo, i štampa njena slova, redom u linijama jedno ispod drugog -- Slika 4.2



```
C:\Python31\python.exe
Enter a word: Loop
Here's each letter in your word:
L
o
o
p
Press the enter key to exit.
```

- Odgovarajući programski kod `loopy_string.py` je:

```
# Loopy String
# Demonstrates the for loop with a string

word = input("Enter a word: ")

print("\nHere's each letter in your word:")
for letter in word:
    print(letter)

input("\n\nPress the enter key to exit.")
```

Opis for petlje

- U prikazanom kodu novitet je petlja `for`:

```
for letter in word:
    print(letter)
```

- Svaka sekvenca se sastoji od elemenata; string je sekvenca čiji su elementi karakteri: u primeru stringa (sekvence) "Loop", karakter "L" je prvi element, i tako redom do četvrtog
- Osnovno pravilo za `for` petlju je sledeće: ona prolazi/iterira kroz sekvencu redom, element po element: prvi, pa drugi, pa treći, i tako dalje do poslednjeg (posle koga se završava); svaki od ovih prolazaka naziva se *iteracija*
- U navedenom primeru, `for` petlja prolazi kroz string (poseban tip sekvence) "Loop" uzimajući redom karakter po karakter (to jest, element po element navedene sekvence) od prvog do poslednjeg, posle čega se završava
- Pri tom prolasku, `for` petlja koristi promenljivu (koja se formalno naziva promenljiva petlje -- "loop variable") kojoj se u svakoj iteraciji dodeljuje aktuelni element (onaj na koga je došao red) te sekvence; u ovom primeru, promenljiva je `letter` koja redom (po iteracijama) uzima vrednosti "L", pa "o", pa "o", pa "p"

- U okviru tela petlje, obavlja se neka operacija sa tom promenljivom; u primeru, program ispisuje na ekranu promenljivu letter (to jest aktuelni karakter); promenljiva petlje se u kodu tretira kao i svaka druga promenljiva; ako pre petlje nije postojala (kao u primeru), ona se automatski kreira

- Znači, konstrukcija petlje je:

for P in S:

gde je S sekvenca nekih elemenata, a P promenljiva petlje, koja redom preuzima vrednosti elemenata iz te sekvence

- U slučaju petlje iz primera:

- kada petlja započne, promenljiva letter se automatski kreira

- u prvoj iteraciji, letter dobija vrednost prvog elementa zadate sekvence (to jest prvog karaktera datog stringa), to jest "L"

- zatim se u telu petlje ovaj karakter ispisuje na ekranu

- kada se prva iteracija završi, obavi se povratak na vrh petlje (naredbu for) i počinje druga iteracija: letter postaje "o", i tako redom dok se sekvenca ne iscrpi

- Većina programskih jezika ima for petlju; međutim, te petlje imaju veoma ograničenu upotrebu -- one uglavnom sadrže brojačku promenljivu (brojač = "counter"), kojoj mora biti dodeljena neka numerička vrednost; prilikom svake iteracije petlje, ovaj brojač se menja za istu vrednost

- Primer iz Visual Basic-a -- linija:

For i = 1 To 9 Step 2

generisaće redom sledeće vrednosti i: 1,3,5,7,9

- Python ima mogućnost da petlju izvršava direktno u sekvenci, što je mnogo fleksibilnije; na primer (kao što je prikazano), obrada stringa postaje vrlo jednostavna ("za svako slovo u reči")

Kreiranje for petlje

- For petlja se kreira kao što je pokazano u primeru: posle naredbe for ide promenljiva petlje koja će redom u iteracijama uzimati vrednosti elemenata, zatim ide odredba in, posle čega ide sekvenca kroz koju će petlja da prolazi/iterira, posle čega ide dvotačka

4.3 Brojanje pomoću for petlje

- Nekada se u toku pisanja programa ukaže potreba da se nešto broji; za to se koristi funkcija range() u kombinaciji sa for petljom

Program Counter

- Program Counter (brojač) pokazuje tri primera brojanja uz upotrebu funkcije range() --

Slika 4.3



```
C:\Python31\python.exe
Counting:
0 1 2 3 4 5 6 7 8 9
Counting by fives:
0 5 10 15 20 25 30 35 40 45
Counting backwards:
10 9 8 7 6 5 4 3 2 1
Press the enter key to exit.
```

- Odgovarajući kod counter.py je:

```
# Counter
# Demonstrates the range() function

print("Counting:")
for i in range(10):
    print(i, end=" ")

print("\n\nCounting by fives:")
for i in range(0, 50, 5):
    print(i, end=" ")

print("\n\nCounting backwards:")
for i in range(10, 0, -1):
    print(i, end=" ")

input("\n\nPress the enter key to exit.\n")
```

- U slučaju brojačke petlje, promenljiva petlje se naziva *brojačka promenljiva*; programerska praksa nalaže izuzetak da se kao oznaka brojačke promenljive koriste slova i/j/k umesto deskriptivnih imena (kao kod običnih petlji)

Brojanje unapred

- U prvoj petlji obavlja se brojanje unapred

```
print("Counting:")
for i in range(10):
    print(i, end=" ")
```

u slučaju ove for petlje, sekvenca kroz koju se prolazi je povratna vrednost funkcije range() kojoj je zadat argument 10

- Funkcija range() znači generiše sekvencu brojeva; ako se funkciji zada pozitivni ceo broj N, ona će vratiti sekvencu brojeva, sa korakom 1, koja počinje nulom, a završava se prvim manjim brojem od zadatog, to jest N-1; primer: range(10) generiše i vraća sekvencu [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
- Znači u slučaju pomenute petlje, u prvoj iteraciji/prolasku brojačka promenljiva i dobija vrednost 0, u drugoj 1, i tako redom sve do poslednje u kojoj dobija vrednost 9

Brojanje po pet

- U drugoj petlji obavlja se brojanje u koracima po 5 uz pomoć sledeće linije:

```
for i in range(0, 50, 5):
```

- Ako se funkciji range() zadaju tri argumenta (vrednosti), ona će prvi uzeti za prvu vrednost sekvence, drugi za poslednju, i treći za korak brojanja; prva vrednost se nalazi u sekvenci ali poslednja ne; znači: range(prva,poslednja,korak)

- U ovom primeru, generisana sekvenca je [0, 5, 10, 15, 20, 25, 30, 35, 40, 45]; u slučaju da je potrebno uključiti i 50, poziv bi mogao da bude

```
for i in range(0, 51, 5):
```

Brojanje unazad

- U trećoj petlji postiže se brojanje unazad pomoću linije:

```
for i in range(10, 0, -1):
```

- U ovom slučaju, treći argument (korak) je -1, znači da se od početne do krajnje vrednosti ide tako što se vrednosti smanjuju za 1 -- generisana sekvenca je [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]; kao i obično, krajnja vrednost se ne pojavljuje; početna vrednost mora biti veća od krajnje

- Vrednost brojačke promenljive nije obavezno koristiti u telu petlje; na primer, ako jedno isto treba ponoviti više puta, brojačka petlja se kreira ali se njena promenljiva u telu ne pojavljuje; primer: za ispisivanje teksta "Hi!" deset puta na ekranu, kod je sledeći:

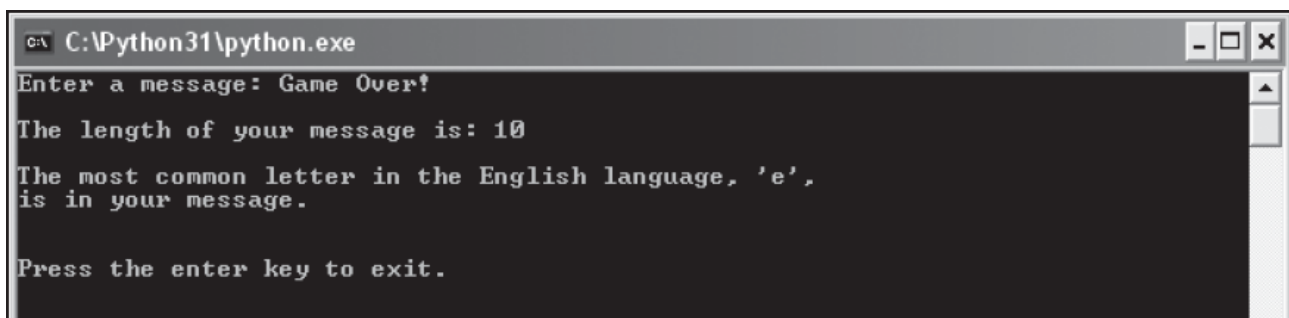
```
for i in range(10):  
    print("Hi!")
```

4.4 Operatori i funkcije za rad sa stringovima kao sekvencama

- Kao što je pomenuto, string je tip sekvence čiji su elementi pojedinačni karakteri; Python raspolaže operatorima i funkcijama koji rade sa svakim tipom sekvence, uključujući i stringove; pomoću njih se određuju neke važne karakteristike sekvence, kao što su koliko je dugačka (koliko sadrži elemenata) i da li sadrži određeni element

Program Message Analyzer

- Program Message Analyzer analizira poruku (tekst) koju korisnik unese: koliko je dugačka, i da li sadrži najčešće korišćeno slovo u engleskom, slovo e -- Slika 4.4



- Odgovarajući kod, message_analyzer.py, je:

```
# Message Analyzer
# Demonstrates the len() function and the in operator

message = input("Enter a message: ")

print("\nThe length of your message is:", len(message))

print("\nThe most common letter in the English language, 'e',")
if "e" in message:
    print("is in your message.")
else:
    print("is not in your message.")

input("\n\nPress the enter key to exit.")
```

Upotreba funkcije len()

- Funkcija len() prihvata kao argument sekvencu, i vraća dužinu ("length") te sekvence; dužina neke sekvence je broj elemenata koji je čine; u navedenom primeru, string (to jest

sekvenca karaktera) message sadrži 10 karaktera (računaju se i razmak i uzvičnik), što znači da ta sekvenca sadrži 10 elemenata, to jest vraća se vrednost 10

Upotreba operatora in

- Program koristi if naredbu da odredi da li se slovo "e" nalazi u unetoj poruci:

```
if "e" in message:
    print("is in your message.")
else:
    print("is not in your message.")
```

i prema tome ispisuje odgovarajuću poruku

- Uslov u if naredbi se proverava pomoću operatora in (= u):

```
"e" in message
```

to jest ako message sadrži slovo "e" uslov je True (što važi u ovom primeru), a ako ne onda je False

- Operator in se koristi uvek da se proverí da li se neki element sadrži u sekvenci: treba znači napisati element, zatim in, i zatim sekvencu -- tako se generiše uslov; ako se element sadrži onda je uslov True, a ako ne onda je False

4.5 Indeksiranje stringova

- Pomoću for petlje, kao što je pokazano, vrši se prolazak kroz string, redom karakter po karakter; ovo se naziva *sekvencijalni pristup* ("sequential access"); u generalnom slučaju, ovo znači da se kroz neku sekvencu prolazi redom, element po element, bez preskakanja od prvog do poslednjeg
- Za razliku od ovog pristupa, postoji *slučajni pristup* ("random access"), u kome se svakom elementu sekvence (to jest svakom karakteru stringa) pristupa direktno; to se

obavlja pomoću *indeksiranja* ("indexing") -- specificira se pozicija, to jest redni broj (koji se naziva *indeks*) elementa u sekvenci, i njemu se pomoću indeksa pristupa direktno

- Napomena: ovde se ne radi o slučajnim/nasumičnim procesima (kao kod generisanja slučajnih brojeva); ispravniji termin na srpskom bi možda bio "proizvoljan pristup" (ili "direktan pristup"), ali pridev "slučajan" se univerzalno koristi u srpskoj literaturi

Program Random Access

- Program Random Access koristi indeksiranje kako bi se direktno pristupilo nekom karakteru u stringu: program slučajno/nasumično bira poziciju karaktera u stringu "index", i ispisuje taj karakter i odgovarajuću poziciju; ovo se odvija 10 puta -- Slika 4.5



```
C:\Python31\python.exe
The word is:  index
word[-3 ]      d
word[ 3 ]      e
word[-4 ]      n
word[ 2 ]      d
word[ 3 ]      e
word[-3 ]      d
word[ 0 ]      i
word[-5 ]      i
word[-5 ]      i
word[-1 ]      x

Press the enter key to exit.
```

- Odgovarajući kod random_access.py je:

```
# Random Access
# Demonstrates string indexing

import random

word = "index"
print("The word is: ", word, "\n")

high = len(word)
low = -len(word)

for i in range(10):
    position = random.randrange(low, high)
    print("word[" + position + "]\t", word[position])

input("\n\nPress the enter key to exit.")
```

Rad sa pozitivnim indeksima

- Pomoću linije:

```
word = "index"
```

generiše se sekvenca (string) pod nazivom word; njen svaki element (karakter) ima svoju poziciju u sekvenci, to jest redni broj, to jest indeks: prvo slovo, "i", nalazi se na poziciji 0 (računari obično broje počev od nule), "n" na poziciji 1, i tako redom do 4

- Da bi se pristupilo karakteru na poziciji 0 (to jest sa indeksom 0), piše se word[0], i tako redom do word[4]; kada bi korisnik tražio word[5], dobio bi poruku o grešci ("string index out of range")

Rad sa negativnim indeksima

- Sa stringovima se može raditi na dva načina:
 - sa pozitivnim indeksima (kao što je već pokazano): referenca je početak sekvence (stringa), i broji se od početka nadesno, i to počev od 0
 - sa negativnim indeksima: referenca je kraj sekvence (stringa), i broji se od kraja nalevo, i to počev od -1 (0 je već rezervisana)
- Tako se u primeru poslednji karakter "x" nalazi na poziciji -1 (isti onaj koji se nalazio na pozitivnoj poziciji 4), preposlednji "e" na poziciji -2, i tako redom do -5; negativno indeksiranje je uvedeno za slučaj kada je programeru lakše da referencu postavi na kraj stringa

- Pristup pomoću pozitivnih i negativnih indeksa je ilustrovan na Slici 4.6

0	1	2	3	4
i	n	d	e	x
-5	-4	-3	-2	-1

Pristup slučajnom karakteru stringa

- Da bi se pristupilo slučajnom (tačnije: proizvoljnom) karakteru stringa, u programu se prvo uveze modul random; dalje, program omogućava da se slučajno generiše neki validni indeks, pozitivan ili negativan -- u slučaju stringa "index", validne vrednosti indeksa se nalaze, kao što je pokazano, između -5 i 4; program će odabrati jednu vrednost iz tog opsega

- Da se to uradi, prvo se određuju dve krajnje vrednosti:

```
high = len(word)
low = -len(word)
```

U ovom primeru, high postaje 5 a low postaje -5 -- tako se kreiraju poslednji i prvi validni indeksi

- Zatim se poziva funkcija random.randrange() kojoj se zadaju dve dobijene vrednosti:

```
random.randrange(low, high)
```

Kao što je ranije pokazano, ova funkcija generiše slučajan broj između low i high (to jest u ovom primeru između -5 i 5), s tim što je low uključen ($\leq$) a high nije ($<$); to je tačno ono šta u ovom slučaju odgovara -- u primeru, generisaće se jedan od brojeva -5, -4, -3, -2, -1, 0, 1, 2, 3, 4

- Konačno, sve se odvija u for petlji koja se, kao što je ranije pokazano, izvrši 10 puta -- bira se slučajni karakter i ispisuje njegova pozicija/indeks

4.6 Nepromenljivost stringova

- Sekvence mogu biti:

- promenljive ("mutable") -- može im se menjati vrednost, i
- nepromenljive ("immutable") -- vrednost im je uvek jedna te ista

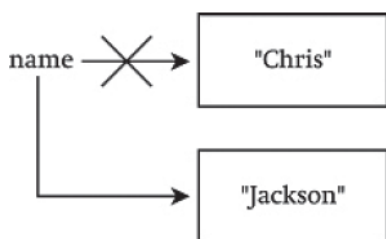
- Stringovi, kao poseban tip sekvenci, spadaju u nepromenljive sekvence, znači ne mogu se menjati: string "Game Over! " će uvek biti string "Game Over! "

- Ovo deluje na prvi pogled neobično; na primer: ako se odradi sledeća interaktivna sesija:

```
>>> name = "Chris"
>>> print(name)
Chris
>>> name = "Jackson"
>>> print(name)
Jackson
```

izgledaće da je string promenjen iz "Chris" u "Jackson", ali to nije tačno: prvo je kreiran string "Chris" i dodeljen je promenljivoj name, a zatim je kreiran string "Jackson" i dodeljen je istoj promenljivoj name

- Drugim rečima, oba stringa su različiti i nepromenljivi; ono što se promenilo je vrednost promenljive name: njoj je dodeljen jedan pa onda drugi -- ilustracija na Slici 4.7



- Kao posledicu navedene nepromenljivosti, pomoću indeksiranja nije moguće promeniti neki karakter u stringu; postoje načini da se kreiraju novi stringovi od postojećih (biće pokazani kasnije), ali na ovako direktan način ne

- Primer: interaktivna sesija u kojoj se prvom karakteru stringa hoće da "dodeli" nova vrednost:

```
>>> word = "game"
>>> word[0] = "l"
```

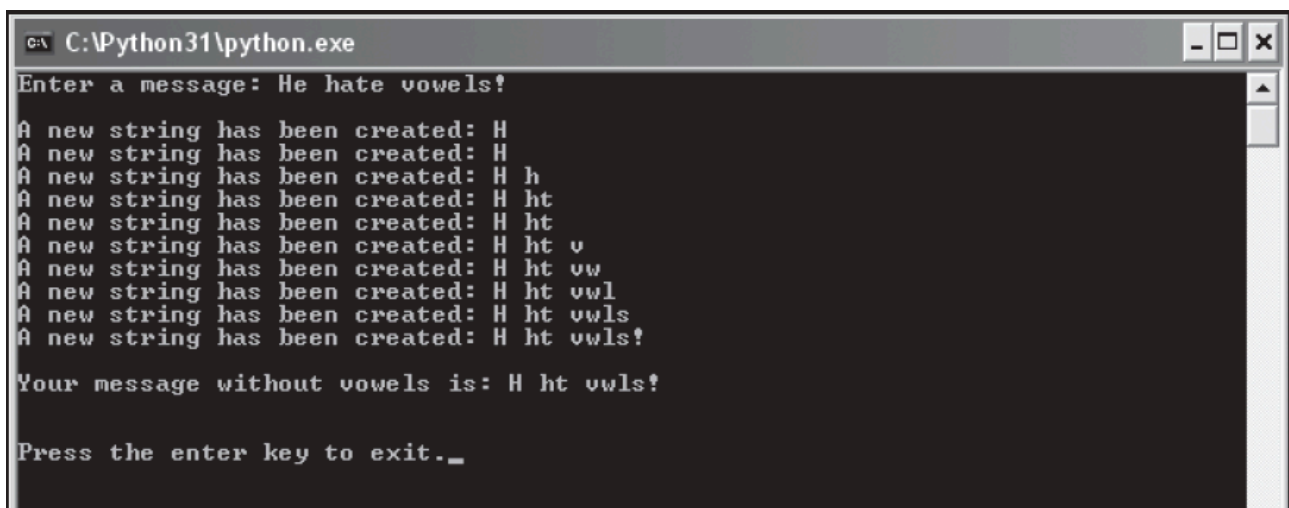
će proizvesti grešku: nije moguće promeniti "game" u "lame"

4.7 Kreiranje novog stringa

- Već je pokazano kako se nadovezuju dva stringa pomoću operatora +; isti operator se koristi kada treba kreirati neki string karakter po karakter; pošto su stringovi nepromenljivi, svaki put kada se primeni operator + se zapravo kreira novi string

Program No Vowels

- Program No Vowels (bez samoglasnika) uzima neki tekst/poruku od korisnika, iz njega izbacuje sve samoglasnike, i ispisuje dobijeni tekst -- Slika 4.8



```
C:\Python31\python.exe
Enter a message: He hate vowels!
A new string has been created: H
A new string has been created: H
A new string has been created: H h
A new string has been created: H ht
A new string has been created: H ht
A new string has been created: H ht v
A new string has been created: H ht vw
A new string has been created: H ht vwl
A new string has been created: H ht vwls
A new string has been created: H ht vwls!
Your message without vowels is: H ht vwls!
Press the enter key to exit._
```

- Ono što program zapravo radi je kreiranje serije novih stringova (koje prikazuje samo radi ilustracije) -- odgovarajući kod no_vowels.py je:

```
# No Vowels
# Demonstrates creating new strings with a for loop

message = input("Enter a message: ")
new_message = ""
VOWELS = "aeiou"

print()
for letter in message:
    if letter.lower() not in VOWELS:
        new_message += letter
        print("A new string has been created:", new_message)

print("\nYour message without vowels is:", new_message)

input("\n\nPress the enter key to exit.")
```

Definisanje konstanti

- Pošto program preuzme tekst od korisnika, i kreira novi tekst (koji je inicijalno prazan), on kreira string koji sadrži sve samoglasnike:

```
VOWELS = "aeiou"
```

- Ime promenljive je u ovom slučaju napisano velikim slovima, što odstupa od svih dosadašnjih promenljivih koje su bile ispisane samo malim slovima: u Pythonu, po konvenciji, kada je ime promenljive napisano svim velikim slovima, to znači da je ona konstanta, to jest vrednost koja se neće menjati (primer: $\pi = 3.141592$)
- Programera ništa naravno ne sprečava da vrednost takve promenljive promeni kasnije u kodu, ali je to sasvim u suprotnosti sa dobrom praksom: smatra se da je toj promenljivoj dodeljena jedna određena vrednost (najbolje na početku programa), i da joj se druge vrednosti u toku programa više neće dodeljivati

Kreiranje novog stringa od postojećeg

- Najvažniji deo koda je petlja u kojoj program kreira novi tekst, sa izbačenim samoglasnicima:

```
for letter in message:
    if letter.lower() not in VOWELS:
        new_message += letter
    print("A new string has been created:", new_message)
```

- U svakoj iteraciji petlje, računar proverava naredno slovo u originalnom tekstu:
 - ako nije samoglasnik, to slovo se dodaje u tekst koji se kreira
 - ako jeste samoglasnik, ne radi se ništa i ide na naredno slovo

- Preciznije rečeno: ako nije samoglasnik, on se nadovezuje (+) na kraj do tada kreiranog novog teksta (i istovremeno se ispisuje novonastali tekst, sa novim dodatim slovom)

- Kao što je poznato, Python pravi razliku između malih i velikih slova; pošto konstanta VOWELS sadrži samo mala slova, moraju se mala slova i proveravati, zbog čega se svako slovo iz teksta pretvara u malo -- za to služi `letter.lower()`

- Pri poređenju stringova ovo pretvaranje u mala slova se uvek radi; na primer, ako se od korisnika očekuje neki odgovor, ako odgovori "Yes" ili "YES", od toga se mora napraviti "yes"

- Još jedan primer: da bi se uporedila dva stringa, `name` i `winner`, uz uslov da je nebitno da li su slova velika ili mala, odgovarajući kod je:

```
name.lower() == winner.lower()
```

Uslov će biti True ako su na primer stringovi Larry i larry, ili LARRY i larRry

- I jedna napomena: pokazano je ranije da operator `+=` radi sa brojevima; on radi i sa stringovima (za njihovo nastavljajanje), tako da je linija:

```
new_message += letter
```

isto što i linija:

```
new_message = new_message + letter
```

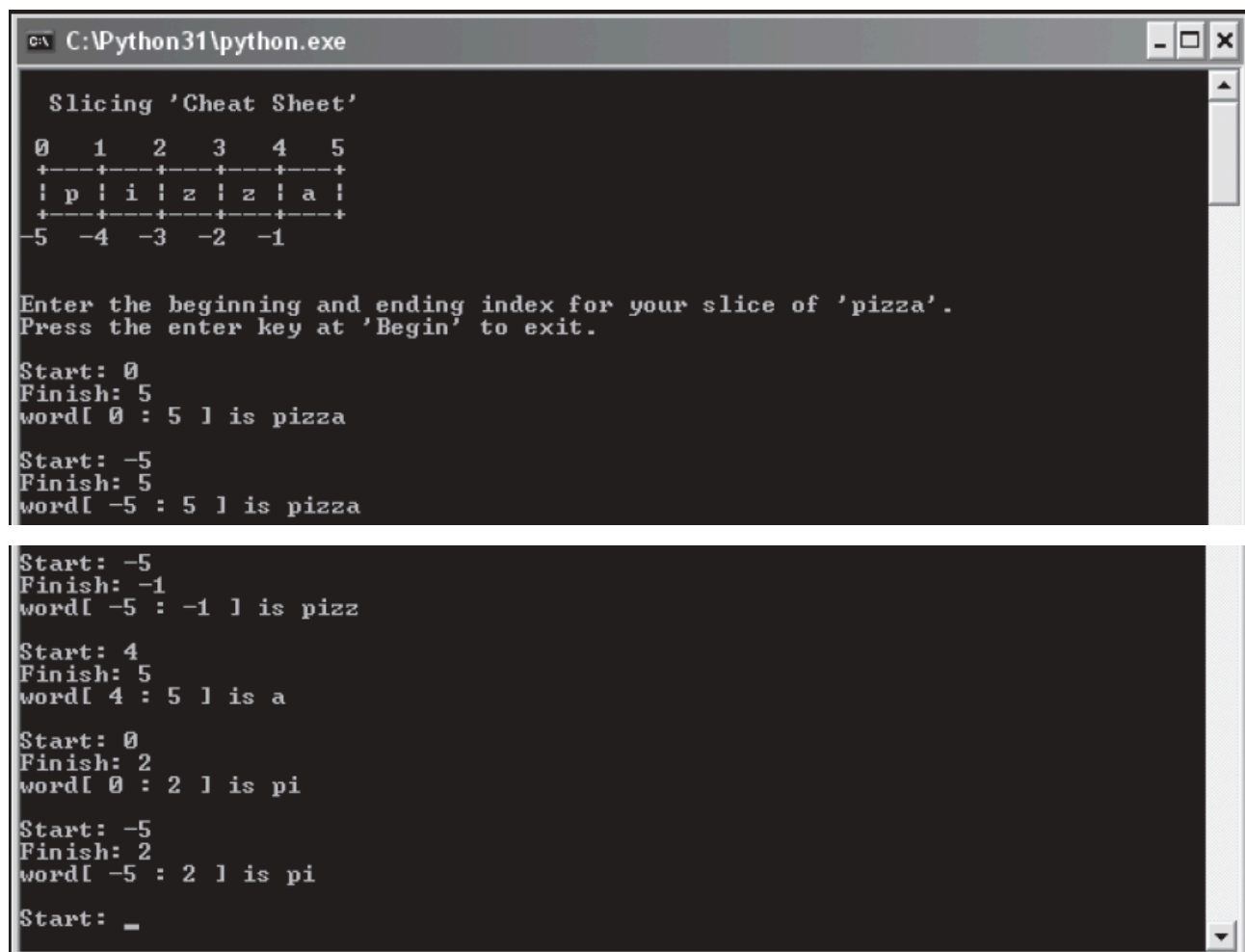
4.8 Isecanje stringova

- Indeksiranje je vrlo korisna tehnika, ali je ograničena u smislu da omogućava rad samo sa jednim po jednim elementom sekvence (to jest karakterom stringa)

- Pored indeksiranja, postoji i tehnika koja se zove *isecanje* ("slicing"), koja omogućava rad sa skupom od nekoliko susednih elemenata sekvence istovremeno -- na primer, poslednja tri elementa; taj skup se naziva *isečak* ("slice")
- Može se isecati sve u opsegu od jednog elementa (što se svodi na indeksiranje), pa sve do svih elemenata, to jest cele sekvence (što je trivijalan slučaj)

Program Pizza Slicer

- Program Pizza Slicer omogućava da se string "pizza" iseca po korisnikovoj želji: on unese početni i krajnji indeks isečka, i program mu prikaže rezultat -- Slika 4.9



```

C:\Python31\python.exe

Slicing 'Cheat Sheet'
0  1  2  3  4  5
+--+--+--+--+
| p | i | z | z | a |
+--+--+--+--+
-5 -4 -3 -2 -1

Enter the beginning and ending index for your slice of 'pizza'.
Press the enter key at 'Begin' to exit.

Start: 0
Finish: 5
word[ 0 : 5 ] is pizza

Start: -5
Finish: 5
word[ -5 : 5 ] is pizza

Start: -5
Finish: -1
word[ -5 : -1 ] is pizz

Start: 4
Finish: 5
word[ 4 : 5 ] is a

Start: 0
Finish: 2
word[ 0 : 2 ] is pi

Start: -5
Finish: 2
word[ -5 : 2 ] is pi

Start: _

```

- Odgovarajući kod pizza_slicer.py je:

```

# Pizza Slicer
# Demonstrates string slicing

word = "pizza"

print(
    """
    Slicing 'Cheat Sheet'

    0   1   2   3   4   5
    +---+---+---+---+---+
    | p | i | z | z | a |
    +---+---+---+---+---+
   -5  -4  -3  -2  -1

    """
)

print("Enter the start and finish index for your slice of 'pizza'.")
print("Press the enter key at 'start' to exit.")

start = None
while start != "":
    start = (input("\nStart: "))

    if start:
        start = int(start)

        finish = int(input("Finish: "))

        print("word[", start, ":", finish, "] is", end=" ")
        print(word[start:finish])

input("\n\nPress the enter key to exit.")

```

Vrednost None

- Linija:

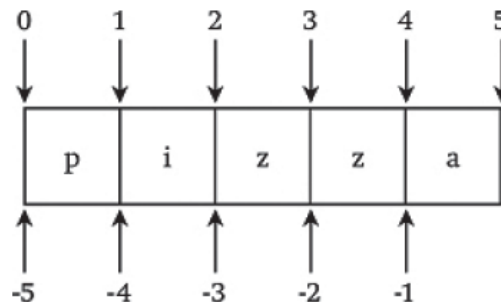
```
start = None
```

dodeljuje jednu specijalnu vrednost, zvanu None, promenljivoj start: pomoću None, Python označava "ništa" (to jest "ne postoji"); kada se ono tretira kao uslov (pomoću if), daje False (jer ga "nema"); ovde se None koristi da se promenljiva start inicijalizuje za upotrebu u while petlji, jer mora imati neku početnu vrednost kada krene petlja

- Vrednost None ne odgovara ničemu, pa čak ni praznom stringu "", niti brojnoj vrednosti 0, jer pomenute dve vrednosti postoje, a None ne

Tehnika isecanja

- Isecanje je slično indeksiranju, samo što umesto jedne pozicije (koja je predstavljala indeks) ovde treba dati dve: početnu i krajnju; svi elementi između ove dve pozicije bivaju uključeni u isečak
- Slika 4.10 prikazuje string "pizza" sa odgovarajućim pozicijama označenim strelicama ("Cheat sheet"); kao i kod indeksiranja, mogu se koristiti i negativne vrednosti



- Način numerisanja pozicija je različit od onoga koji je prikazan za indeksiranje na Slici 4.6 -- ovde se kao pozicije ne numerišu karakteri, već "prostor" između njih; tako je pitanje koji je početni a koji krajnji element isečka rešeno bez ikakvih dilema
- Da se odradi isecanje, piše se ime sekvence (to jest stringa), i zatim početna i krajnja pozicija u uglastim zagradama, razdvojene dvotačkom: `sekvenca[poč_poz:kraj_poz]`
- Primer za to daje sledeća interaktivna sesija:

```
>>> word = "pizza"
>>> print(word[0:5])
pizza
>>> print(word[1:3])
iz
>>> print(word[-4:-2])
iz
>>> print(word[-4:3])
iz
```

Mogu se mešati i pozitivne i negativne vrednosti, pravilo ostaje isto: sve što je između dve pozicije (pozitivne ili negativne), preciznije -- desno od početne a levo od krajnje, postaje isečak

- Ako se kao početna i krajnja pozicija zadaju takve vrednosti kod koje je početna desno od krajnje (kao na primer [2:1]), Python neće dati poruku o grešci već će vratiti praznu sekvencu (string)

Kreiranje isečaka

- U okviru petlje, program ispisuje sintaksu naredbe za kreiranje isečka:

```
print("word[" + start + ":", finish + "] is", end=" ")
```

i zatim ispisuje sam isečak:

```
print(word[start:finish])
```

Prečice za isecanje

- Prilikom isecanja postoje sledeće tri "prečice" (skraćeni/nepotpuni zapis):

1) ako se izostavi početna pozicija, smatra se da se ona nalazi levo od prvog elementa (karaktera) -- u navedenom primeru, word[:4] je prečica za word[0:4]

2) ako se izostavi krajnja pozicija, smatra se da se ona nalazi desno od poslednjeg elementa (karaktera) -- u navedenom primeru, word[2:] je prečica word[2:5]

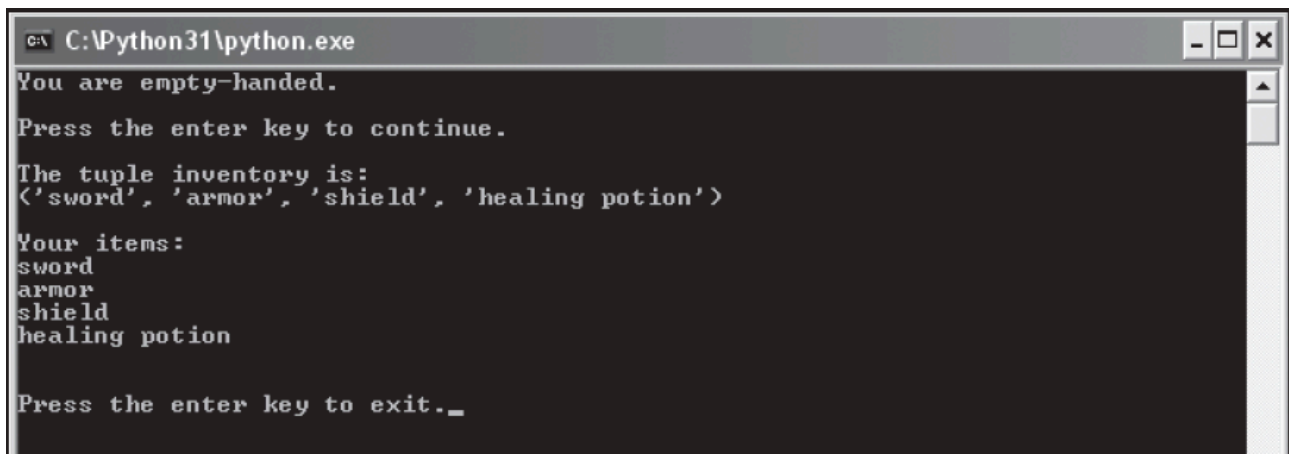
3) ako se izostave obe pozicije, smatra se da se one nalaze levo od prvog i desno od poslednjeg elementa (karaktera), to jest da se uzima cela sekvenca (string) -- u navedenom primeru, `word[]` je prečica za `word[0:5]`

Kreiranje n-torke

- *N-torka* ("tuple") je jedan tip sekvence, kao i string; za razliku od stringa, koji kao elemente sadrži samo pojedinačne karaktere, n-torka može sadržati elemente proizvoljnog tipa; naziv n-torka se čuva nezavisno od brojne vrednosti *n*, to jest *n* se ne menja konkretnom vrednošću kao u govornom jeziku (na primer, trojka)
- Tipovi elemenata koji čine n-torku ne moraju biti jednaki: na primer, n-torka može sadržati nekoliko brojnih vrednosti ili nekoliko stringova, ali može sadržati i brojne vrednosti i stringove
- Usput: tipovi ne moraju biti samo brojne vrednosti i stringovi; mogu biti i fajlovi: sve što može biti dodeljeno nekoj promenljivoj može se grupisati zajedno i čuvati kao n-torka

Program Hero's Inventory

- Program Hero's Inventory se bavi "imovinom" junaka jedne igre (to jest svime šta poseduje): ta imovina je predstavljena u obliku n-torke: ona sadrži stringove, od kojih svaki označava nešto što junak poseduje; junak kreće bez ičega, ali kasnije stiče imovinu (mač, oklop, štit, lekoviti napitak) -- Slika 4.11



```
C:\Python31\python.exe
You are empty-handed.
Press the enter key to continue.
The tuple inventory is:
(<'sword', 'armor', 'shield', 'healing potion')
Your items:
sword
armor
shield
healing potion
Press the enter key to exit._
```

- Odgovarajući kod hero's_inventory.py je:

```
# Hero's Inventory
# Demonstrates tuple creation

# create an empty tuple
inventory = ()

# treat the tuple as a condition
if not inventory:
    print("You are empty-handed.")

input("\nPress the enter key to continue.")

# create a tuple with some items
inventory = ("sword",
            "armor",
            "shield",
            "healing potion")

# print the tuple
print("\nThe tuple inventory is:")
print(inventory)

# print each element in the tuple
print("\nYour items:")
for item in inventory:
    print(item)

input("\n\nPress the enter key to exit.")
```

Kreiranje prazne n-torke

- N-torka se kreira tako što se sekvenca nekih vrednosti, odvojenih zarezima, upiše u zagradu: (vred_1, vred_2,...); prazna n-torka je jednostavno (), kao što je u programu

inicijalno dodeljeno promenljivoj inventory (što označava da junak u početku nema ništa od imovine)

N-torka kao uslov

- Ranije je pokazano da se svaka vrednost u Pythonu može tretirati kao uslov; to je slučaj i sa n-torkom; u kodu je to urađeno u linijama:

```
if not inventory:  
    print("You are empty-handed.")
```

- Pravilo za uslov je sledeće: prazna n-torka je False; ona koja sadrži bar jedan element je True; u primeru, inventory je inicijalno prazna znači False, to jest not inventory je True, pa se ispisuje poruka da je junak praznih ruku

Kreiranje n-torke koja sadrži elemente

- U narednim linijama, junaku se uvećava imovina, to jest n-torka inventory dobija elemente koji su stringovi:

```
inventory = ("sword",  
            "armor",  
            "shield",  
            "healing potion")
```

- Ova dodela je izvršena kao što je ranije objašnjeno: elementi n-torke (u ovom slučaju četiri stringa) su odvojeni zarezima; tako string "sword" postaje prvi element, i tako redom
- Prethodna naredba je napisana u više linija -- Python ovo dozvoljava, i to može da se radi, ali je preporučljivo da se ide sa nekoliko elemenata, a ne samo jednim, u jednom redu

Ispisivanje n-torke

- N-torka se na ekranu ispisuje na uobičajeni način, pomoću linije:

```
print(inventory)
```

Pristup elementima n-torke u okviru for petlje

- Na kraju programa, pomoću for petlje pristupa se svakom elementu (promenljiva item) n-torke inventory pojedinačno, i svaki se ispisuje na ekranu:

```
for item in inventory:  
    print(item)
```

- Ova petlja izgleda kao ona koja je prikazana za rad sa stringovima; na ovaj način se radi sa elementima bilo koje sekvence
- Drugi programski jezici imaju strukture podataka slične n-torci, kao što su nizovi ili vektori; ali oni najčešće mogu sadržati samo promenljive istog tipa, što mnogo smanjuje fleksibilnost

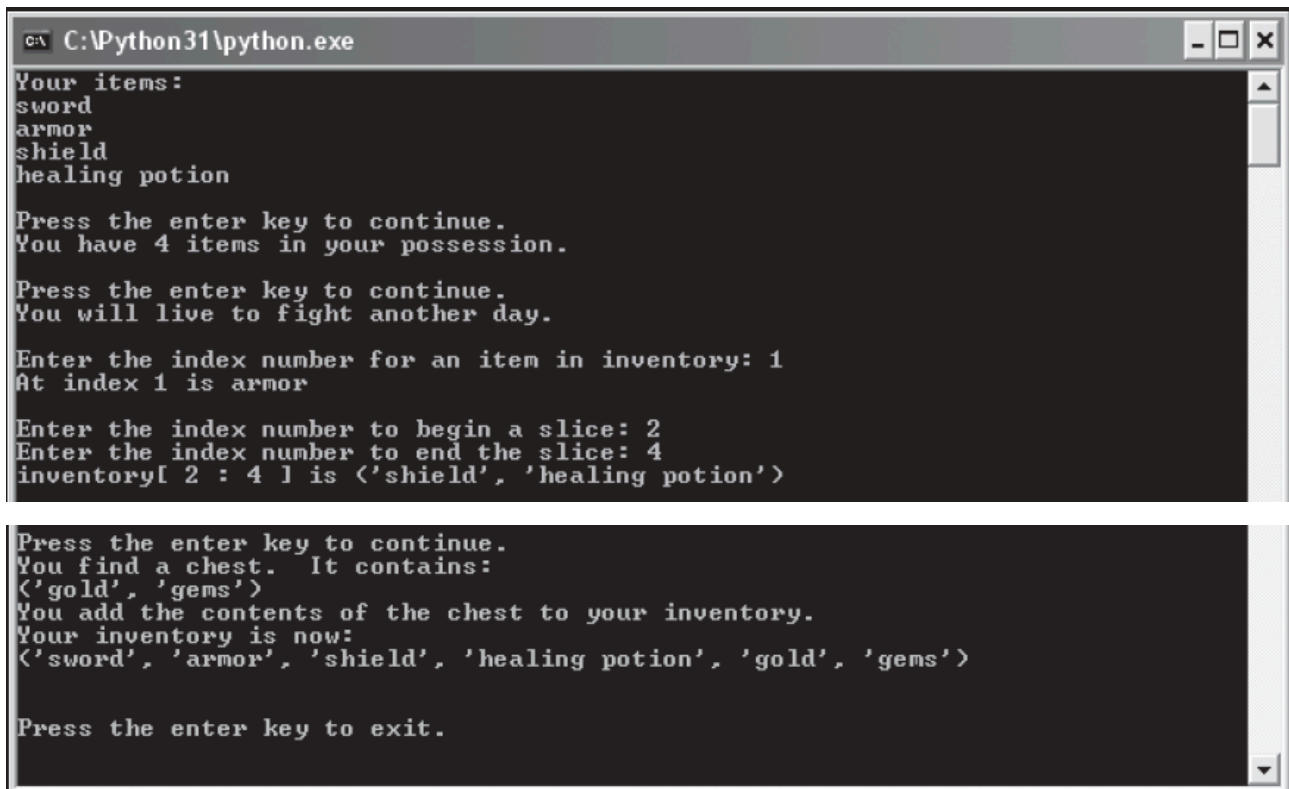
4.9 Upotreba n-torki

- Pošto je n-torka samo jedna od vrsta sekvence, sve što je dosad pokazano o sekvencama (to jest stringovima) važi i za n-torke: može se određivati dužina n-torke, pristupati elementima pomoću for petlje, i pomoću in operator testirati da li je nešto element n-torke; za n-torke takođe važi indeksiranje, isecanje i nadovezivanje

Program Hero's inventory 2.0

- U ovom programu, junak nastavlja svoje putovanje: njegova imovina se broji, proverava, indeksira i iseca; junak će naći i kovčeg (predstavljen drugom n-torkom) sa nekim

dragocenim predmetima; pomoću nadovezivanja n-torki, junakova imovina će postati bogatija za predmete koje je našao u kovčegu -- Slika 4.12



```
C:\Python31\python.exe
Your items:
sword
armor
shield
healing potion

Press the enter key to continue.
You have 4 items in your possession.

Press the enter key to continue.
You will live to fight another day.

Enter the index number for an item in inventory: 1
At index 1 is armor

Enter the index number to begin a slice: 2
Enter the index number to end the slice: 4
inventory[ 2 : 4 ] is <'shield', 'healing potion'>

Press the enter key to continue.
You find a chest. It contains:
('gold', 'gems')
You add the contents of the chest to your inventory.
Your inventory is now:
('sword', 'armor', 'shield', 'healing potion', 'gold', 'gems')

Press the enter key to exit.
```

- Odgovarajući kod, hero's_inventory2.py, biće predstavljen deo po deo: prvi deo koda je kao u verziji 1

```
# Hero's Inventory 2.0
# Demonstrates tuples

# create a tuple with some items and display with a for loop
inventory = ("sword",
             "armor",
             "shield",
             "healing potion")
print("Your items:")
for item in inventory:
    print(item)

input("\nPress the enter key to continue.")
```

Upotreba funkcije len() sa n-torkama

- Funkcija `len()` radi sa n-torkama isto kao što je pokazano sa stringovima: `len` pa naziv n-torke u zagradi vraća broj elemenata n-torke; prazna n-torka (kao i svaka prazna sekvenca) ima dužinu 0

- Ovo se obavlja u delu koda:

```
# get the length of a tuple
print("You have", len(inventory), "items in your possession.")

input("\nPress the enter key to continue.")
```

što daje rezultat 4

Upotreba operatora `in` sa n-torkom

- Kao što je pokazano sa stringovima, operator `in` se koristi da bi se proverilo da li je nešto element n-torke (provera pripadnosti) -- ovo se radi u delu koda:

```
# test for membership with in
if "healing potion" in inventory:
    print("You will live to fight another day.")
```

Indeksiranje n-torki

- Indeksiranje se takođe radi kao što je pokazano sa stringovima: indeks (pozicija) se stavlja u uglaste zagrade, da bi se pristupilo pojedinačnom elementu

- U odgovarajućem delu koda, korisnik unosi indeks a program vraća odgovarajući element imovine:

```
# display one item through an index
index = int(input("\nEnter the index number for an item in inventory: "))
print("At index", index, "is", inventory[index])
```

- I n-torka se može prikazati (kao što je ranije pokazano sa stringom), da bi se ilustrovalo indeksiranje, kao na Slici 4.13

0	1	2	3
"sword"	"armor"	"shield"	"healing potion"
-4	-3	-2	-1

Isecanje n-torke

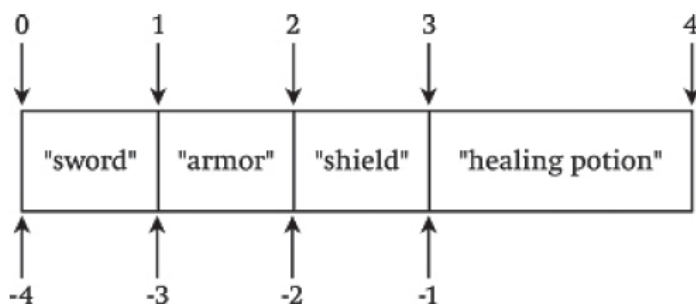
- Isecanje se radi kao što je pokazano sa stringovima -- zadaju se početna i krajnja pozicija, a vraća se n-torka koja sadrži sve elemente između te dve pozicije

- U odgovarajućem delu koda, korisnik unosi te dve pozicije a program vraća odgovarajući isečak:

```
# display a slice
start = int(input("\nEnter the index number to begin a slice: "))
finish = int(input("Enter the index number to end the slice: "))
print("inventory[" + start + ":", finish + "] is", end=" ")
print(inventory[start:finish])

input("\nPress the enter key to continue.")
```

- I n-torka se može prikazati (kao što je ranije pokazano sa stringom), da bi se ilustrovalo isecanje, kao na Slici 4.14



Nepromenljivost n-torke

- Kao i stringovi, i n-torke su nepromenljive ("immutable"), znači ne mogu se menjati u kodu pomoću dodele

- To je pokazano u narednoj interaktivnoj sesiji u kojoj je pokušano sa dodelom:

```
>>> inventory = ("sword", "armor", "shield", "healing potion")
>>> inventory[0] = "battleax"
```

posle koje se dobija poruka o grešci -- n-torka se ne može menjati; ono što se može uraditi je kreiranje nove od postojeće

Nadovezivanje n-torki

- N-torke se nadovezuju ("concatenate") isto kao i stringovi, pomoću operatora +, što je prikazano u delu koda:

```
# concatenate two tuples
chest = ("gold", "gems")
print("You find a chest. It contains:")
print(chest)
print("You add the contents of the chest to your inventory.")
inventory += chest
print("Your inventory is now:")
print(inventory)

input("\n\nPress the enter key to exit.")
```

- Prvo se kreira nova n-torka, chest (=kovčeg), koja sadrži dva elementa, "gold" i "gems" (= zlato i drago kamenje); zatim se ona ispisuje na ekranu; na kraju, na n-torku inventory se nadovezuje n-torka chest

4.10 Povratak na igru Word Jumble

- Odgovarajući kod je word_jumble.py, i biće prikazan deo po deo

Pripremne radnje

- Posle početnih komentara, učitava/uvozi se random modul:

```
import random
```

- Zatim se kreira n-torka WORDS kao sekvenca reči sa kojima će se igrati; ona se u programu tretira kao konstanta:

```
# create a sequence of words to choose from
WORDS = ("python", "jumble", "easy", "difficult", "answer", "xylophone")
```

- Zatim se pomoću funkcije random.choice() slučajno izabere jedna reč, koja se dodeljuje promenljivoj word, iz n-torke (sekvence) WORDS:

```
# pick one word randomly from the sequence
word = random.choice(WORDS)
```

- Promenljiva word je ona reč koju će igrač pokušati da pogodi (pošto mu se zada sa ispreturanim redosledom slova); uvodi se još jedna promenljiva, correct, koja je inicijalno jednaka word; ona će kasnije biti korišćena da se proverí da li je igrač pogodio reč, jer će word biti menjano tokom izvršenja:

```
# create a variable to use later to see if the guess is correct
correct = word
```

Planiranje "preturanja" redosleda slova

- "Preturanje" redosleda slova u zadatoj/izabranoj/traženoj reči, to jest formiranje nove reči sa "ispreturanim" redosledom slova (kraći naziv: ispreturane reči), je najvažniji deo programa; pre nego što se pređe na kodiranje, predstaviće se pseudokod za "preturanje":

```
kreiraj praznu ispreturanu reč
sve dok zadata reč sadrži slova
    izbriši slučajno odabrano slovo iz zadate reči
    dodaj slučajno izabrano slovo u ispreturanu reč
```

- Na primer, ako je zadata reč "easy":
 - prvo slučajno odabrano slovo je "y" -- ono se briše iz zadate reči a dodaje u ispreturanu, pa zadata postaje "eas", a ispreturana (koja je inicijalno bila prazna) postaje "y"
 - drugo slučajno slovo je "a", čime zadata reč postaje "es" a ispreturana "ya"
 - i tako redom sve dok se iscrpu još dva preostala slova

- Pseudokod je konceptualno tačan, ali ignoriše neke detalje koji će prilikom kodiranja biti veoma važni:
 - iz stringa se ne može izbrisati slovo, jer su stringovi nepromenljivi; ali se može kreirati novi string koji je jednak datom stringu bez tog slova
 - u string se ne može dodati slovo, jer su stringovi nepromenljivi; ali se može kreirati novi string tako što se na dati string nadoveže to slovo

Kreiranje prazne ispreturane reči

- Program kreira prazan string i dodeljuje ga promenljivoj jumble, koja označava ispreturanu reč:

```
# create a jumbled version of the word
jumble = ""
```

Definisanje petlje

- Ispreturana reč se kreira u okviru while petlje -- uslov petlje je veoma jednostavan:

```
while word:
```

Petlja će se odvijati sve dok word ne postane prazan string ("sve dok postoji word"): svaki put kada se petlja izvrši, računar kreira novu verziju promenljive word, sa jednim slovom manje; tako će na kraju word postati prazan string i petlja će se završiti

Generisanje slučajne pozicije (indeksa) u promenljivoj/stringu word

- Prva linija u telu petlje generiše slučajnu poziciju (position) u stringu word, na osnovu njegove dužine:

```
position = random.randrange(len(word))
```

tako da je slovo/karakter word[position] ono koje će biti "izbrisano" iz reči word, i biti "dodato" ispreturanoj reči jumble

Kreiranje nove verzije jumble

- Naredna linija kreira novu verziju stringa jumble: on postaje jednak svojoj staroj vrednosti, na koju je nadovezano slovo word[position]:

```
jumble += word[position]
```

Kreiranje nove verzije word

- Naredna linija pravi, uz pomoć isecanja, novu verziju stringa word; ona se sastoji od stare verzije "minus" slovo na poziciji/indeksu position:

```
word = word[:position] + word[(position + 1):]
```

- Detaljnije objašnjenje: pomoću isecanja, od word se kreiraju dva nova stringa-isečka:
 - prvi isečak, word[:position], se sastoji od svih slova pre onog koje se izbacuje,
 - drugi isečak, word[(position + 1):], se sastoji od svih slova posle onog koje se izbacuje,i na kraju se ova dva isečka nadovežu, čime je pomenuto slovo izbačeno

Dobrodošlica igraču

- Pošto je kreirana ispreturana reč, sledeći deo programa želi igraču dobrodošlicu, i prikazujemo mu ispreturanu reč:

```
# start the game
print(
    """
        Welcome to Word Jumble!

        Unscramble the letters to make a word.
        (Press the enter key at the prompt to quit.)
    """
)
print("The jumble is:", jumble)
```

Uzimanje odgovora od igrača

- Program zatim traži od igrača da unese svoj odgovor; on to nastavlja da radi sve dok igrač ili ne unese tačan odgovor, ili dok ne pritisne Enter da završi igru:

```
guess = input("\nYour guess: ")
while guess != correct and guess != "":
    print("Sorry, that's not it.")
    guess = input("Your guess: ")
```

- Drugim rečima: uslov da se petlja izvršava je da igrač još nije pogodio (to jest, odgovor koga je dao nije tačan), i da igrač nije pritisnuo Enter (to jest, nije uneo prazan string)

Čestitanje igraču

- U ovoj fazi izvršenja, igrač je ili pogodio ili prekinuo: ako je pogodio, dobija čestitku, a ako nije, ne dešava se ništa:

```
if guess == correct:
    print("That's it! You guessed it!\n")
```

Kraj igre

- Na kraju, program se zahvaljuje igraču i završava:

```
print("Thanks for playing.")
input("\n\nPress the enter key to exit.")
```

Poglavlje 5: Liste i rečnici

- N-torke su struktura podataka pogodna za rad sa sekvencama proizvoljnog tipa, ali njihova nepromenljivost ih ograničava; postoji drugi tip sekvence -- lista, koja omogućava sve što i n-torka i još ponešto više; te dodatne funkcionalnosti postoje jer su liste promenljive -- mogu im se dodavati i oduzimati elementi, ili izvršiti njihovo sortiranje
- Pored lista, postoji i struktura podataka koji se naziva rečnik; oni barataju sa parovima podataka; kao i u običnim rečnicima, može se zadati jedan podatak da bi se dobio drugi (parnjak)
- U ovom poglavlju biće pokazano:
 - kreiranje, indeksiranje i isecanje liste
 - dodavanje i brisanje elementa liste
 - sortiranje liste
 - korišćenje ugnježdenih sekvenci da se predstave još složeniji podaci
 - korišćenje rečnika pri radu sa parovima podataka
 - dodavanje i brisanje odrednice u rečniku

5.1 Igra Hangman

- Pravila igre Hangman (Slike 5.1 do 5.3) su:
 - računar bira i zadaje igraču tajnu reč
 - igrač treba da pogodi tu reč, tako što pogađa slovo po slovo
 - svaki put kada igrač promaši slovo (to jest unese ono koje se ne sadrži u traženoj reči), računar napravi novu sliku vešala na kojima visi čovečuljak
 - svaka nova slika sadrži jednu crticu (karakter) više u odnosu na prethodnu
 - krajnja slika, koja se prikaže ako igrač konačno ne pogodi iz datog broja pokušaja, je obešen čovečuljak


```
C:\Python31\python.exe

Your items:
sword
armor
shield
healing potion

Press the enter key to continue.
You have 4 items in your possession.

Press the enter key to continue.
You will live to fight another day.

Enter the index number for an item in inventory: 1
At index 1 is armor

Enter the index number to begin a slice: 2
Enter the index number to end the slice: 4
inventory[ 2 : 4 ] is ['shield', 'healing potion']

Press the enter key to continue.
You find a chest which contains:
['gold', 'gems']
You add the contents of the chest to your inventory.
Your inventory is now:
['sword', 'armor', 'shield', 'healing potion', 'gold', 'gems']

Press the enter key to continue._
```

```
C:\Python31\python.exe

Press the enter key to continue.
You trade your sword for a crossbow.
Your inventory is now:
['crossbow', 'armor', 'shield', 'healing potion', 'gold', 'gems']

Press the enter key to continue.
You use your gold and gems to buy an orb of future telling.
Your inventory is now:
['crossbow', 'armor', 'shield', 'healing potion', 'orb of future telling']

Press the enter key to continue.
In a great battle, your shield is destroyed.
Your inventory is now:
['crossbow', 'armor', 'healing potion', 'orb of future telling']

Press the enter key to continue.
Your crossbow and armor are stolen by thieves.
Your inventory is now:
['healing potion', 'orb of future telling']

Press the enter key to exit._
```

Kreiranje liste

- Na početku programa kreira se nova lista, dodeljuje se promenljivoj inventory, i ispisuju svi njeni elementi; razlika u odnosu na verziju 2.0 je što se sada koriste uglaste umesto običnih zagrada, kako bi se formirala lista umesto n-torke; odgovarajući deo koda hero's_inventory3.py je:

```
# Hero's Inventory 3.0
# Demonstrates lists

# create a list with some items and display with a for loop
inventory = ["sword", "armor", "shield", "healing potion"]
print("Your items:")
for item in inventory:
    print(item)

input("\nPress the enter key to continue.")
```

Upotreba funkcije len()

- Sledeći deo koda je isti kao u verziji 2.0 -- funkcija len() radi sa listom isto što i sa n-torkom:

```
# get the length of a list
print("You have", len(inventory), "items in your possession.")

input("\nPress the enter key to continue.")
```

Upotreba operatora in

- Sledeći deo koda je takođe isti kao u verziji 2.0 -- operator in radi sa listom isto što i sa n-torkom:

```
# test for membership with in
if "healing potion" in inventory:
    print("You will live to fight another day.")
```

Indeksiranje liste

- Sledeći deo koda je takođe isti kao u verziji 2.0 -- indeksiranje liste se radi isto kao kod n-torke

```
# display one item through an index
index = int(input("\nEnter the index number for an item in inventory: "))
print("At index", index, "is", inventory[index])
```

Isecanje liste

- Sledeći deo koda je takođe isti kao u verziji 2.0 -- isecanje liste se radi isto kao kod n-torke

```
# display a slice
start = int(input("\nEnter the index number to begin a slice: "))
finish = int(input("Enter the index number to end the slice: "))
print("inventory[" , start, ":", finish, "] is", end=" ")
print(inventory[start:finish])
```

Nadovezivanje lista

- Sledeći deo koda je takođe isti kao u verziji 2.0 -- nadovezivanje lista se radi isto kao kod n-torki (naravno, mogu se nadovezivati samo lista na listu i n-torka na n-torku)

```
# concatenate two lists
chest = ["gold", "gems"]
print("You find a chest which contains:")
print(chest)
print("You add the contents of the chest to your inventory.")
inventory += chest
print("Your inventory is now:")
print(inventory)

input("\nPress the enter key to continue.")
```

Promenljivost liste

- Kao što je pomenuto, liste su promenljive, i pružaju veću fleksibilnost od n-torki

Dodela nove vrednosti elementu liste pomoću indeksiranja

- Da bi se neki element liste promenio, samo mu se dodeli nova vrednost uz upotrebu indeksa:

```
# assign by index
print("You trade your sword for a crossbow.")
inventory[0] = "crossbow"
print("Your inventory is now:")
print(inventory)

input("\nPress the enter key to continue.")
```

- Pre izvršenja ovog dela koda, element liste sa indeksom 0 je bio "sword"; kada se izvrši treća linija u prethodnom delu koda, ovaj element dobija novu vrednost, "crossbow" -- promena se vidi prilikom ispisivanja na ekranu
- Na pokazani način, postojećem elementu liste se zadaje nova vrednost; ono šta nije moguće je da se kreira njegov novi element; ako se pokuša da se neka vrednost dodeli nepostojećem elementu, dobiće se poruka o grešci

Dodela nove vrednosti isečku liste

- Pored toga što se nova vrednost može dodeliti jednom elementu, ona se može dodeliti isečku liste:

```
# assign by slice
print("You use your gold and gems to buy an orb of future telling.")
inventory[4:6] = ["orb of future telling"]
print("Your inventory is now:")
print(inventory)

input("\nPress the enter key to continue.")
```

- Pomoću ovog dela koda, isečak od dva člana liste, sa indeksima 4 i 5, je zamenjen stringom "orb of future telling"; kako je lista sa jednim elementom dodeljena isečku od dva elementa, dužina liste se smanjuje sa jedan

Brisanje elementa liste

- Element liste se briše pomoću naredbe del (od "delete"):

```
# delete an element
print("In a great battle, your shield is destroyed.")
del inventory[2]
print("Your inventory is now:")
print(inventory)

input("\nPress the enter key to continue.")
```

- Kada se izvrši ovaj deo koda, element liste sa indeksom 2 se briše; time se dužina liste smanjuje za jedan, i indeksi svih elemenata koji nastupaju posle izbrisanog se smanjuju za jedan (element sa indeksom 3 postaje novi element sa indeksom 2, i tako redom)

Brisanje isečka liste

- Takođe se može obrisati i isečak liste:

```
# delete a slice
print("Your crossbow and armor are stolen by thieves.")
del inventory[:2]
print("Your inventory is now:")
print(inventory)

input("\n\nPress the enter key to exit.")
```

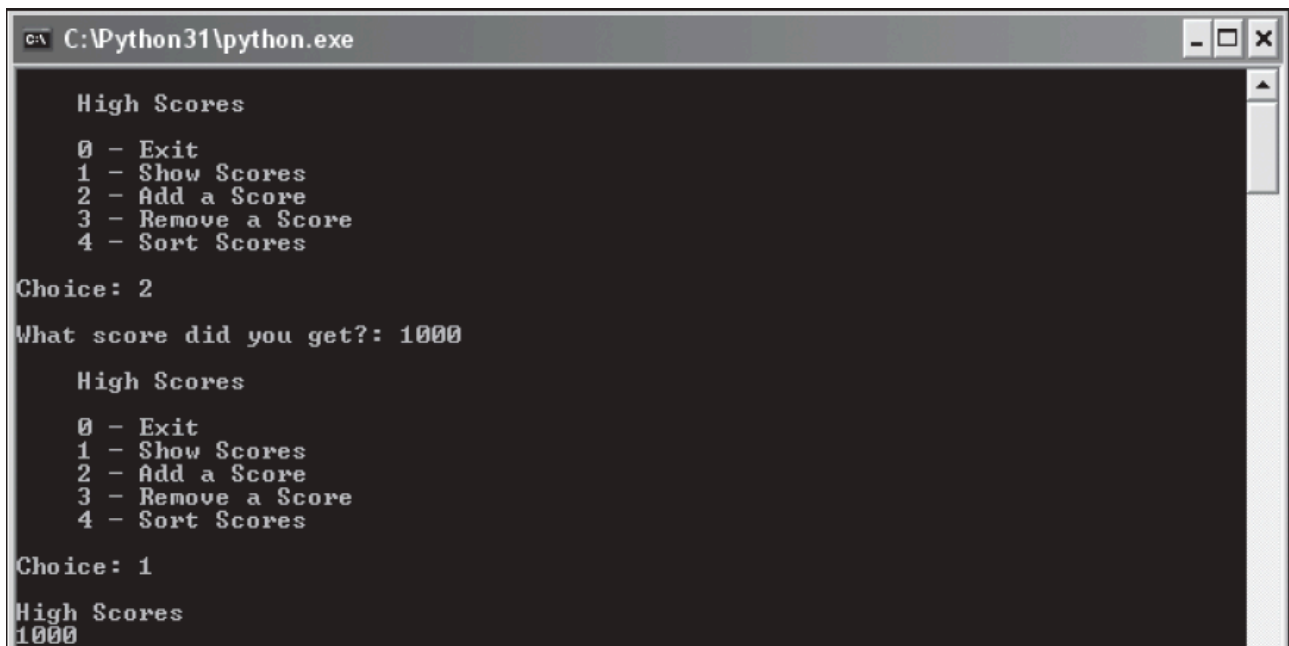
- Pomoću ovog dela koda, briše se isečak koga čine prva dva elementa liste; dužina liste se smanjuje za dva, a preostali elementi dobijaju nove indekse redom počev od 0

5.3 Metode liste

- *Metoda* (ženski rod, "method") je koncept sličan funkciji; postoje metode koje služe za rad sa listama, i nazivaju se skraćeno *metode liste* ("list method") -- one na primer služe za dodavanje elementa liste, brisanje elementa na osnovu njegove vrednosti, sortiranje liste, obrtanje redosleda elemenata...

Program High Scores

- Program High Scores koristi metode liste kako bi kreirao i ažurirao listu najboljih rezultata koje su igrači postigli u nekoj igri
- Program koristi jednostavan interfejs: korisnik ima nekoliko opcija -- može prikazati rezultate, dodati novi rezultat, obrisati rezultat, sortirati rezultate ili izaći iz programa (Slika 5.6)



```

C:\Python31\python.exe

High Scores
0 - Exit
1 - Show Scores
2 - Add a Score
3 - Remove a Score
4 - Sort Scores
Choice: 2
What score did you get?: 1000

High Scores
0 - Exit
1 - Show Scores
2 - Add a Score
3 - Remove a Score
4 - Sort Scores
Choice: 1
High Scores
1000

```

- Odgovarajući kod je high_scores.py, i biće prikazan u delovima

Pripremne radnje

- Deo koda koji obavlja pripremne radnje je:

```

# High Scores
# Demonstrates list methods

scores = []

choice = None

```

- Pripremne radnje podrazumevaju kreiranje dve promenljive:
 - scores je lista koja će sadržavati rezultate; inicijalno je postavljena na praznu listu
 - choice je opcija koju je igrač odabrao; inicijalno je postavljena na None

Prikaz menija

- Glavni deo programa je while-petlja: ona se "vrti" sve dok korisnik ne pritisne 0 kao znak da hoće da prekine izvršenje programa (da "izađe")
- Ostatak glavnog dela programa prvo ispisuje meni, i traži da korisnik unese svoj izbor (opciju); odgovarajući deo koda je:

```
while choice != "0":  
    print(  
        ""  
        High Scores  
  
        0 - Exit  
        1 - Show Scores  
        2 - Add a Score  
        3 - Remove a Score  
        4 - Sort Scores  
        ""  
    )  
  
    choice = input("Choice: ")  
    print()
```

Izlaz iz programa

- Ako korisnik hoće da izađe, dobija poruku "Good-bye."

```
# exit  
if choice == "0":  
    print("Good-bye.")
```

- Ako/kada korisnik unese vrednost 0, kada se posle ispisivanja poruke program bude vratio na naredbu while, uslov će postati False i petlja i program će se prekinuti

Prikaz rezultata

- Ako/kada korisnik unese vrednost 1, program ispisuje listu:

```
# list high-score table
elif choice == "1":
    print("High Scores")
    for score in scores:
        print(score)
```

Dodavanje rezultata

- Ako/kada korisnik unese vrednost 2, program mu traži novu vrednost rezultata koju treba uneti, i dodeljuje je promenljivoj score; zatim program koristi metodu liste append() da doda promenljivu score na kraj liste scores, čime lista postaje duža za dodati element

- Odgovarajući deo koda je:

```
# add a score
elif choice == "2":
    score = int(input("What score did you get?: "))
    scores.append(score)
```

- Znači, metoda liste append() se poziva tako što se napiše ime liste, pa tačka, pa append, pa ime promenljive (novog elementa) koja se dodaje u listu, u zagradama:

ime_liste.append(ime_promenljive)

Brisanje rezultata

- Ako/kada korisnik unese vrednost 3, program mu traži vrednost rezultata koju treba izbrisati:

- ako je rezultat u listi, briše se na prvom mestu gde se pojavljuje u njoj
- ako rezultat nije u listi, korisnik se o tome obaveštava

- Odgovarajući deo koda je:

```
# remove a score
elif choice == "3":
    score = int(input("Remove which score?: "))
    if score in scores:
        scores.remove(score)
    else:
        print(score, "isn't in the high scores list.")
```

- Prvo se proverava da li se vrednost score nalazi u listi scores: ako je to slučaj, poziva se metoda liste remove() -- ona ide kroz listu, počev od pozicije 0, i traži zadatu vrednost:
 - kada naiđe na prvu poziciju gde se ta vrednost nalazi, taj element se briše iz liste; ona time postaje kraća za jedan element, uz ažuriranje indeksa
 - na svim narednim pozicijama gde se ta vrednost nalazi, ne dolazi do brisanja
- Znači, metoda liste remove() se poziva tako što se: napiše ime liste, pa tačka, pa remove, pa ime promenljive čija se vrednost briše iz liste, u zagradama:
ime_liste.remove(ime_promenljive)
- Remove() briše element na osnovu njegove vrednosti, a del na osnovu pozicije (indeksa)
- Ako se pokuša brisanje vrednosti koja ne postoji u listi, dobija se poruka o grešci; zato se prvo koristi naredba if kao što je pokazano

Sortiranje rezultata

- Rezultati u listi (to jest njeni elementi) su poređani onim redom kako ih je korisnik unosio: svaki novi element dodaje se na kraj liste
- Da bi sortirao rezultate, korisnik treba da unese vrednost 4, posle čega se izvršava sledeći deo koda:

```
# sort scores
elif choice == "4":
    scores.sort(reverse=True)
```

- Metoda liste `sort()` sortira elemente liste; po defaultu, sortira se u rastućem poretku; da bi se sortiralo u opadajućem poretku (najveći na prvo mesto), parametru metode koji se naziva `reverse` treba zadati vrednost `True` (za default se ništa ne piše)

Tretiranje izbora nepostojeće opcije

- Ako korisnik kao svoj izbor unese vrednost koja nije validna (nije 0, 1, 2, 3 ili 4), programu to javlja:

```
# some unknown choice
else:
    print("Sorry, but", choice, "isn't a valid choice.")
```

Čekanje na odziv korisnika

- Pošto korisnik unese vrednost 0, po običaju čeka se na njega da završi izvršenje:

```
input("\n\nPress the enter key to exit.")
```

- Tabelarni pregled metoda lista dat je u Tabeli 5.1

Method	Description
<code>append(value)</code>	Adds <i>value</i> to end of a list.
<code>sort()</code>	Sorts the elements, smallest value first. Optionally, you can pass a Boolean value to the parameter <code>reverse</code> . If you pass <code>True</code> , the list will be sorted with the largest value first.
<code>reverse()</code>	Reverses the order of a list.
<code>count(value)</code>	Returns the number of occurrences of <i>value</i> .
<code>index(value)</code>	Returns the first position number of where <i>value</i> occurs.
<code>insert(i, value)</code>	Inserts <i>value</i> at position <i>i</i> .
<code>pop([i])</code>	Returns value at position <i>i</i> and removes value from the list. Providing the position number <i>i</i> is optional. Without it, the last element in the list is removed and returned.
<code>remove(value)</code>	Removes the first occurrence of <i>value</i> from the list.

5.4 N-torke i liste: komparacija

- Iako se pomoću lista može uraditi mnogo više nego pomoću n-torki, ove druge ne treba zanemariti iz sledećih razloga:

- 1) n-torke su brže od lista; naime, računar zna da se one neće menjati i zato se u memoriji čuvaju i tretiraju na efikasniji način; ipak, ova razlika u brzini dolazi do izražaja samo kada se radi sa ogromnim količinama podataka
- 2) n-torke su idealne za kreiranje konstanti, jer su nepromenljive
- 3) u nekim slučajevima, kao što je rad sa rečnicima (što će biti pokazano u nastavku poglavlja), n-torke se moraju koristiti

5.5 Ugnježdene sekvence

- Liste i n-torke, kao što je navedeno, mogu biti sekvence bilo kojih podataka; automatski važi i da:

- liste mogu sadržati druge liste ili n-torke
- n-torke mogu sadržati druge n-torke ili liste

- Ovakve sekvence nazivaju se ugnježdene sekvence ("nested sequences"): kod njih se sekvence pojavljuju kao elementi drugih sekvenci; ovo se koristi za organizaciju kompleksnih podataka

- Primer: kupovina poklona za novu godinu -- majci ogrlica i narukvica, ocu džemper, bratu sat, majicu i čarape -- pravi se lista imena, a uz svako od imena iz liste ide lista poklona

Program High Scores 2.0

- Prethodna verzija programa High Score je sadržala samo rezultate; nova verzija 2.0: uvodi imena igrača, automatski sortira listu prema rezultatu, i limitira je na samo pet najboljih (Slika 5.7)

```
C:\Python31\python.exe
High Scores 2.0
0 - Quit
1 - List Scores
2 - Add a Score
Choice: 2
What is the player's name?: Curly
What score did the player get?: 3000
High Scores 2.0
0 - Quit
1 - List Scores
2 - Add a Score
Choice: 1
High Scores
NAME    SCORE
Curly  3000
Larry   2000
Moe     1000
```

Kreiranje ugnježdjenih sekvenci

- Ugnježdene sekvence (liste odnosno n-torke) kreiraju se kao i obično, pomoću elemenata odvojenih zarezima (u uglastim odnosno običnim zagradama); ti elementi mogu biti druge liste ili n-torke

- Primer:

```
>>> nested = ["first", ("second", "third"), ["fourth", "fifth", "sixth"]]
```

znači, nested je lista u kojoj važi:

- prvi element je string
- drugi element je n-torka (čiji su elementi stringovi)
- treći element je lista (čiji su elementi stringovi)

- Korisno je pri kreiranju ugnježdene sekvence koristiti neki konzistentan šablon kako bi se pratila logika po kojoj su podaci organizovani

- Primer -- linija:

```
>>> scores = [("Moe", 1000), ("Larry", 1500), ("Curly", 3000)]
```

kreira listu od tri elementa, od kojih je svaki n-torka: svaka ima tačno dva elementa, od kojih je prvi string a drugi broj -- ime igrača i broj poena koje je osvojio

- Ugnježdavanje sekvenci se teorijski može organizovati na proizvoljnom broju nivoa, ali se u praksi ide samo do jednog (kao u poslednjem primeru sa scores); drugim rečima, sekvenca kao elemente ima sekvence, ali one ne mogu kao elemente imati nove sekvence; u suprotnom, lako dolazi do konfuzije, kao na primer:

```
>>> nested = ("deep", ("deeper", ("deepest", "still deepest")))
```

Pristup elementima u ugnježdenim sekvencama

- Pristup elementima se obavlja uobičajeno, pomoću indeksiranja, kao na primer:

```
>>> scores = [("Moe", 1000), ("Larry", 1500), ("Curly", 3000)]
>>> print(scores[0])
('Moe', 1000)
```

svaki element liste scores je n-torka, kojoj se pristupa pomoću indeksa u okviru scores

- Dalje, direktan pristup jednom elementu u jednoj od n-torki ostvaruje se pomoću duplog indeksa:

```
>>> print(scores[2][0])
Curly
```

- Znači, na ovaj način se računaru saopštava da treba da pristupi elementu sa indeksom 2 u listi -- što je n-torka ("Curly", 3000) -- i da zatim treba da pristupi elementu sa indeksom 0 u n-torci -- što je "Curly"

Raspakivanje sekvence

- Ukoliko je poznato koliko elemenata sadrži neka sekvenca, svaki od njih se može dodeliti posebnoj promenljivoj na sledeći način:

```
>>> name, score = ("Shemp", 175)
>>> print(name)
Shemp
>>> print(score)
175
```

- Ovo se naziva raspakivanje ("unpacking") sekvence, i važi za svaki tip sekvence -- bitno je samo da broj promenljivih i broj elemenata sekvence budu jednaki

Pripremne radnje

- Kao i u prethodnoj verziji programa High Scores, postave se promenljive i while petlja; ako/kada korisnik unese 0, štampa se poruka "Good-bye."; odgovarajući deo koda je:

```
# High Scores 2.0
# Demonstrates nested sequences

scores = []

choice = None
while choice != "0":

    print(
        """
    High Scores 2.0

    0 - Quit
    1 - List Scores
    2 - Add a Score
    """
    )

    choice = input("Choice: ")
    print()

    # exit
    if choice == "0":
        print("Good-bye.")
```

Ispisivanje rezultata pomoću ugnježenih n-torki

- Ako/kada korisnik unese opciju 1, program prolazi kroz svaki element u scores, raspakuje rezultat i ime u promenljive score i name, i zatim ih ispisiuje:

```
# display high-score table
elif choice == "1":
    print("High Scores\n")
    print("NAME\tSCORE")
    for entry in scores:
        score, name = entry
        print(name, "\t", score)
```

Dodavanje rezultata pomoću ugnježenih n-torki

- Ako/kada korisnik unese opciju 2:
 - računar od korisnika traži da unese novi rezultat i ime
 - sa unete dve vrednosti, kreira se n-torka entry
 - ta n-torka se dodaje na kraj liste
 - sortira se lista u opadajućem poretку elemenata (tako da se najviši rezultat nađe na prvom mestu)
 - napravi se isečak i dodela tako da se čuva samo prvih pet najboljih rezultata
- Izabrano je da se, u okviru n-torke, unese prvo rezultat pa onda ime da bi se sortiranje vršilo po rezultatu; naime, sortiranje liste se uvek vrši po vrednostima njenog prvog elementa
- Odgovarajući deo koda je:

```
# add a score
elif choice == "2":
    name = input("What is the player's name?: ")
    score = int(input("What score did the player get?: "))
    entry = (score, name)
    scores.append(entry)
    scores.sort(reverse=True)
    scores = scores[:5]      # keep only top 5 scores
```

Tretiranje izbora nepostojeće opcije

- Izbor nepostojeće opcije se tretira kao ranije:

```
# some unknown choice
else:
    print("Sorry, but", choice, "isn't a valid choice.")
```

Čekanje na odziv korisnika

- Pošto korisnik unese vrednost 0 i završi petlja, čeka se na njega da završi izvršenje programa:

```
input("\n\nPress the enter key to exit.")
```

5.6 Deljene reference

- Prilikom dodele neke vrednosti nekoj promenljivoj, kaže se da se vrši *referenciranje* pomoću te promenljive; tako na primer, ako se napiše linija dodele:

```
language = "Python"
```

- računar će string "Python" postaviti na neku memorijsku lokaciju

- zatim će kreirati promenljivu language koja će ukazivati (referencirati) na tu lokaciju

Drugim rečima, vrednost neke promenljive je fizički sadržaj lokacije na koju ona ukazuje (referencira)

- Pri radu sa nepromenljivim ("immutable") tipovima podataka, kao što su brojevi, stringovi ili n-torke (to jest, sa odgovarajućim promenljivama), ako se napravi kopija neke promenljive (originala), kopija i original će biti dve promenljive nezavisne jedna od druge

- Preciznije, ako se obavi kopiranje pomoću linije:

```
copy = original
```

desiće se sledeće:

- uvodi se nova promenljiva (copy) koja referencira na NOVU memorijsku lokaciju (različitu od one na koju referencira original)

- vrednost promenljive copy je, posle izvršenja gornje linije, jednaka vrednosti promenljive original: fizički, sadržaj lokacije na koju ukazuje original se iskopira u lokaciju na koju ukazuje copy

- sa promenljivom copy se nadalje radi nezavisno od promenljive original: tako na primer u narednoj liniji vrednost copy može biti promenjena, i to se neće odraziti na vrednost originala

- Primer -- posle izvršenja koda:

```
a = 3  
b = a  
b = 4
```

a će imati vrednost 3, a b vrednost 4

- Međutim, pri radu sa promenljivim ("mutable") tipovima podataka, kao što su liste (to jest, sa odgovarajućim promenljivama), ako se napravi kopija neke promenljive, "kopija" i "original" će biti dve promenljive koje ukazuju (referišu) na istu lokaciju u memoriji -- kaže se da dele referencu ("shared reference")

- Kao posledica deljene reference, ako se vrednost jedne od te dve promenljive (originala ili kopije) promeni pomoću naredbe dodele, to će automatski promeniti i vrednost one druge promenljive (iako za nju nije navedena nijedna dodela)

- Preciznije, ako se obavi kopiranje pomoću linije:

```
copy = original
```

desiće se sledeće:

- uvodi se nova promenljiva (copy) koja referencira na ISTU memorijsku lokaciju

- vrednost promenljive copy je, posle izvršenja gornje linije, jednaka vrednosti promenljive original, jer se radi o istoj lokaciji

- pošto te dve promenljive ukazuju na istu lokaciju, sadržaj te lokacije se može menjati pomoću bilo koje od njih (a ona druga će tu promenu naravno "registrovati")

- Primer -- kod:

```
>>> mike = ["khakis", "dress shirt", "jacket"]
>>> mr_dawson = mike
>>> honey = mike
>>> print(mike)
['khakis', 'dress shirt', 'jacket']
>>> print(mr_dawson)
['khakis', 'dress shirt', 'jacket']
>>> print(honey)
['khakis', 'dress shirt', 'jacket']
```

pravi dve kopije promenljive mike; sve tri promenljive ukazuju (referišu) na istu memorijsku lokaciju, u kojoj je smeštena navedena lista (Slika 5-9)



- Dalje, ako se pomoću jedne od tri promenljive izvrši neka promena sadržaja lokacije, to će se odraziti i na vrednosti druge dve promenljive; na primer:

```
>>> honey[2] = "red sweater"
>>> print(honey)
['khakis', 'dress shirt', 'red sweater']
>>> print(mike)
['khakis', 'dress shirt', 'red sweater']
>>> print(mr_dawson)
['khakis', 'dress shirt', 'red sweater']
```

- Na programeru je da odluči da li hoće da koristi deljene reference ili ne; da se one izbegnu, koristi se isecanje; na primer:

```
>>> mike = ["khakis", "dress shirt", "jacket"]
>>> honey = mike[:]
>>> honey[2] = "red sweater"
>>> print(honey)
['khakis', 'dress shirt', 'red sweater']
>>> print(mike)
['khakis', 'dress shirt', 'jacket']
```

- U poslednjem primeru, u drugoj liniji je pomoću isecanja (koje je trivijalno: iseca se cela lista) naglašeno da kopija (honey) treba da ukaže na neku drugu memorijsku lokaciju, i da se u nju iskopira sadržaj lokacije na koju ukazuje original (mike); zbog toga su u nastavku te dve promenljive nezavisne jedna od druge

5.7 Rečnici

- Kao što je pokazano, liste i n-torke omogućavaju organizaciju podataka u okviru sekvenci, gde se podaci ređaju redom jedan za drugim; *rečnik* ("dictionary") je struktura podataka koja omogućava način organizacije u obliku njihovih parova
- Organizacija podataka u rečniku je slična kao u klasičnom rečniku stranih reči: u njemu je data reč (na jednom jeziku), i njoj ekvivalentne reči (na drugom jeziku); slična je i rečniku sinonima: u njima je data jedna reč, i njeni ekvivalenti iz istog jezika

бусија (тур.) -- заклон, заседа; место подесно
за скривање и изненадни напад

geek (gi:k )

▶ Definitions

noun (slang)

1. a person who is knowledgeable and enthusiastic about a specific subject

2. a boring and unattractive social misfit

- Kada koristi klasični rečnik, korisnik traži/zada željenu (nepoznatu) reč (one su složene po azbučnom redu), i nađe/dobije njen ekvivalent (parnjak); u Pythonu se radi na sličan način: nađe se željena reč koja se naziva *ključ* ("key") i dobije se njen ekvivalent koji se naziva *vrednost* ("value")

Program Geek Translator

- Kao rezultat najnovijih tehnologija, nove reči se svakodnevno pojavljuju (i uz njih i nova potkultura); program Geek Translator pravi rečnik "gikovskih" reči; korisnik ne samo što može da potraži određenu reč, već može i da doda novu, napravi korekciju postojeće ili da je obriše -- Slika 5.10



Kreiranje rečnika

- Prvo što je urađeno je kreiranje rečnika: gikovske reči i njihova objašnjenja; odgovarajući deo koda iz `geek_translator.py` je:

```
# Geek Translator
# Demonstrates using dictionaries

geek = {"404": "clueless. From the web error message 404, meaning page not found.",
        "Googling": "searching the Internet for background information on a person.",
        "Keyboard Plaque" : "the collection of debris found in computer keyboards.",
        "Link Rot" : "the process by which web page links become obsolete.",
        "Uninstalled" : "being fired. Especially popular during the dot-bomb era."}
```

- Kreirani rečnik se zove (dodeljen je promenljivoj) `geek`; sastoji se od parova podataka (u ovom slučaju, stringova); svaki par, koji se naziva *odrednica* ("item"), čine jedan ključ i odgovarajuća vrednost

- Pri kreiranju rečnika: piše se njegovo ime, pa znak jednakosti, pa u vitičastim zagradama odrednice, međusobno odvojene zarezima:

```
ime_rečnika = {odrednica_1, odrednica_2, ...}
```

- Za definisanje odrednice, piše se prvo ključ, pa dvotačka, pa vrednost:

odrednica = ključ : vrednost

U ovom primeru, svi ključevi i vrednosti su stringovi

Pristup vrednostima u rečniku

- Sa rečnikom se tipično zadaje ključ da bi se dobila odgovarajuća vrednost; to se može obaviti na više načina, kao što će biti pokazano

Unošenje ključa da se dobije vrednost

- Najprostiji način da se dobije vrednost iz rečnika je da joj se direktno pristupi pomoću ključa -- to se radi tako što se napiše ime rečnika, za kojim sledi ime ključa u uglastim zagradama:

ime_rečnika[ime_ključa]

- Primer:

```
>>> geek["404"]  
clueless. From the web error message 404, meaning page not found.
```

- Ovo je slično indeksiranju, ali je suštinska razlika u tome što se kod indeksiranja zadaje indeks (pozicija) u sekvenci, a u rečniku se zadaje ključ, pa računar pronađe odgovarajuću vrednost; rečnici nemaju nikakve indekse, to jest redne brojeve odrednica; sam redosled pojavljivanja odrednica u rečniku je nebitan

- Vrednost ne može biti zadata da bi se dobio ključ (kao i u klasičnim rečnicima); ako se zada ključ koji ne postoji, javiće se poruka o grešci (KeyError)

Provera da li ključ postoji pomoću operatora in

- Pošto zadavanje nepostojećeg ključa javlja grešku, preporučljivo je prvo proveriti da li ključ uopšte postoji u rečniku; to se radi pomoću operatora in; primer:

```
if "Dancing Baloney" in geek:
    print("I know what Dancing Baloney is. ")
else:
    print("I have no idea what Dancing Baloney is. ")
```

- Operator in se koristi isto kao kod lista i n-torki: piše se if, posle koga sledi ključ, posle koga ide in i ime rečnika:

```
if ime_ključa in ime_rečnika
```

Očigledno, in samo proverava postojanje ključa (vrati True/False), ali ne vraća odgovarajuću vrednost

Upotreba metode get() da se dobije vrednost

- Drugi način da se na osnovu ključa dobije vrednost je pomoću metode rečnika ("dictionary method", kompletnije: metode za rad sa rečnicima) get() -- piše se ime rečnika, posle koga sledi tačka, pa get, pa ključ u zagradi:

```
ime_rečnika.get(ime_ključa)
```

- Ako se unese nepostojeći ključ, ova metoda ne javlja grešku, već vraća vrednost None -- primer:

```
>>> print(geek.get("Dancing Baloney"))
None
```

- Može se zadati da program, u slučaju da je ključ nepostojeći, umesto vrednosti None ispiše na ekranu neku difolt vrednost; da bi se to uradilo, tu difolt vrednost treba navesti kao parametar posle vrednosti ključa -- primer:

```
>>> print(geek.get("Dancing Baloney", "I have no idea. "))
I have no idea.
```

Pripremne radnje

- Pošto je kreiran rečnik geek, napravljen je (kao i ranije) meni sa pet opcija; ako/kada korisnik izabere opciju 0, program se prekida:

```
choice = None
while choice != "0":

    print(
        """
    Geek Translator

    0 - Quit
    1 - Look Up a Geek Term
    2 - Add a Geek Term
    3 - Redefine a Geek Term
    4 - Delete a Geek Term
    """
    )

    choice = input("Choice: ")
    print()

    # exit
    if choice == "0":
        print("Good-bye.")
```

Zadavanje ključa da se dobije vrednost

- Ako/kada korisnik izabere opciju 1, program mu traži ključ (promenljiva term), i zatim ispisuje odgovarajuću vrednost (promenljiva definition) kao što je već pokazano:

```
# get a definition
elif choice == "1":
    term = input("What term do you want me to translate?: ")
    if term in geek:
        definition = geek[term]
        print("\n", term, "means", definition)
    else:
        print("\nSorry, I don't know", term)
```

Dodavanje odrednice

- Rečnici su promenljivi ("mutable"), i mogu se modifikovati; ako/kada korisnik odabere opciju 2, program mu traži da unese novu odrednicu, to jest par ključ-vrednost

```
# add a term-definition pair
elif choice == "2":
    term = input("What term do you want me to add?: ")
    if term not in geek:
        definition = input("\nWhat's the definition?: ")
        geek[term] = definition
        print("\n", term, "has been added.")
    else:
        print("\nThat term already exists! Try redefining it.")
```

- Program traži novi ključ (promenljiva term): ako on već ne postoji u rečniku, program traži vrednost (promenljiva definition) i dodaje novi par (to jest odrednicu) u rečnik, u liniji:

```
geek[term] = definition
```

- Znači, nova odrednica u rečniku se kreira tako što se piše ime rečnika, posle koga ide ključ u uglastim zagradama, posle koga ide znak dodele, i na kraju vrednost (definicija):

```
ime_rečnika[ključ] = vrednost
```

- Ako ključ već postoji, program odbija da doda novu odrednicu; ovo se radi da korisnik ne bi greškom promenio postojeću odrednicu (takav događaj se se mora predvideti); ako on zaista hoće da modifikuje (redefiniše) neku postojeću odrednicu, treba da odabere opciju 3 iz menija

Modifikacija vrednosti

- Ako/kada korisnik izabere opciju 3, to znači da želi da modifikuje (izmeni/redefiniše) vrednost za neki već postojeći ključ; program traži da on unese ključ i novu vrednost:

```
# redefine an existing term
elif choice == "3":
    term = input("What term do you want me to redefine?: ")
    if term in geek:
        definition = input("What's the new definition?: ")
        geek[term] = definition
        print("\n", term, "has been redefined.")
    else:
        print("\nThat term doesn't exist! Try adding it.")
```

- Da se promeni vrednost, to jest cela odrednica, koristi se ista linija koda kao ranije:

```
geek[term] = definition
```

program samo zameni aktuelnu odrednicu (to jest odgovarajuću vrednost, za dati ključ) novom

- Ako korisnik hoće da izmeni vrednost ali zada nepostojeći ključ, program ga to obaveštava i preporučuje mu opciju 2

Brisanje odrednice

- Ako/kada korisnik izabere opciju 4, izvršava se sledeći deo koda:

```
# delete a term-definition pair
elif choice == "4":
    term = input("What term do you want me to delete?: ")
    if term in geek:
        del geek[term]
        print("\nOkay, I deleted", term)
    else:
        print("\nI can't do that!", term, "doesn't exist in the dictionary.")
```

- Program traži od korisnika da unese ključ od odrednice koju bi da izbriše; zatim proverava (pomoću operatora in) da li taj ključ postoji, pa zatim:

- ako ključ postoji, odrednica se briše pomoću del na sledeći način: piše se naredba del, pa ime rečnika posle koga ide ključ u uglastim zagradama:

```
del ime_rečnika[ključ]
```

- ako ključ ne postoji, program o tome obaveštava korisnika

- Kao što je već pomenuto, bez operatora in, kada bi korisnik hteo da obriše odrednicu za koju ne postoji ključ, generisala bi se poruka o grešci

Završetak programa

- Poslednje linije u programu tretiraju slučaj kada korisnik unese nepostojeću opciju:

```
# some unknown choice
else:
    print("\nSorry, but", choice, "isn't a valid choice.")

input("\n\nPress the enter key to exit.")
```

Važne pojedinosti u vezi sa rečnicima

- Neke važne pojedinosti u vezi sa rečnicima su:
 - rečnik ne može sadržati više od jedne odrednice sa istim ključem
 - ključ je nepromenljiv, to jest korisnik ga ne može menjati; on može biti string, broj, n-torka i slično
 - vrednosti mogu biti promenljive ili nepromenljive
- Tabelarni pregled metoda rečnika dat je u Tabeli 5.2

Method	Description
<code>get(key, [default])</code>	Returns the value of <i>key</i> . If <i>key</i> doesn't exist, then the optional <i>default</i> is returned. If <i>key</i> doesn't exist and <i>default</i> isn't specified, then <code>None</code> is returned.
<code>keys()</code>	Returns a view of all the keys in a dictionary.
<code>values()</code>	Returns a view of all the values in a dictionary.
<code>items()</code>	Returns a view of all the items in a dictionary. Each item is a two-element tuple, where the first element is a key and the second element is the key's value.

5.8 Povratak na igru Hangman

- Igra Hangman je sinteza materije predstavljene u ovom poglavlju

Pripremne radnje

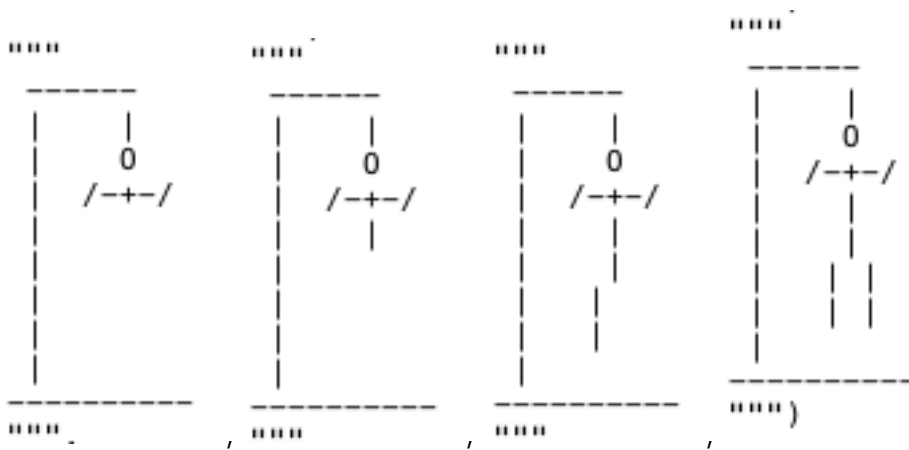
- Kod startuje komentarima; zatim se učitava/uvozi modul `random` -- odgovarajući deo koda `hangman.py` je:

```
# imports
import random
```

Kreiranje konstanti

- Odgovarajući deo koda (stringovi su ovde predstavljeni po horizontalama zbog uštede prostora, a razdvojeni su zarezima u okviru n-torke) je:

[illegible]



- Dalje, kreira se konstanta `MAX_WRONG` koja označava maksimalni broj pokušaja koje igrač ima:

```
MAX_WRONG = len(HANGMAN) - 1
```

taj broj je za 1 manji od broja slika; to je zato jer se prva slika prikazuje pre nego što igrač počne da igra; znači, iako u ovom slučaju ima 8 slika, igrač zapravo ima 7 pokušaja

- Na kraju, kreira se n-torka `WORDS`, kolekcija koja sadrži reči koje će se zadavati:

```
WORDS = ("OVERUSED", "CLAM", "GUAM", "TAFFETA", "PYTHON")
```

Inicijalizacija promenljivih

- Zatim se inicijalizuju promenljive; prvo se pomoću funkcije `random.choice()` slučajno odabere tražena reč iz kolekcije, i ona se dodeljuje promenljivoj `word`:

```
# initialize variables
word = random.choice(WORDS) # the word to be guessed
```

- Kreira se novi string `so_far` (= do sada) koji predstavlja ono šta je igrač do datog trenutka pogodio; ovaj string je inicijalno serija crtica, po jedna za svako slovo u reči; kada igrač pogodi slovo, crtice u reči se menjaju tim slovom:

```
so_far = "-" * len(word) # one dash for each letter in word to be guessed
```

- Kreira se i promenljiva `wrong` (inicijalna vrednost 0) koja broji neuspele pokušaje igrača; kreira se i (inicijalno prazna) lista `used` koja sadrži sva slova koja je igrač unosio tokom svojih pokušaja:

```
wrong = 0                # number of wrong guesses player has made
used = []                # letters already guessed
```

Glavna petlja

- Glavna petlja se izvršava sve dok igrač: ili napravi previše neuspehlih pokušaja, ili pogodi sva slova u reči:

```
print("Welcome to Hangman.  Good luck!")

while wrong < MAX_WRONG and so_far != word:
    print(HANGMAN[wrong])
    print("\nYou've used the following letters:\n", used)
    print("\nSo far, the word is:\n", so_far)
```

- Znači, posle dobrodošlice, ulazi se u `while` petlju -- u okviru nje se ispisuju na ekranu trenutna slika vešala, slova koja je do tada igrač odigrao, i trenutno stanje reči koje se pogađa (crtice zamenjene pogodnim slovima)

Unos slova od strane igrača

- Igrač unese slovo koje se pretvara u odgovarajuće veliko (u traženoj reči sva slova su velika); posle toga proverava se da li je igrač ranije odigrao to slovo -- ako jeste, traži se da unese drugo slovo, sve dok ne unese neko koje već nije odigrao; kada se to desi, ono se pretvara u veliko slovo, i dodaje listi već korišćenih slova:

```
guess = input("\n\nEnter your guess: ")
guess = guess.upper()

while guess in used:
    print("You've already guessed the letter", guess)
    guess = input("Enter your guess: ")
    guess = guess.upper()

used.append(guess)
```

Provera unetog/odigranog slova

- Zatim se proverava da li je igrač pogodio slovo: ako jeste, on se obaveštava; so_far se ažurira tako da se u njega uključi to slovo (umesto crtice):

```
if guess in word:
    print("\nYes!", guess, "is in the word!")

    # create a new so_far to include guess
    new = ""
    for i in range(len(word)):
        if guess == word[i]:
            new += guess
        else:
            new += so_far[i]
    so_far = new
```

- Ako igrač nije pogodio slovo, on se obaveštava, i broj neuspešnih pokušaja se povećava za jedan:

```
else:
    print("\nSorry,", guess, "isn't in the word.")
    wrong += 1
```

Kraj igre

- Ovim je igra završena: ako je broj neuspešnih pokušaja dostigao dozvoljeni maksimum, igrač je izgubio (i prikazuje se poslednja slika, na kojoj je čovečuljak "obešen"); inače, igraču se čestita na pogotku; u oba slučaja, obaveštava se koja je tražena reč:

```
if wrong == MAX_WRONG:
    print(HANGMAN[wrong])
    print("\nYou've been hanged!")
else:
    print("\nYou guessed it!")

print("\nThe word was", word)

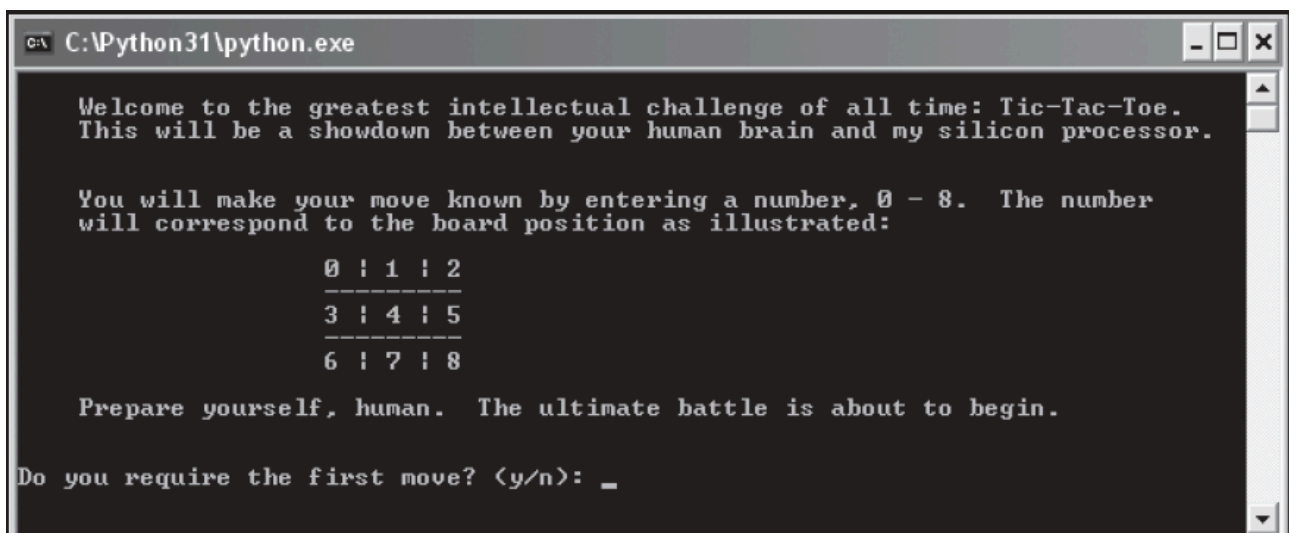
input("\n\nPress the enter key to exit.")
```

Poglavlje 6: Funkcije

- Svaki program u dosadašnjim poglavljima je bio jedna jedinstvena, kontinualna serija instrukcija; međutim, kada kod dostigne određenu dužinu ili nivo kompleksnosti, takav način kodiranja postaje neprikladan
- Iz tog razloga se složeniji programi razbijaju u manje celine, tako da postanu modularni (pa se takve celine nazivaju moduli) -- u ovom poglavlju, to će biti pokazano tako što će se kreirati *funkcije* ("function")
- U poglavlju će biti pokazano:
 - kreiranje (definisanje) funkcija
 - zadavanje vrednosti funkcijama
 - prihvatanje vrednosti od funkcija
 - globalne promenljive i konstante
 - simuliranje protivnika u igri strategije

6.1. Igra Tic-Tac-Toe

- U poznatoj igri Tic-Tac-Toe (TTT), biće pokazana simulacija igre protivnika pomoću veštačke inteligencije ("artificial intelligence" AI): igrač igra protiv računara -- Slike 6.1-6.3



```
C:\Python31\python.exe

Welcome to the greatest intellectual challenge of all time: Tic-Tac-Toe.
This will be a showdown between your human brain and my silicon processor.

You will make your move known by entering a number, 0 - 8. The number
will correspond to the board position as illustrated:

    0 | 1 | 2
    ---
    3 | 4 | 5
    ---
    6 | 7 | 8

Prepare yourself, human. The ultimate battle is about to begin.

Do you require the first move? (y/n): _
```

```
C:\Python31\python.exe

Where will you move? <0 - 8>:1
Fine...

  X | X | O
  ---
  | O |
  ---
  |   | X
  ---

I shall take square number 6

  X | X | O
  ---
  | O |
  ---
  O |   | X
  ---

O won!

As I predicted, human, I am triumphant once more.
Proof that computers are superior to humans in all regards.

Press the enter key to quit.
```

```
C:\Python31\python.exe

I shall take square number 3

  X |   | O
  ---
  O | O |
  ---
  X |   | X
  ---

Where will you move? <0 - 8>:7
Fine...

  X |   | O
  ---
  O | O |
  ---
  X | X | X
  ---

X won!

No, no! It cannot be! Somehow you tricked me, human.
But never again! I, the computer, so swear it!

Press the enter key to quit.
```

6.2 Kreiranje funkcija

- Neke ugrađene ("built-in") funkcije su već prikazane u ranijim poglavljima, kao što su `len()` i `range()`; na primer, funkcija `len()` računa dužinu stringa koji joj se zada; njihov kod nije vidljiv korisniku
- Python omogućava programeru da sam kreira funkcije po svojoj želji; one će funkcionisati na isti način kao i ugrađene:

- funkcija se pozove (u nekoj liniji koda)
 - odradi određeni zadatak (tako što na primer izračuna neku vrednost, ili ispiše nešto na ekranu)
 - zatim se nastavlja naredna linija koda
- Kreiranje funkcija pruža mnogo pogodnosti: ceo kod se razbija u manje celine, koje su "izolovane" jedna od druge (izvršavaju se nezavisno jedne od drugih), pa ih je zato lakše pisati, razumeti i menjati

Program Instructions

- Program Instructions ispisuje na ekranu uputstva za igru TTT, uz upotrebu funkcije -- Slika 6.4

```

C:\Python31\python.exe
Here are the instructions to the Tic-Tac-Toe game:

Welcome to the greatest intellectual challenge of all time: Tic-Tac-Toe.
This will be a showdown between your human brain and my silicon processor.

You will make your move known by entering a number, 0 - 8. The number
will correspond to the board position as illustrated:

    0 | 1 | 2
    -----
    3 | 4 | 5
    -----
    6 | 7 | 8

Prepare yourself, human. The ultimate battle is about to begin.

Here they are again:

Welcome to the greatest intellectual challenge of all time: Tic-Tac-Toe.
This will be a showdown between your human brain and my silicon processor.

You will make your move known by entering a number, 0 - 8. The number
will correspond to the board position as illustrated:

    0 | 1 | 2
    -----
    3 | 4 | 5
    -----
    6 | 7 | 8

Prepare yourself, human. The ultimate battle is about to begin.

You probably understand the game by now.

Press the enter key to exit.

```

- Odgovarajući kod instructions.py je:

```
# Instructions
# Demonstrates programmer-created functions

def instructions():
    """Display game instructions."""
    print(
        """
        Welcome to the greatest intellectual challenge of all time: Tic-Tac-Toe.
        This will be a showdown between your human brain and my silicon processor.

        You will make your move known by entering a number, 0 - 8.  The number
        will correspond to the board position as illustrated:

                0 | 1 | 2
                -----
                3 | 4 | 5
                -----
                6 | 7 | 8

        Prepare yourself, human.  The ultimate battle is about to begin. \n
        """
    )

# main
print("Here are the instructions to the Tic-Tac-Toe game:")
instructions()
print("Here they are again:")
instructions()
print("You probably understand the game by now.")

input("\n\nPress the enter key to exit.")
```

Definisanje funkcije

- Definisanje nove funkcije započeto je sledećom linijom:

```
def instructions():
```

Ova linija, koja se naziva *zaglavlje* ("header") funkcije, kaže računaru da blok koda koji sledi treba da tretira kao funkciju instructions() -- kad god se pozove ta funkcija, računar će odraditi taj blok

- Zaglavlje i blok koji sledi se zajedno nazivaju definicija funkcije: ona računaru saopštava šta funkcija radi, ali je računar (još uvek) ne pokreće; umesto toga, on "zapamti" da ta

funkcija postoji, tako da je može koristiti kada se bude ukazala potreba, to jest kada se iz nekog drugog dela programa ta funkcija pozove

- Znači, funkcija se definiše na sledeći način: piše se naredba `def`, pa ime funkcije po želji, pa par zagrada (u nastavku teksta će biti pokazano šta u njima treba da stoji), pa dvotačka, pa indentirani blok naredbi:

```
def ime_funkcije():  
    blok
```

Dokumentovanje funkcije

- Funkcija se dokumentuje (zapisuje se komentar šta ona radi) pomoću dokstringa ("docstring") -- u primeru, on je dat linijom:

```
"""Display game instructions."""
```

Dokstring se navodi pod trostrukim navodnicima, i piše se indentirano u prvoj liniji posle definicije funkcije

Poziv funkcije

- Funkcija se poziva tako što se navede njeno ime, posle koje ide par zagrada (biće pokazano šta u njima treba da stoji):

```
ime_funkcije()
```

U primeru, funkcija se poziva dva puta, pomoću linije:

```
instructions()
```

- Funkcija se poziva iz glavnog dela koda (označenog sa `#main`) -- to je deo koda koji se normalno izvršava kada se kod pokrene; čini ga sve ono što nije definicija funkcija; kada se naiđe na poziv funkcije, izvrši se odgovarajući blok, i zatim nastavi dalje sa glavnim delom

- Obično se u kodu prvo redom izdefinišu sve funkcije; ono što ostane na kraju, posle svih definicija, je glavni deo koda:

```
def funkcija1()
    blok1
...
def funkcijaN()
    blokN
#main
glavni_blok
```

Apstrakcija

- Prilikom pisanja i pozivanja funkcija, primenjuje se koncept koji se naziva *apstrakcija*; ona omogućava da programer ima širu (globalniju) sliku, bez vođenja računa o detaljima
- Na primer, ako programer dobije tuđi kod za neku funkciju, on uopšte ne mora da zna kako taj kod radi, i jedino što mu je važno je da ispravno obavlja zadatak; slično, pri korišćenju ugrađenih funkcija, onaj ko koristi funkciju ne zna (niti treba da zna) šta se u njoj nalazi; ona samo treba da odradi svoj zadatak
- Primer apstrakcije iz života: čovek naruči pljeskavicu na kiosku, to jest poziva funkciju `daj_pljeskavicu()` -- njega ne zanima gde je kupljeno meso, koje je meso, koliko se dugo peklo, itd; ti detalji su za njega nepotrebni; jedino što ga zanima je da funkcija odradi svoj zadatak, to jest da on dobije svoju pljeskavicu

6.3 Parametri i povratne vrednosti

- Kao što je pokazano ranije, ugrađenoj funkciji se mogu zadati neke vrednosti ("ulaz"), a one mogu da vrate neke vrednosti koje su odredile ("izlaz"); tako se funkciji `len()` zadavala neka sekvenca, a ona je vraćala njenu dužinu
- Funkcije koje napiše programer takođe mogu prihvatati i vraćati neke vrednosti; to omogućava da funkcija komunicira (razmenjuje podatke) sa ostalim delom programa

- Rekapitulacija pojmova:

- funkcija se poziva ("call") to jest izvršava ("run")

- funkciji se zadaju ("pass") vrednosti koje se nazivaju argumenti ("arguments")

[postoji i srpski termin "ulazna vrednost", ali ga treba izbegavati da se ne stvara konfuzija]

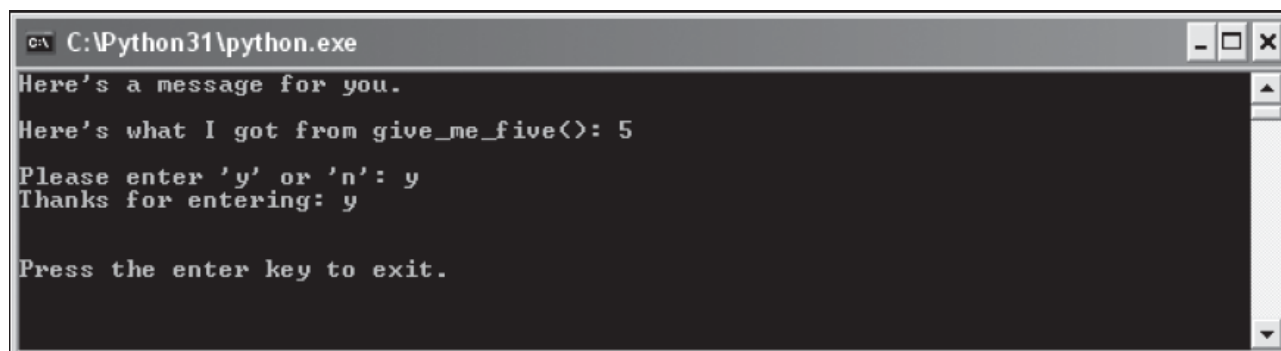
- funkcija prihvata ("receive") te argumente; interno u okviru same funkcije, oni se nazivaju parametri ("parameter")

- funkcija vraća ("return") vrednosti koje se nazivaju povratne vrednosti ("return values")

- prva linija funkcije naziva se zaglavlje ("header")

Program Receive and Return

- Program Receive and Return će sadržati tri funkcije: jedna prihvata vrednost, druga vraća vrednost, a treća i prihvata i vraća -- Slika 6.5



The screenshot shows a Windows command prompt window titled "C:\Python31\python.exe". The text displayed in the window is as follows:

```
Here's a message for you.  
Here's what I got from give_me_five<>: 5  
Please enter 'y' or 'n': y  
Thanks for entering: y  
  
Press the enter key to exit.
```

- Odgovarajući kod receive_and_return.py je:

```

# Receive and Return
# Demonstrates parameters and return values

def display(message):
    print(message)

def give_me_five():
    five = 5
    return five

def ask_yes_no(question):
    """Ask a yes or no question."""
    response = None
    while response not in ("y", "n"):
        response = input(question).lower()
    return response

# main
display("Here's a message for you.\n")

number = give_me_five()
print("Here's what I got from give_me_five():", number)

answer = ask_yes_no("\nPlease enter 'y' or 'n': ")
print("Thanks for entering:", answer)

input("\n\nPress the enter key to exit.")

```

Prihvatanje parametara

- Prva funkcija koja je definisana, display(), prihvata zadatu vrednost i ispisuje je na ekranu; kao što je pomenuto, ta vrednost se naziva argument (gledano iz ugla glavnog programa), odnosno parametar (gledano iz ugla funkcije); argumenti/parametri su obične promenljive, i pišu se u zagradama u zaglavlju funkcije:

```
def display(message):
```

- Kada se funkcija pozove i zadaju se odgovarajuće vrednosti-argumenti, ona prihvata te vrednosti i dodeljuje ih interno svojim parametrima; u ovom slučaju, kada se pozove display(), vrednost argumenta "Here's a message for you.\n" se dodeljuje parametru message

- U glavnom delu programa, funkcija se poziva pomoću linije:

```
display("Here's a message for you.\n")
```

tako da znači, u okviru funkcije, parametar message dobija vrednost navedenog stringa; zatim se funkcija izvrši

- Tokom izvršenja, parametar message se tretira kao obična promenljiva, tako da linija:

```
print(message)
```

ispisuje string na ekranu na uobičajeni način

- Ako bi se pozvala funkcija display() bez zadavanja argumenta, dobila bi se poruka o grešci; ova funkcija zahteva tačno jedan argument; kada se nekoj funkciji zadaje više argumenata, oni se navedu redom, odvojeni zarezima

Vraćanje povratnih vrednosti

- Druga funkcija koja je definisana, give_me_five(), vraća vrednost (koja se kako je navedeno naziva povratna vrednost), pomoću naredbe return:

```
return five
```

- Naredba return vraća vrednost promenljive five (to jest 5) onoj liniji iz koje je funkcija pozvana, posle čega se izvršenje funkcije završi; zatim, linija iz koje je funkcija pozvana prihvata tu vrednost i dalje radi sa njom

- U glavnom delu programa, funkcija se poziva sa:

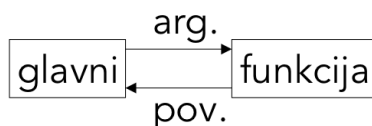
```
number = give_me_five()  
print("Here's what I got from give_me_five():", number)
```

Povratna vrednost funkcije se dodeljuje promenljivoj number; znači, number postaje jednako 5; zatim sledeća linija tu vrednost ispisuje na ekranu

- Funkcija može da vrati i više povratnih vrednosti: one se samo razdvoje zarezima; pri dodeli, mora se dati i odgovarajući broj promenljivih na levoj strani jednakosti

Kapsulacija

- U prethodnom primeru, važno je napomenuti da promenljiva `five` "ne postoji" izvan svoje funkcije; tako joj se na primer ne može pristupiti iz glavnog programa; to je slučaj sa svim promenljivama u okviru neke funkcije, uključujući tu i parametre -- ovo svojstvo se u Pythonu naziva *kapsulacija* ("encapsulation")
- Kao posledica kapsulacije, delovi koda su potpuno međusobno "izolovani" jedan od drugog; šta se dešava u okviru jednog dela, za sve druge delove je sakriveno i nepoznato
- Između delova koda, razmenjuju se samo podaci (varijable) koje su predviđene za "komunikaciju" to jest razmenu, i ništa više -- glavni deo zadaje argumente i dobija povratne vrednosti; funkcija prihvata argumente i vraća povratnu vrednost, i to je sve od komunikacije



- Kao posledica takve izolacije, promenljive koje se koriste u jednom delu koda ostaju lokalizovane samo u njemu, i o njima se u drugim delovima koda ne mora voditi računa; na primer, može postojati promenljiva sa istim imenom u više funkcija (iako se to po pravilu izbegava)
- Kapsulacija podseća na apstrakciju; one su tesno povezane; potrebno ju je samo načelno shvatiti, a detalji neće biti dalje izlagani

Prihvatanje i vraćanje vrednosti od strane jedne funkcije

- Treća definisana funkcija, `ask_yes_no()`, prihvata jednu vrednost -- pitanje i vraća drugu -- odgovor korisnika ("y" ili "n"); ona se definiše pomoću:

```
def ask_yes_no(question):
```

Parametar `question` dobija vrednost argumenta koji je zadat funkciji; u ovom slučaju to je string `"\nPlease enter 'y' or 'n': "`

- Sledeće linije u funkciji koriste dobijeni string (pitanje) `question` da traže odgovor od korisnika:

```
    response = None
    while response not in ("y", "n"):
        response = input(question).lower()
```

`while`-petlja se izvršava sve dok korisnik ne unese `y`, `Y`, `n` ili `N` (odgovor korisnika se uvek pretvara u sva mala slova)

- Prva linija, inicijalna dodela vrednosti `None` promenljivoj `response`, potrebna je jer je u drugoj liniji to kontrolna promenljiva u uslovu, a bez te dodele o njoj se inicijalno ne bi znalo ništa

- Konačno, kada korisnik unese validan odgovor, funkcija šalje taj odgovor glavnom programu pomoću:

```
    return response
```

i izvršenje funkcije se završava

- U glavnom delu programa, povratna vrednost se dodeljuje promenljivoj `answer` i ispisuje na ekranu:

```
answer = ask_yes_no("\nPlease enter 'y' or 'n': ")
print("Thanks for entering:", answer)
```

Višekratna upotreba funkcija

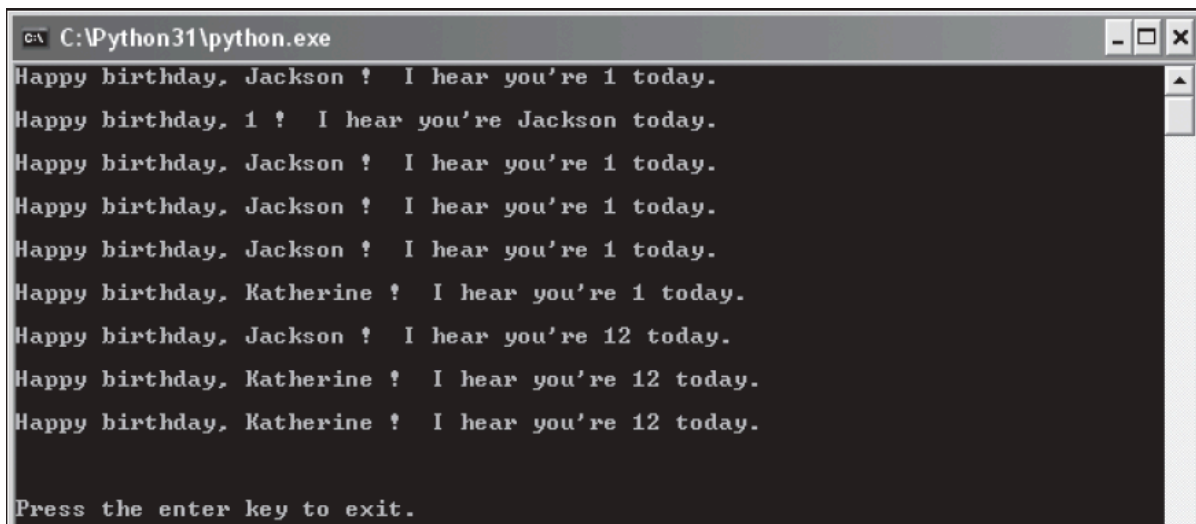
- Dodatna prednost funkcija je što one mogu biti jednostavno korišćene od strane drugih programa; zato je, za zadatke koji se često ponavljaju, praktično pisati funkcije i čuvati ih na posebnom mestu, kako bi se ponovo iskoristile kada se ukaže potreba; ovo se naziva *višekratna upotreba* ("reuse")
- Najdirektniji način da se neka funkcija ponovo upotrebi je da se iskopira u novi programski kod; ali postoji i bolji način: mogu se kreirati moduli, i zatim napraviti uvoz (pomoću naredbe `import` koja je ranije pokazana) funkcije u novi kod -- ovo se dalje neće objašnjavati

6.4 Keyword argumenti i default vrednosti parametara

- Kao što je pokazano, glavni program šalje podatak funkciji tako što joj zadaje vrednosti argumenata, koje se u okviru funkcije dodeljuju njenim parametrima; dosad je prikazan samo najjednostavniji način da se to uradi
- Python omogućava još veću kontrolu i fleksibilnost prilikom izmena podataka (komunikacije) između glavnog programa i funkcije, i to pomoću:
 - keyword (= ključne reči) argumenata, i
 - default (= podrazumevan) vrednosti parametara

Program Birthday Wishes

- Program Birthday Wishes, šalje rođendanske čestitke pomoću dve funkcije -- Slika 6.6:

A screenshot of a Windows command prompt window titled "C:\Python31\python.exe". The window has a black background with white text. It displays a series of birthday wishes. The first four lines are "Happy birthday, Jackson ! I hear you're 1 today.", followed by "Happy birthday, 1 ! I hear you're Jackson today.", then another "Happy birthday, Jackson ! I hear you're 1 today.", and another "Happy birthday, Jackson ! I hear you're 1 today.". The next four lines are "Happy birthday, Katherine ! I hear you're 1 today.", "Happy birthday, Jackson ! I hear you're 12 today.", "Happy birthday, Katherine ! I hear you're 12 today.", and "Happy birthday, Katherine ! I hear you're 12 today.". At the bottom, it says "Press the enter key to exit.".

```
C:\Python31\python.exe
Happy birthday, Jackson ! I hear you're 1 today.
Happy birthday, 1 ! I hear you're Jackson today.
Happy birthday, Jackson ! I hear you're 1 today.
Happy birthday, Jackson ! I hear you're 1 today.
Happy birthday, Jackson ! I hear you're 1 today.
Happy birthday, Katherine ! I hear you're 1 today.
Happy birthday, Jackson ! I hear you're 12 today.
Happy birthday, Katherine ! I hear you're 12 today.
Happy birthday, Katherine ! I hear you're 12 today.

Press the enter key to exit.
```

- Odgovarajući kod birthday_wishes.py je:

```
# Birthday Wishes
# Demonstrates keyword arguments and default parameter values

# positional parameters
def birthday1(name, age):
    print("Happy birthday,", name, "!", " I hear you're", age, "today.\n")

# parameters with default values
def birthday2(name = "Jackson", age = 1):
    print("Happy birthday,", name, "!", " I hear you're", age, "today.\n")

# main
birthday1("Jackson", 1)
birthday1(1, "Jackson")
birthday1(name = "Jackson", age = 1)
birthday1(age = 1, name = "Jackson")

birthday2()
birthday2(name = "Katherine")
birthday2(age = 12)
birthday2(name = "Katherine", age = 12)
birthday2("Katherine", 12)

input("\n\nPress the enter key to exit.")
```

Pozicioni argumenti i parametri

- U zaglavlju funkcije se, kao što je pokazano, navodi lista promenljivih, to jest parametara; u ovom primeru, zaglavlje prve funkcije je:

```
def birthday1(name, age):
```

- Kada se funkcija pozove pomoću liste vrednosti-argumenata, definišu se *pozicioni argumenti* ("positional argument"); u primeru, poziv funkcije je:

```
birthday1("Jackson", 1)
```

- Primena pozicionih argumenata znači da parametri funkcije dobijaju svoje vrednosti redom, na osnovu pozicije (rednog broja u nizu) argumenata koji su navedeni u pozivu: prvi parametar dobija vrednost prvog argumenta, drugi drugog, i tako redom (zato parametre nazivamo pozicioni)

- U ovom primeru, name dobija vrednost "Jackson", age dobija vrednost 1, što rezultuje porukom na ekranu:

Happy Birthday, Jackson ! I hear you're 1 today.

- Ako se u primeru izmene mesta dva argumenta, pošto se poštuje pozicija, parametri će dobiti obrnute vrednosti; znači, posle poziva:

```
birthday1(1, "Jackson")
```

name dobija vrednost 1, a age "Jackson", što je neispravna dodela, pa poruka glasi:

Happy Birthday, 1 ! I hear you're Jackson today.

- U Pythonu postoje i drugi načini da parametrima dodele vrednosti, kao što će nastavak teksta pokazati

Keyword argumenti

- Kao što je pokazano, parametri dobijaju vrednosti argumenata po redu; ali funkciji se može narediti da dodeljuje određene vrednosti nekim parametrima, nezavisno od redosleda (pozicije), tako što se koriste *keyword argumenti* (čuva se engleski naziv)

- Pomoću keyword argumenata, koriste se (kao ključne reči) nazivi parametara iz zaglavlja funkcije, kako bi se nekom parametru dodelila određena vrednost; u ovom primeru, ako se funkcija pozove pomoću linije:

```
birthday1(name = "Jackson", age = 1)
```

ona će dati ispravnu poruku (kao posledicu ispravne dodele)

- Prilikom upotrebe keyword argumenata, redosled nije bitan, jer se dodela obavlja prema nazivima parametara funkcije; tako će linija:

```
birthday1(age = 1, name = "Jackson")
```

takođe proizvesti ispravnu dodelu; argumenti se tako mogu zadavati u proizvoljnom redosledu, što je fleksibilno

- Teorijski se mogu kombinovati pozicioni i keyword argumenti ali to nije preporučljivo: programer treba da se opredeli za jedno: ili poziciono, ili uz keywords, i da ostane dosledan u celom programu
- Takođe se može primetiti da se prilikom upotrebe keyword argumenata delimično narušava koncept kapsulacije: da bi se ispisao kod u glavnom delu programa mora se "zaviriti" u kod funkcije (što znači i da on mora biti vidljiv)

Default vrednost parametara

- Postoji intuitivna opcija da se parametrima neke funkcije dodele default (podrazumevane, koje ostaju validne sve dok se drugačije ne kaže) vrednosti; u slučaju da se, prilikom poziva funkcije, odgovarajućem argumentu ne dodeli nikakva vrednost (koju bi preuzeo odgovarajući parametar), parametar čuva svoju default vrednost
- U pokazanom primeru, zaglavlje druge funkcije je:

```
def birthday2(name = "Jackson", age = 1):
```

što znači da ako promenljivoj-parametru name nije zadata vrednost pri pozivu, ona treba da bude (podrazumeva se da je) "Jackson" (slično i za age, za koju se podrazumeva vrednost 1)

- Tako će se prvi poziv:

```
birthday2()
```

izvršiti bez generisanja greške, jer će parametri zadržati svoje default vrednosti, i ispisaće se poruka:

Happy Birthday, Jackson ! I hear you're 1 today

- Pažnja: default vrednost se dodeljuje ili svakom od parametara funkcije, ili nijednom; ako se nekima dodeli a nekima ne, generisaće se greška

- Pomoću drugog, trećeg i četvrtog poziva (pomoću keyword argumenata):

```
birthday2(name = "Katherine")  
birthday2(age = 12)  
birthday2(name = "Katherine", age = 12)
```

name dobija vrednost "Katherine" (umesto default), a age čuva default; age dobija vrednost 12 (umesto default) a name čuva default; oba parametra dobijaju nove vrednosti (umesto default)

- Konačno, pomoću petog poziva (pomoću pozicionih argumenata):

```
birthday2("Katherine", 12)
```

postigne se isto što i pomoću četvrtog

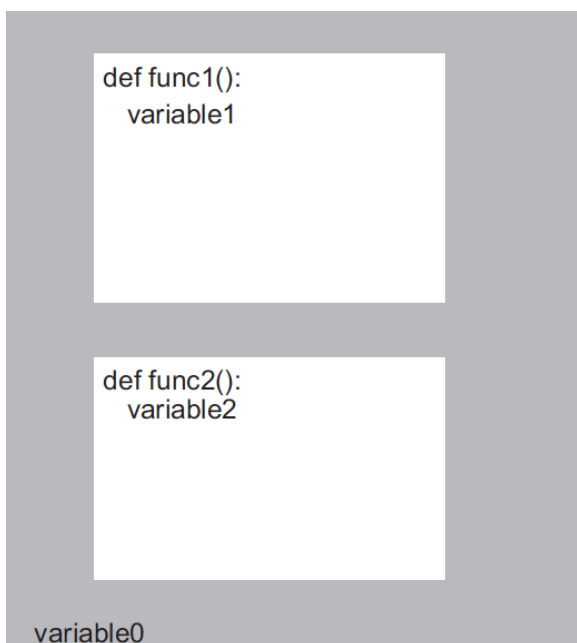
- Default vrednosti parametara su pogodne za upotrebu kada se pri pozivu neke funkcije skoro uvek šalje jedna ista vrednost argumenta (koja se proglašava za default)

6.5 Globalne promenljive i konstante

- Zahvaljujući kapsulaciji, sve funkcije koje su dosad pokazane su bile "zapečaćene", to jest "crne kutije", i nezavisne jedna od druge i od glavnog programa; jedini način komunikacije između njih je tako što jedna drugoj mogu da šalju vrednosti argumenata i dobiju povratne vrednosti
- Ipak, postoji još jedan način komunikacije između funkcija, to jest izmene podataka između njih: pomoću globalnih promenljivih

Područje funkcije

- Svaka funkcija ima svoje *područje* ("scope"); područja funkcija se, osim ako nije drugačije određeno, ne preklapaju; zbog toga funkcije koje su dosad prikazane nisu mogle "videti" promenljive drugih funkcija -- Slika 6.7

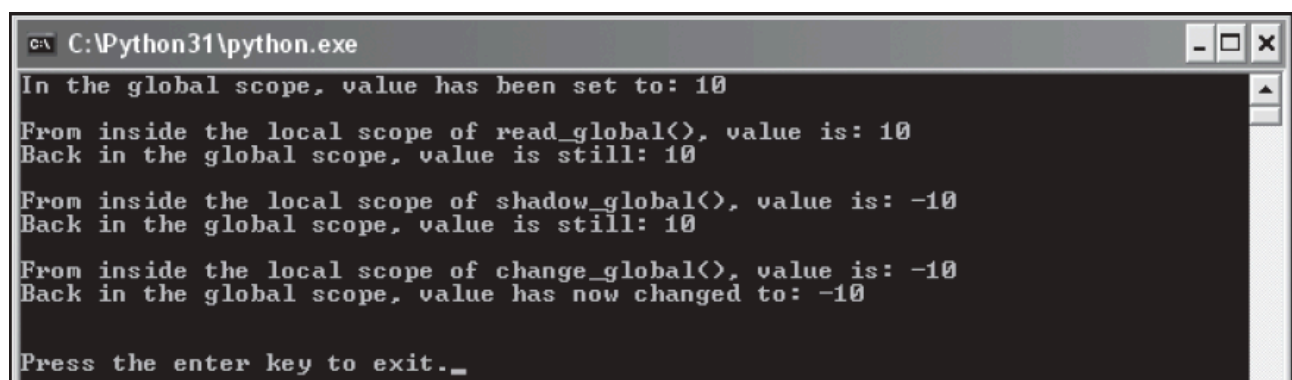


- Program na slici ima tri različita područja:
 - prvo je lokalno, definisano funkcijom func1() -- gornji beli pravougaonik
 - drugo je lokalno, definisano funkcijom func2() -- donji beli pravougaonik
 - treće je globalno, definisano glavnim programom -- siva površina

- Svaka promenljiva definisana u globalnom području se naziva *globalna promenljiva*; svaka promenljiva definisana u nekom lokalnom području (funkciji) se naziva *lokalna promenljiva* -- ona je lokalizovana samo u okviru date funkcije
- Promenljiva `variable1` je definisana u okviru funkcije `func1()`, pa je to lokalna promenljiva koja postoji samo u okviru te funkcije, i druge je ne mogu "videti"
- Znači, u okviru jedne funkcije, može se raditi sa svim njenim promenljivama, ali se nijedna od njih ne "vidi" iz spoljnjeg sveta; navedeno omogućava da, na primer, u `func1()` i `func2()` postoje lokalne promenljive sa istim imenom -- jedna drugoj neće smetati
- Sa druge strane, `variable0` je *globalna promenljiva*; ona se, kao što će biti detaljnije pokazano, "vidi" iz svake funkcije (što narušava koncept kapsulacije), kao naravno i iz glavnog programa (u kome je i definisana)
- Jedna ilustracija kako ovo funkcioniše: svako područje se može posmatrati kao jedna kuća sa zatamnjenim prozorima: od spolja se ne vidi šta je u kući; da bi se videlo, u nju mora da se uđe; iz nje se kroz zatamnjeni prozor vidi ono što se nalazi napolju

Program Global Reach

- Program Global Reach (globalni pristup) prikazuje kako se uvode globalne promenljive, i kako se one čitaju i menjaju iz funkcija -- Slika 6.8



```

C:\Python31\python.exe
In the global scope, value has been set to: 10
From inside the local scope of read_global(), value is: 10
Back in the global scope, value is still: 10
From inside the local scope of shadow_global(), value is: -10
Back in the global scope, value is still: 10
From inside the local scope of change_global(), value is: -10
Back in the global scope, value has now changed to: -10
Press the enter key to exit._

```

- Odgovarajući kod `global_reach.py` je:

```
# Global Reach
# Demonstrates global variables

def read_global():
    print("From inside the local scope of read_global(), value is:", value)

def shadow_global():
    value = -10
    print("From inside the local scope of shadow_global(), value is:", value)

def change_global():
    global value
    value = -10
    print("From inside the local scope of change_global(), value is:", value)

# main
# value is a global variable because we're in the global scope here
value = 10
print("In the global scope, value has been set to:", value, "\n")

read_global()
print("Back in the global scope, value is still:", value, "\n")

shadow_global()
print("Back in the global scope, value is still:", value, "\n")

change_global()
print("Back in the global scope, value has now changed to:", value)

input("\n\nPress the enter key to exit.")
```

Čitanje globalne promenljive iz funkcije

- Vrednost globalne promenljive može se čitati iz svake funkcije nekog programa; u prikazanom kodu, promenljivoj `value` je dodeljena vrednost 10 u glavnom programu, i ona time automatski (bez da se to eksplicitno naglasi) postaje globalna; prilikom poziva funkcije `read_global()`, ta vrednost se očitava

- Međutim, iako se može očitati, vrednost globalne promenljive se ne može menjati/modifikovati iz funkcije (osim ako to nije posebno omogućeno) -- ako bi na primer funkcija `read_global()` sadržala liniju:

```
value += 1
```

generisala bi se greška

- U ilustraciji sa kućama: iz kuće se kroz prozor može videti ono šta je napolju, ali se ništa napolju ne može izmeniti (jer je prozor zatvoren)

Maskiranje globalne promenljive iz funkcije

- Ako se nekoj promenljivoj unutar funkcije da isto ime kao ime neke globalne promenljive, kaže se da je izvršeno *maskiranje* ("shadowing") globalne promenljive; to znači da je lokalno zaklonjena/zamenjena drugom, lokalnom, promenljivom
- Na taj način, u okviru funkcije ne radi se sa globalnom promenljivom, već samo sa kreiranom lokalnom promenljivom (maskom), koja samo nosi isto ime kao globalna, i koja ostaje validna samo u okviru te funkcije
- U prikazanom kodu, u funkciji `shadow_global()`, promenljivoj `value` se dodeljuje vrednost u liniji:

```
value = -10
```

- Ovom linijom, vrednost globalne promenljive `value` nije promenjena, već je samo u okviru pomenute funkcije kreirana nova, lokalna promenljiva istog imena, i dobila je vrednost `-10`; kada se u glavnom programu vrednost globalne promenljive bude ispisala na ekranu, videće se da i dalje iznosi `10`
- Iako je maskiranje teorijski uvek moguće, treba ga izbegavati jer može dovesti do konfuzije: nije pametno koristiti isto ime za dve različite promenljive, u kom god području se nalazile

Promena vrednosti globalne promenljive iz funkcije

- Da bi se u okviru neke funkcije omogućio potpuni pristup (koji omogućava i modifikaciju) nekoj globalnoj promenljivoj, koristi se ključna reč `global`; u primeru, to je urađeno u liniji:

```
global value
```

Ovim je stavljeno do znanja da će se u okviru funkcije `change_global()` promenljivoj `value` pristupati na uobičajeni način

- Na ovaj način, funkcija može sa tom promenljivom da obavlja svaku operaciju, na primer promenu vrednosti:

```
value = -10
```

Da je vrednost promenjena, vidi se prilikom ispisivanja na ekranu u glavnom programu

Kada koristiti globalne promenljive i konstante

- Dobra programerska praksa nalaže da, iako je nešto moguće uraditi, možda nije pametno: globalne promenljive mogu uneti veliku konfuziju u program, i preporuka je da ih se što manje koristi
- Sa druge strane, globalne konstante (to jest globalne promenljive koje se tretiraju kao konstante) su pogodne: na primer, `TAX_RATE=0.28`

6.6 Povratak na igru TTT

- Pre ispisivanja koda za igru TTT, biće prikazano prvo njegovo planiranje, kako bi se dobila šira slika i shvatilo kako se prave obimniji programi

Planiranje igre TTT

- Najvažniji deo programiranja je planiranje programa -- biće prikazano na primeru igre TTT, korak po korak

Pisanje pseudokoda

- Prvo se piše pseudokod; u ovom primeru, pošto će svi zadaci biti povereni funkcijama, može se o glavnom programu razmišljati na prilično visokom nivou: svaka linija pseudokoda će postati poziv jedne ili više funkcija

- Pseudokod je:

```
display the game instructions
determine who goes first
create an empty tic-tac-toe board
display the board
while nobody's won and it's not a tie
    if it's the human's turn
        get the human's move
        update the board with the move
    otherwise
        calculate the computer's move
        update the board with the move
    display the board
    switch turns
congratulate the winner or declare a tie
```

- Apstraktni (grubi) pseudokod je vrlo bitan, jer će direktno proizvesti strukturu programa: na primer, očigledno je da će se na početku obaviti četiri pripremne radnje, i da će se glavni deo programa (sama igra) izvršavati u okviru while-petlje (u kojoj će igrači igrati naizmenično)

Predstavljanje podataka (table)

- Ideje iz pseudokoda su znači za sada prilično apstraktne, ali se postepeno moraju konkretizovati u smislu da se odredi kako tačno da se implementiraju u kodu; u narednim tačkama će se ta konkretizacija postepeno obavljati

- Aktuelno stanje table sa $3 \times 3 = 9$ polja, na kojoj se igra, će se prikazivati na ekranu; u svakom polju će se nalaziti ili slova-karakter ("X" ili "O") ili razmak " " (da označi prazno polje); sama tabla treba da bude lista, koja će se menjati/ažurirati prilikom svakog poteza
- Znači, lista koja predstavlja tablu treba da ima 9 elemenata; svako polje na tabli će odgovarati jednoj poziciji u listi -- Slika 6.9

0	1	2
3	4	5
6	7	8

- Svako polje na tabli je predstavljeno brojem, od 0 do 8; to znači da će lista imati indekse od 0 do 8; pošto se svaki potez (upisivanje slova X ili O) igra na određeno polje, to će se i svaki potez predstavljati brojem u istom opsegu
- Strane na kojima igraju igrač i računar se predstavljaju sa "X" i sa "O"; promenljiva koja predstavlja stranu koja je na potezu imaće takođe jednu od ove dve vrednosti

Kreiranje liste funkcija

- Na osnovu pseudokoda može se kreirati lista potrebnih funkcija; ona sadrži ime funkcije i opis argumenata (parametara) i povratnih vrednosti -- Tabela 6.1

FUNKCIJA	OPIS
<code>display_instruct()</code>	prikazuje pravila igre
<code>ask_yes_no(question)</code>	postavlja pitanje i očekuje odgovor "y" ili "n"; prihvata pitanje; vraća "y" ili "n"
<code>ask_number(question, low, high)</code>	traži broj u nekom opsegu; prihvata pitanje, donji limit i gornji limit; vraća broj
<code>pieces()</code>	određuje ko igra prvi; vraća računarovo i igračevo slovo (X ili O)
<code>new_board()</code>	pravi novu, praznu tablu; vraća tablu
<code>display_board(board)</code>	prikazuje tablu na ekranu; prihvata tablu
<code>legal_moves(board)</code>	pravi listu dozvoljenih poteza; prihvata tablu; vraća listu
<code>winner(board)</code>	određuje porednika; prihvata tablu; vraća slovo porednika, "TIE" (nerešeno) ili None
<code>human_move(board, human)</code>	dobija igračev potez; prihvata tablu i igračevo slovo; vraća igračev potez
<code>computer_move(board, computer, human)</code>	proračunava računarov potez; prihvata tablu, računarovo slovo, i igračevo slovo; vraća računarov potez
<code>next_turn(turn)</code>	određuje ko igra sledeći; prihvata slovo; vraća slovo
<code>congrat_winner(the_winner, computer, human)</code>	čestita poredniku ili proglašava nerešeno; prihvata porednikovo slovo, računarovo slovo i igračevo slovo

Pripremne radnje

- Kod za igru TTT, `tic-tactoe.py`, biće predstavljan redom deo po deo
- Prvo što se u programu uradi je kreiranje globalnih konstanti -- to su vrednosti koje će se koristiti u više funkcija; ako ih treba menjati, menjaju se samo na jednom mestu; ovaj korak nije eksplicitno napisan u pseudokodu, jer je njegov nivo previsok
- Globalne konstante se kreiraju u delu koda:

```
# global constants
X = "X"
O = "O"
EMPTY = " "
TIE = "TIE"
NUM_SQUARES = 9
```

- Konstante su: X je skraćenica za string "X", slovo jednog učesnika; O je za "O", slovo drugog učesnika; EMPTY je razmak, koji će označavati prazno polje; TIE predstavlja nerešeni ishod; NUM_SQUARES je broj polja na tabli, to jest 9

Funkcija display_instruct()

- Ova funkcija ispisuje pravila igre na ekranu (trivijalna je i neće se opisivati), što odgovara prvoj liniji pseudokoda

Funkcija ask_yes_no()

- Ova funkcija postavlja pitanje na koje igrač odgovara sa yes ili no -- ona prihvata tekst pitanja i vraća odgovor igrača, "y" ili "n"

```
def ask_yes_no(question):
    """Ask a yes or no question."""
    response = None
    while response not in ("y", "n"):
        response = input(question).lower()
    return response
```

- Očigledno je da u ovom primeru ne postoji velika potreba za funkcijom kao što je navedena (kao i neke naredne); program se može jednostavnije napisati i bez nje, to jest ona deluje kao da otežava kod; ali ovde se radi o školskom primeru koji pokazuje kako se koriste funkcije; uz to, ovakvu funkciju programer može jedanput da napiše, i zatim da je koristi u svim budućim programima koje će pisati

Funkcija ask_number()

- Ova funkcija traži da joj igrač zada celi broj u zadatom opsegu; ona prihvata tekst pitanja, donji i gornji limit opsega, a vraća broj koga je igrač uneo:

```
def ask_number(question, low, high):
    """Ask for a number within a range."""
    response = None
    while response not in range(low, high):
        response = int(input(question))
    return response
```

Funkcija pieces()

- Ova funkcija na početku igre pita igrača da li hoće da igra prvi, i vraća informaciju čije je koje slovo; po tradiciji, onaj ko prvi igra ima slovo X

```
def pieces():
    """Determine if player or computer goes first."""
    go_first = ask_yes_no("Do you require the first move? (y/n): ")
    if go_first == "y":
        print("\nThen take the first move. You will need it.")
        human = X
        computer = O
    else:
        print("\nYour bravery will be your undoing... I will go first.")
        computer = X
        human = O
    return computer, human
```

- Ova funkcija odgovara drugoj liniji u pseudokodu; ona poziva funkciju ask_yes_no(), što je sasvim moguće -- funkcije mogu pozivaju jedna drugu
- Ova funkcija kao konačni rezultat vraća dve promenljive: computer i human, koje će imati vrednosti X i O, ili O i X

Funkcija new_board()

- Ova funkcija, koja odgovara trećoj liniji pseudokoda, generiše i vraća praznu tablu -- to je lista board sa NUM_SQUARES elemenata, i svakom od njih je dodeljena vrednost EMPTY:

```
def new_board():
    """Create new game board."""
    board = []
    for square in range(NUM_SQUARES):
        board.append(EMPTY)
    return board
```

Funkcija display_board()

- Ova funkcija, koja u pseudokodu odgovara četvrtoj liniji i pretposlednjoj liniji u while-petlji, ispisuje na ekranu tablu koju je prihvatila u pozivu:

```
def display_board(board):
    """Display game board on screen."""
    print("\n\t", board[0], "|", board[1], "|", board[2])
    print("\t", "-----")
    print("\t", board[3], "|", board[4], "|", board[5])
    print("\t", "-----")
    print("\t", board[6], "|", board[7], "|", board[8], "\n")
```

- Pomoću horizontalnih i vertikalnih linija, ova funkcija ispisuje na ekranu listu board, po tri njena elementa u svakoj vrsti

Funkcija legal_moves()

- Ova funkcija dobija tablu i vraća listu dozvoljenih (mogućih) poteza; nju koriste druge funkcije (koje će kasnije biti uvedene):

- human_move() da bi se osiguralo da je igrač odigrao dozvoljeni potez
- computer_move() da bi računar, prilikom odluke koji potez da odigra, uzeo u obzir samo dozvoljene

- Svi dozvoljeni potezi se nalaze u listi moves, koju ova funkcija vraća; broj u toj listi (između 0 i 8) ukazuje na to da je odgovarajuće polje prazno, to jest da se na tom mestu u listi board nalazi razmak; na primer, ako je centralno polje prazno, onda se u listi moves nalazi broj 4

- Znači, ova funkcija skenira listu board element po element, i svaki put kad naiđe na razmak (to jest EMPTY) ona dodaje odgovarajući broj u listu moves:

```
def legal_moves(board):
    """Create list of legal moves."""
    moves = []
    for square in range(NUM_SQUARES):
        if board[square] == EMPTY:
            moves.append(square)
    return moves
```

Funkcija winner()

- Ova funkcija prihvata tablu i vraća ime pobjednika; postoje četiri mogućnosti:
 - ako je jedan učesnik pobedio, funkcija vraća njegovo slovo (X ili O)
 - ako su sva polja popunjena i niko nije pobedio (nerešeno), funkcija vraća TIE
 - ako nisu sva polja popunjena (ima još da se igra), funkcija vraća None
- Prvo što se u funkciji radi je kreiranje konstante WAYS_TO_WIN koja sadrži svih 8 pobjedničkih situacija na tabli, kada se ista tri znaka nađu u nizu
- Svaki pobjednička situacija je predstavljena kao n-torka; svaka n-torka je sekvenca od tri polja na tabli -- kada se isti znak nađe u tim poljima, to označava pobjedu
- Primer: prva n-torka je (0, 1, 2) i označava prvu vrstu matrice (table), to jest pozicije 0, 1 i 2; druga je (3, 4, 5) i označava drugu vrstu matrice; n-torki ima ukupno 8 (za tri vrste, tri kolone i dve dijagonale):

```
def winner(board):
    """Determine the game winner."""
    WAYS_TO_WIN = ((0, 1, 2),
                    (3, 4, 5),
                    (6, 7, 8),
                    (0, 3, 6),
                    (1, 4, 7),
                    (2, 5, 8),
                    (0, 4, 8),
                    (2, 4, 6))
```

- Zatim se koristi for petlja: za svaku od pobedničkih n-torki se proverava, pomoću if, da li su odgovarajuća slova sva ista; ako jesu, i nisu razmak, igrač čije je to slovo je pobednik

- Primer:

- uzima se druga "pobednička" n-torka (3, 4, 5) i dodeli promenljivoj row
- zatim se proverava da li se na pozicijama table row[0]=3, row[1]=4 i row[2]=5 nalazi isto slovo
- ako se nalazi, i ako ono nije razmak (to jest EMPTY), to znači da je to slovo X ili O: igrač čije je to slovo je pobedio (jer je upisao svoje slovo u celu drugu vrstu matrice)
- to slovo se dodeljuje promenljivoj winner koju funkcija vraća

- Ako niko nije pobedio, funkcija se nastavlja tako što dalje proverava da li je ostalo neko prazno polje:

- ako nije, vraća TIE, to jest igra je završena bez pobednika (nerešeno)
- ako jeste, vraća None, to jest pobednika nema i igra se nastavlja

- Odgovarajući deo koda je:

```
for row in WAYS_TO_WIN:
    if board[row[0]] == board[row[1]] == board[row[2]] != EMPTY:
        winner = board[row[0]]
        return winner

if EMPTY not in board:
    return TIE

return None
```

- U drugoj liniji pokazano je kako se istovremeno obavlja više operacija poređenja (da li su tri vrednosti iste, i istovremeno različite od četvrte) -- to Python dozvoljava

Funkcija human_move()

- Ova funkcija prihvata tablu i igračevo slovo, a vraća redni broj polja gde je igrač rešio da odigra:

- funkcija prvo traži listu dozvoljenih poteza od funkcije `legal_moves()`
- zatim, funkcija pita igrača koji potez hoće da odigra, s tim što mu dozvoljava da odigra samo jedan od dozvoljenih
- kad igrač odigra (dozvoljeni) potez, funkcija vraća vrednost koja odgovara tom potezu

- Odgovarajući deo koda je:

```
def human_move(board, human):
    """Get human move."""
    legal = legal_moves(board)
    move = None
    while move not in legal:
        move = ask_number("Where will you move? (0 - 8):", 0, NUM_SQUARES)
        if move not in legal:
            print("\nThat square is already occupied. Choose another.\n")
    print("Fine...")
    return move
```

Funkcija `computer_move()`

- Ova funkcija dobija tablu, računarovo slovo (X ili O), i igračevo slovo; ona vraća računarov potez -- ovo je najsloženija funkcija u programu

- Prva ideja koja pada na pamet kao najjednostavnija je da se slučajno odabere neki potez od dozvoljenih; može se napisati odgovarajuća funkcija (koju bismo na primer nazvali `computer_move_random()`), kako bi se program što pre kompletirao

- Kasnije se može umesto te funkcije napisati funkcija koja odgovara nekom sofisticiranijem izboru poteza; pošto funkcije omogućavaju modularnost, jednostavno se umesto poziva jedne funkcije odradi poziv druge; ovde će odmah biti prikazana sofisticiranija verzija

- Treba imati u vidu da je tabla lista, što znači da je promenljiva; u ovoj funkciji će se birati najbolji potez koga računar može da odigra, pa će se zato tabla menjati, što će prouzrokovati da bude promenjena i u delu programa odakle je funkcija pozvana, kao rezultat deljenih referenci (postoji samo jedna kopija, koja se na taj način menja)

- Da bi se navedena promena table izbegla, prvo što se uradi u funkciji je pravljenje njene lokalne kopije sa kojom će se u funkciji raditi:

```
def computer_move(board, computer, human):  
    """Make computer move."""  
    # make a copy to work with since function will be changing list  
    board = board[:]
```

- Ponovo: treba obratiti pažnju svaki put kada se neka promenljiva vrednost zadaje nekoj funkciji -- ako će se ta vrednost menjati u okviru funkcije (a da se pri tome ne želi da se promeni i na mestu odakle je pozvana), treba napraviti lokalnu kopiju

- Pada na pamet i sledeća mogućnost: da se u okviru funkcije zaista promeni tabla tako da uvaži novi računarov potez; tako funkcija ne bi morala da vraća računarov potez; ipak, ovo nije pametno raditi: dobra programerska praksa nalaže da funkcija komunicira sa ostatkom programa pomoću povratnih vrednosti i nikako drugačije

- Strategija kojom računar igra je odabrana na sledeći način:

1. Ako postoji potez koga računar može odigrati tako da pobedi (računarov pobednički potez), on bira taj potez
2. Ako postoji potez koga igrač može odigrati, posle tekućeg računarovog poteza, tako da pobedi (igračev pobednički potez), on bira taj potez (tako da preduhitri igrača)
3. Inače, računar treba da izabere najbolji mogući potez: pokazuje se da je najbolje da odigra u centar, pa ako ne može u uglove, pa ako ne može onda u ostala četiri polja

- Za gornju tačku 3, definiše se n-torka konstanta u kojoj se čuvaju potezi od najboljeg do najgoreg:

```
# the best positions to have, in order
BEST_MOVES = (4, 0, 2, 6, 8, 1, 3, 5, 7)

print("I shall take square number", end=" ")
```

- Zatim se kreira lista dozvoljenih poteza pomoću `legal_moves()`
- Zatim se otvara for petlja: računar pokušava da odigra redom svaki od dozvoljenih poteza (to jest da stavi svoje slovo redom na svako od slobodnih polja), pa:
 - ako je neki od tih poteza računarov porednički, onda će ga on odigrati: u tom slučaju funkcija vraća taj potez i završava se
 - inače, računar povlači taj potez (vraća EMPTY na mesto odigranog slova), i pokušava sledeći
- Ovo je urađeno u sledećem delu koda:

```
# if computer can win, take that move
for move in legal_moves(board):
    board[move] = computer
    if winner(board) == computer:
        print(move)
        return move
    # done checking this move, undo it
    board[move] = EMPTY
```

- Ako se stigne do ovog mesta u funkciji, to znači da računar nema na raspolaganju porednički potez (tačka 1 strategije); zato će računar gledati da igrača spreči da odigra svoj porednički potez (tačka 2 strategije)
- Zato se otvara druga for petlja posle prikazane: računar stavlja igračevo slovo na sva prazna polja, pa:
 - ako je neki od tih poteza (igračev) porednički, to je potez koga treba sprečiti, i računar će to odigrati: u tom slučaju funkcija vraća taj potez i završava se
 - inače, taj potez se povlači (vraća se EMPTY) i pokušava sledeći
- Ovo je urađeno u sledećem delu koda:

```
# if human can win, block that move
for move in legal_moves(board):
    board[move] = human
    if winner(board) == human:
        print(move)
        return move
# done checkin this move, undo it
board[move] = EMPTY
```

- Ako se stigne do ovog mesta u funkciji, nijedna strana ne može da pobedi (tačka strategije 3), pa računar zato ide kroz listu najboljih poteza BEST_MOVES, uzima prvi koji je dozvoljen, i taj potez vraća:

```
# since no one can win on next move, pick best open square
for move in BEST_MOVES:
    if move in legal_moves(board):
        print(move)
        return move
```

- Napomena: igra TTT je vrlo jednostavna (trivijalna), i dovoljno je razmatrati samo svoj naredni potez; u ozbiljnijim igrama strategije, kao što je šah, potrebno je (kako bi se dobro igralo) predviđati mnogo više unapred, sopstvene poteze i poteze protivnika; ovom problematikom bavi se veštačka inteligencija

Funkcija next_turn()

- Ova funkcija je jednostavna: na osnovu toga ko je odigrao (turn), ona određuje ko igra sledeći (next_turn); znači, ako joj se zada X ona vraća O, i obrnuto

```
def next_turn(turn):
    """Switch turns."""
    if turn == X:
        return O
    else:
        return X
```

Funkcija congrat_winner()

- Ova funkcija prihvata ime pobjednika (tačnije, slovo pobjednika ili TIE), računarovo slovo i igračevo slovo, ispisuje ime pobjednika i odgovarajući tekst na ekranu; poziva se tek kad je igra završena

- Odgovarajući kod je:

```
def congrat_winner(the_winner, computer, human):
    """Congratulate the winner."""
    if the_winner != TIE:
        print(the_winner, "won!\n")
    else:
        print("It's a tie!\n")

    if the_winner == computer:
        print("As I predicted, human, I am triumphant once more. \n" \
              "Proof that computers are superior to humans in all regards.")

    elif the_winner == human:
        print("No, no! It cannot be! Somehow you tricked me, human. \n" \
              "But never again! I, the computer, so swear it!")

    elif the_winner == TIE:
        print("You were most lucky, human, and somehow managed to tie me. \n" \
              "Celebrate today... for this is the best you will ever achieve.")
```

Funkcija main()

- Glavni program je stavljen u posebnu funkciju, umesto da se ostavi u globalnom području; time je i glavni kod kapsuliran, što je obično dobra ideja; takođe, praksa nalaže da se ta funkcija zove main()
- Glavni kod skoro sasvim oponaša napisani pseudokod:

```

def main():
    display_instruct()
    computer, human = pieces()
    turn = X
    board = new_board()
    display_board(board)

    while not winner(board):
        if turn == human:
            move = human_move(board, human)
            board[move] = human
        else:
            move = computer_move(board, computer, human)
            board[move] = computer
        display_board(board)
        turn = next_turn(turn)

    the_winner = winner(board)
    congrat_winner(the_winner, computer, human)

```

Pokretanje programa

- Sledeće linije formalno čine glavni program -- one pozivaju funkciju main() iz globalnog područja, a main() dalje poziva ostale funkcije:

```

# start the program
main()
input("\n\nPress the enter key to quit.")

```

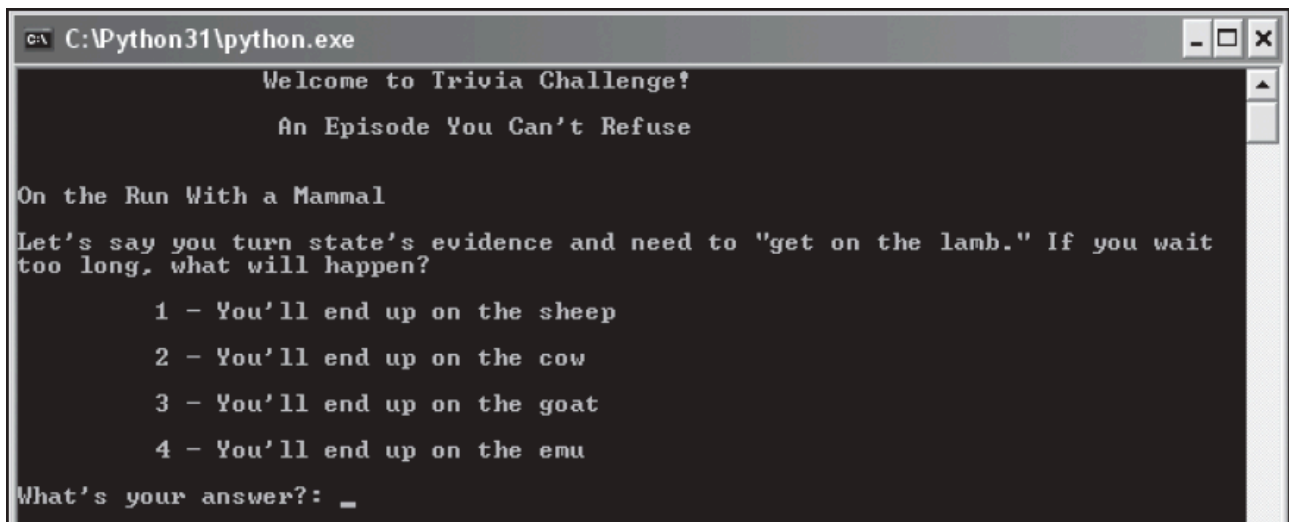
- Prikazana struktura programa je u skladu sa dobrom programerskom praksom:
 - definišu se prvo globalne konstante
 - definišu se funkcije nekim pogodnim redom, s tim što je poslednja main()
 - iz globalnog područja poziva se main(), i ništa više

Poglavlje 7: Fajlovi i izuzeci

- Promenljive su idealne za čuvanje i pristup podacima u toku izvršenja programa; međutim, često je potrebno da neki podaci ostanu sačuvani i kada korisnik završi sa korišćenjem programa (i isključi računar) kako bi se koristili sledeći put, to jest da ostanu sačuvani "za stalno"
- Ovo poglavlje pokazaće kako se, u cilju stalnog čuvanja podataka, koriste fajlovi (zastareli naziv: datoteka) koji se zapisuju na disk (hard ili neki drugi); uz to će pokazati kako se postupa sa greškama koje se generišu u toku izvršenja nekog programa
- Podsećanje: hard-disk (ili neki drugi slični medijum za smeštaj podataka) ima osobinu da podaci koji su na njemu upisani budu očuvani i kada prestane električno napajanje; u slučaju hard-diska, podaci se fizički predstavljaju pomoću namagnetisanih ćelijica (malih površina na disku) -- jedan smer magnetizacije označava logičku 0, a drugi logičku 1
- Preciznije, u poglavlju će biti pokazano:
 - očitavanje ("read") sadržaja iz tekst fajla
 - upisivanje ("write") sadržaja u tekst fajl
 - očitavanje i upisivanje kompleksnih podataka iz/u fajlove
 - postupanje u slučaju grešaka koje kod generiše tokom izvršenja

7.1 Igra Trivia Challenge

- Igra Trivia Challenge (Slika 7.1) testira znanje igrača tako što mu postavlja seriju pitanja uz više ponuđenih odgovora (od kojih je samo jedan tačan); pitanja su u vezi sa mafijom; ona se čuvaju u posebnom fajlu; taj fajl se može uređivati pomoću bilo kog editora (kao što je Notepad u Windows-u), nezavisno od napisanog koda

A screenshot of a Windows command prompt window titled "C:\Python31\python.exe". The window displays a trivia game interface. The text shown is: "Welcome to Trivia Challenge!", "An Episode You Can't Refuse", "On the Run With a Mammal", "Let's say you turn state's evidence and need to 'get on the lamb.' If you wait too long, what will happen?", a list of four options: "1 - You'll end up on the sheep", "2 - You'll end up on the cow", "3 - You'll end up on the goat", "4 - You'll end up on the emu", and a prompt "What's your answer?: _".

```
C:\Python31\python.exe
Welcome to Trivia Challenge!
An Episode You Can't Refuse

On the Run With a Mammal
Let's say you turn state's evidence and need to "get on the lamb." If you wait
too long, what will happen?

1 - You'll end up on the sheep
2 - You'll end up on the cow
3 - You'll end up on the goat
4 - You'll end up on the emu

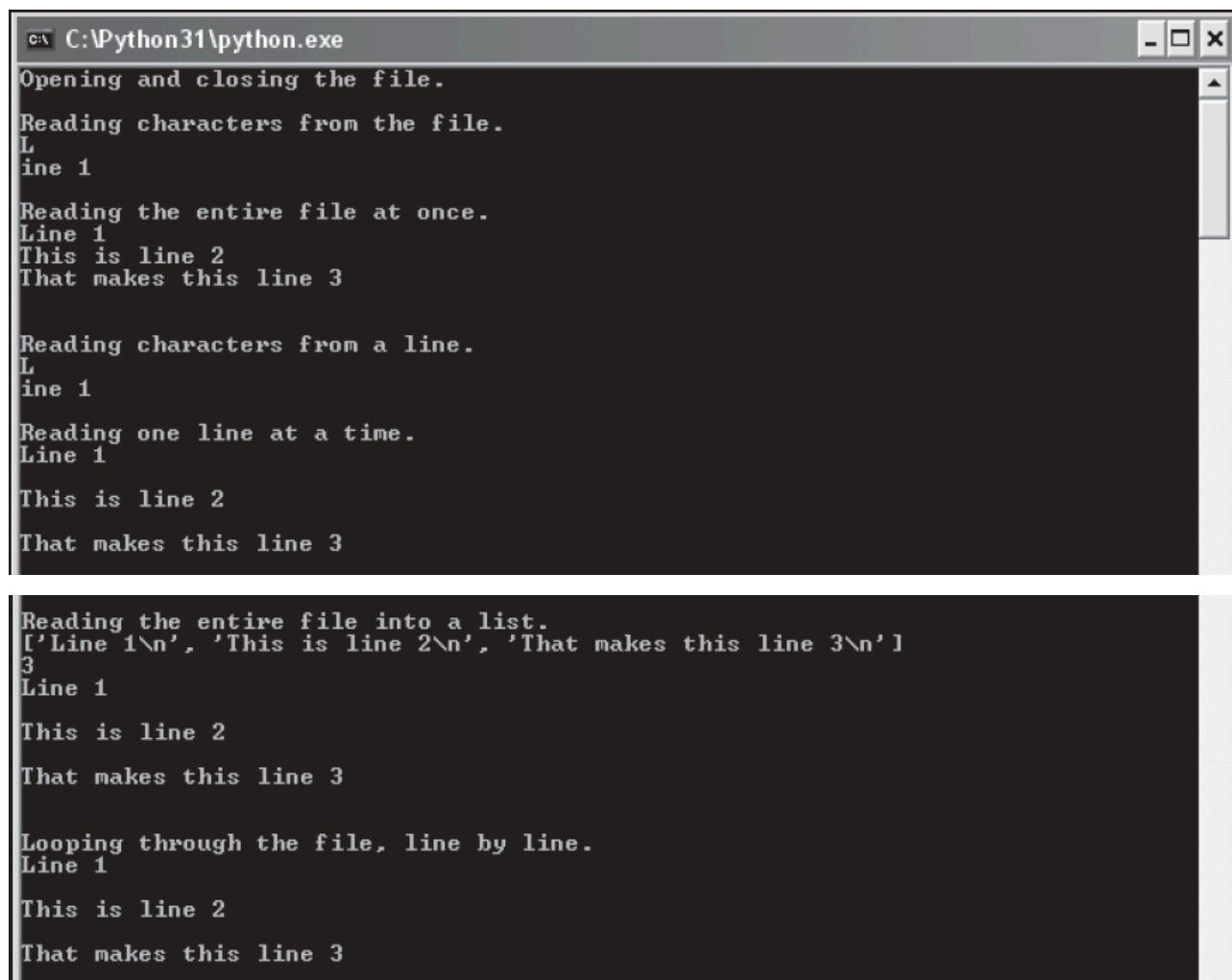
What's your answer?: _
```

7.2 Očitavanje sadržaja iz tekst fajla

- *Tekst fajlovi* ("text file") su najjednostavniji način da se čuvaju tekstualni podaci; njihova karakteristika je da mogu da sadrže samo ASCII karaktere
- Drugi tip fajlova koji mogu da služe za čuvanje tekstualnih podataka, i koji pružaju više mogućnosti (primer: formatiran tekst, na primer italic), su *binarni fajlovi* ("binary file"); oni će biti tretirani kasnije u poglavlju
- Podsećanje: ASCII je standardna šema za kodiranje karaktera; šema uključuje 128 karaktera (slova, cifre, znaci interpunkcije, matematički simboli i razni specijalni karakteri), i svaki je kodiran sa 7 bita; primer: slovo "j" je kodirano kao 1101010; poslednjih godina, primat ovoj šemi je sasvim preuzela nova koja se zove UTF-8
- Tekst fajlovi su najjednostavniji za upotrebu iz dva razloga:
 - nezavisni su od računarske platforme -- koriste se bez problema u svakom OS
 - najlakši su razumevanje i korišćenje, a pokrivaju većinu potreba običnog korisnika

Program Read It

- Program Read It prikazuje razne načine kako se može očitavati sadržaj nekog tekst fajla; pokazuje kako se očitava sve od jednog karaktera do celog fajla; pokazuje i načine kako se očitava fajl liniju po liniju -- Slika 7.2



The screenshot shows a Python console window titled 'C:\Python31\python.exe'. It displays the output of a program that demonstrates various file reading methods. The output is as follows:

```

Opening and closing the file.
Reading characters from the file.
L
ine 1

Reading the entire file at once.
Line 1
This is line 2
That makes this line 3

Reading characters from a line.
L
ine 1

Reading one line at a time.
Line 1

This is line 2

That makes this line 3

Reading the entire file into a list.
['Line 1\n', 'This is line 2\n', 'That makes this line 3\n']
3
Line 1

This is line 2

That makes this line 3

Looping through the file, line by line.
Line 1

This is line 2

That makes this line 3

```

- Program očitava jednostavni tekst fajl, read_it.txt, kreiran u editoru:

```

Line 1
This is line 2
That makes this line 3

```

- Kod programa, read_it.py, koji će biti predstavljen pomoću interaktivnih sesija, je:

```

# Read It
# Demonstrates reading from a text file

print("Opening and closing the file.")
text_file = open("read_it.txt", "r")
text_file.close()

print("\nReading characters from the file.")
text_file = open("read_it.txt", "r")
print(text_file.read(1))
print(text_file.read(5))
text_file.close()

print("\nReading the entire file at once.")
text_file = open("read_it.txt", "r")
whole_thing = text_file.read()
print(whole_thing)
text_file.close()

print("\nReading characters from a line.")
text_file = open("read_it.txt", "r")
print(text_file.readline(1))
print(text_file.readline(5))
text_file.close()

print("\nReading one line at a time.")
text_file = open("read_it.txt", "r")
print(text_file.readline())
print(text_file.readline())
print(text_file.readline())
text_file.close()

print("\nReading the entire file into a list.")
text_file = open("read_it.txt", "r")
lines = text_file.readlines()
print(lines)
print(len(lines))
for line in lines:
    print(line)
text_file.close()

print("\nLooping through the file, line by line.")
text_file = open("read_it.txt", "r")
for line in text_file:
    print(line)
text_file.close()

input("\n\nPress the enter key to exit.")

```

Otvaranje i zatvaranje fajla

- Pre nego što se obavi očitavanje ili upisivanje iz/u nekog tekst fajla, on prvo mora da se otvori; za to služi funkcija `open()`:

```
>>> text_file = open("read_it.txt", "r")
```

- Kao prvi argument funkcije `open()` pojavljuje se ime tekst fajla (ekstenzija mu je uvek `txt`); pošto u primeru drugačije nije navedeno (difolt opcija), računar će tražiti navedeni fajl u tekućem folderu

- U opštem slučaju, za fajl se može navesti kompletna putanja pomoću foldera, na uobičajeni način, na primer:

- Windows: `open("C:/Users/Desktop/alice.txt", "r")`

- MacOS and Linux: `open("/Users/Desktop/alice.txt", "r")`

- Drugi argument je mod pristupa ("access mode") -- u primeru, "r" označava da će se fajl očitavati (mod za očitavanje); inače se fajl može otvarati zbog očitavanja, upisivanja, ili oboje; Tabela 7.1 prikazuje validne modove pristupa

Mode	Description
"r"	Read from a text file. If the file doesn't exist, Python will complain with an error.
"w"	Write to a text file. If the file exists, its contents are overwritten. If the file doesn't exist, it's created.
"a"	Append a text file. If the file exists, new data is appended to it. If the file doesn't exist, it's created.
"r+"	Read from and write to a text file. If the file doesn't exist, Python will complain with an error.
"w+"	Write to and read from a text file. If the file exists, its contents are overwritten. If the file doesn't exist, it's created.
"a+"	Append and read from a text file. If the file exists, new data is appended to it. If the file doesn't exist, it's created.

- Pošto se fajl očita, rezultat očitavanja se dodeljuje promenljivoj `text_file` -- ona predstavlja objekat tipa fajl (objekat je jedan apstraktni termin vezan za OO programiranje, koji se ovde samo pominje bez objašnjavanja)

- Fajl se zatvara pozivanjem metode `close()`:

```
>>> text_file.close()
```

Korišćenje ove funkcije se uvek preporučuje; posle zatvaranja, sa fajlom se više ništa ne može raditi sve dok se sledeći put ne otvori

Očitavanje pojedinačnih karaktera iz fajla

- Kada se fajl otvori, njegov sadržaj se može očitati pomoću metode `read()`; ona omogućava da se očita prvih `n` karaktera iz fajla (gde je `n` argument), koje metoda vraća kao string

- U navedenom primeru, kod:

```
>>> text_file = open("read_it.txt", "r")
>>> print(text_file.read(1))
L
```

kao rezultat daje da metoda vraća prvi karakter, to jest "L"

- Zatim, očitava se narednih pet karaktera:

```
>>> print(text_file.read(5))
ine 1
```

- U prethodnom primeru, očitavaju se pet karaktera posle "L": Python pamti gde se poslednji put zaustavio, tako da u narednom očitavanju odbrojava počev od tog mesta; kada se stigne do kraja fajla, vraća se prazan string

- Da bi se ponovo počelo očitavanje fajla od početka, on se mora zatvoriti pa otvoriti:

```
>>> text_file.close()
>>> text_file = open("read_it.txt", "r")
```

- Ako se pri pozivu `read()` ne navede broj karaktera koje treba učitati, Python će vratiti ceo fajl kao string -- u narednom delu koda, očitava se ceo fajl, vraćeni string se dodeljuje promenljivoj `whole_thing`, koja se zatim ispisuje na ekranu:

```
>>> whole_thing = text_file.read()
>>> print(whole_thing)
Line 1
This is line 2
That makes this line 3
```

- Svako naredno očitavanje bi vratilo prazan string; na kraju fajl treba zatvoriti pomoću `close()`

Očitavanje karaktera iz jedne linije

- Ponekad je pogodno iz fajla očitati samo jednu liniju -- metoda `readline()` omogućava očitavanje karaktera iz tekuće linije: potrebno je kao argument metode navesti broj karaktera koje iz tekuće linije treba očitati, i metoda će ih vratiti kao string

- Pravila:

- i ovde Python pamti do kog karaktera je stigao, i sledeći put nastavlja od te pozicije (koju nazivamo tekuća)
- ako se ne zada broj karaktera, Python će po defaultu očitati sve karaktere od tekuće pozicije pa do kraja linije
- kada se očitaju svi karakteri iz tekuće linije, naredna linija (prva ispod) postaje tekuća
- tekuća linija je inicijalno prva linija

- U navedenom primeru, pošto se ponovo fajl otvori, očitava se prvi karakter tekuće linije:

```
>>> text_file = open("read_it.txt", "r")
>>> print(text_file.readline(1))
L
```

posle čega se očitava narednih pet karaktera iz tekuće linije, i zatvori fajl:

```
>>> print(text_file.readline(5))
ine 1
>>> text_file.close()
```

- U narednim linijama koda, očitava se fajl liniju po liniju:

```
>>> text_file = open("read_it.txt", "r")
>>> print(text_file.readline())
Line 1

>>> print(text_file.readline())
This is line 2

>>> print(text_file.readline())
That makes this line 3

>>> text_file.close()
```

Posle očitavanja linije 1, tekuća linija postaje 2, i zatim 3; očitavaju se svi karakteri od tekućeg karaktera, to jest prvog, pa do kraja linije

- Posle svake očitane linije pojavljuje se jedna prazna; to je zato što se svaka linija u tekst fajlu završava karakterom koji označava novu liniju ("`\n`"), pa se njegovim "ispisivanjem" na ekranu kreira prazna linija

Očitavanje svih linija u listu

- Još jedan način da se pristupa linijama nekog tekst fajla je pomoću metode `readlines()` (množina!), koja očitava tekst fajl u listu tako što svaka linija fajla postaje jedan (string) element liste:

```
>>> text_file = open("read_it.txt", "r")
>>> lines = text_file.readlines()
>>> print(lines)
['Line 1\n', 'This is line 2\n', 'That makes this line 3\n']
```

Obratiti pažnju na prisustvo karaktera za novu liniju `\n` koji u tekst fajlu nisu bili vidljivi

- Nastala lista je kao i svaka druga -- može se određivati njena dužina pomoću `len()`, a može se i očitavati u okviru petlje:

```
>>> print(len(lines))
3
>>> for line in lines:
print(line)

Line 1

This is line 2

That makes this line 3

>>> text_file.close()
```

Pristup sadržaju fajla pomoću petlje

- Moguće je i direktno pristupati linijama nekog tekst fajla pomoću for petlje:

```
>>> text_file = open("read_it.txt", "r")
>>> for line in text_file:
print(line)

Line 1

This is line 2

That makes this line 3

>>> text_file.close()
```

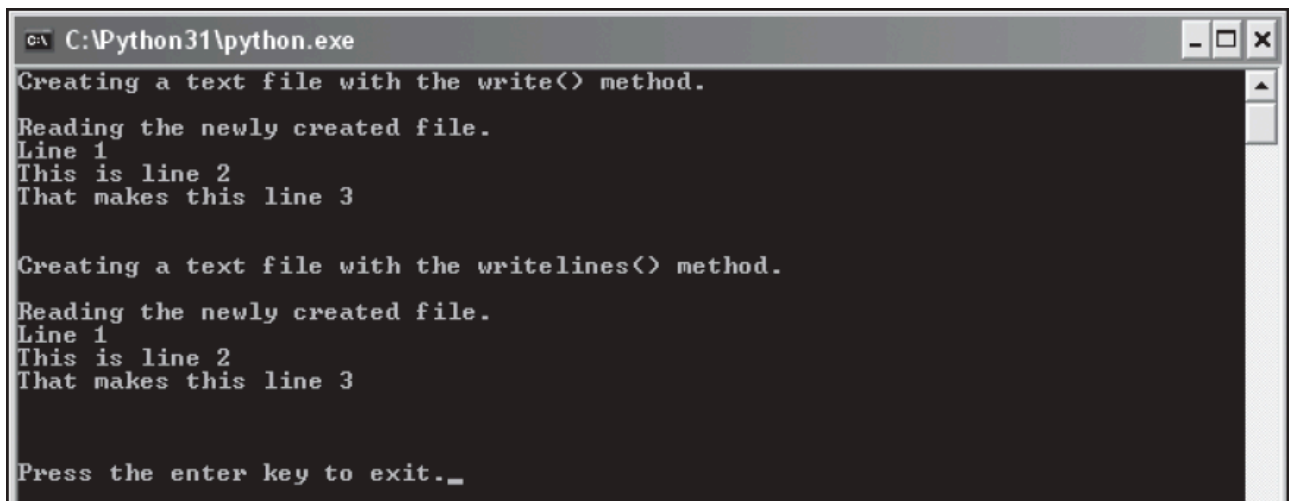
Promenljivoj petlje (line) se dodeljuje redom linija po linija iz tekst fajla

7.3 Upisivanje sadržaja u tekst fajl

- Da bi tekst fajlovi bili od koristi, potrebno je u njih i upisivati iz Pythona; za to će biti pokazana dva načina

Program Write It

- Program Write It pravi tekst fajl istog sadržaja kao read_it.txt; zapravo, pravi takav fajl dva puta, na dva načina, to jest upotrebom dve različite metode -- Slika 7.3



```
C:\Python31\python.exe
Creating a text file with the write() method.
Reading the newly created file.
Line 1
This is line 2
That makes this line 3

Creating a text file with the writelines() method.
Reading the newly created file.
Line 1
This is line 2
That makes this line 3

Press the enter key to exit._
```

- Odgovarajući kod, write_it.py, biće predstavljan deo po deo

Upisivanje stringova u fajl

- Fajl se prvo mora otvoriti u modu za upisivanje (oznaka "w"):

```
# Write It
# Demonstrates writing to a text file

print("Creating a text file with the write() method.")
text_file = open("write_it.txt", "w")
```

- Ovo "otvaranje" dosad nepostojećeg fajla write_it.txt zapravo znači njegovo kreiranje u obliku praznog fajla, koji je spreman da se u njega upisuje; ako fajl sa tim imenom već postoji, biće zamenjen novim praznim fajlom (to jest inicijalni sadržaj se briše)

- Zatim, koristi se metoda write() koja upisuje redom tri stringa u taj fajl, posle čega se fajl zatvara

```
text_file.write("Line 1\n")
text_file.write("This is line 2\n")
text_file.write("That makes this line 3\n")
text_file.close()
```

- Ova metoda automatski ne umeće karakter za novu liniju ("\n"); taj karakter mora upisati programer; da ih ovde nije upisao, stringovi bi se nastavljali jedan na drugi u istoj (jedinoj) liniji

- Isto upisivanje može da se obavi i pomoću jedne linije:

```
text_file.write("Line 1\n This is line 2\n That makes this line 3\n")
```

- Zatim, da bi se proverilo da li je upisivanje bilo uspešno, ceo fajl se očitava i ispisuje na ekranu:

```
print("\nReading the newly created file.")
text_file = open("write_it.txt", "r")
print(text_file.read())
text_file.close()
```

Upisivanje liste stringova u fajl

- Isti fajl se zatim kreira na drugi način, pomoću metode writelines(); ona listu stringova upisuje u fajl; fajl se prvo otvara za upisivanje (čime se prethodni briše):

```
print("\nCreating a text file with the writelines() method.")
text_file = open("write_it.txt", "w")
```

- Zatim se kreira lines, lista stringova koji se upisuju, pri čemu se svaka linija mora označiti karakterom za novu liniju:

```
lines = ["Line 1\n",
        "This is line 2\n",
        "That makes this line 3\n"]
```

- Zatim se lista upisuje u fajl, i fajl zatvara:

```
text_file.writelines(lines)
text_file.close()
```

- Na kraju se ponovo proveriti da li je upisivanje uspešno izvršeno:

```
print("\nReading the newly created file.")
text_file = open("write_it.txt", "r")
print(text_file.read())
text_file.close()

input("\n\nPress the enter key to exit.")
```

- Pregled metoda za očitavanje i upisivanje je dat u Tabeli 7.2:

Method	Description
<code>close()</code>	Closes the file. A closed file cannot be read from or written to until opened again.
<code>read([size])</code>	Reads <i>size</i> characters from a file and returns them as a string. If <i>size</i> is not specified, the method returns all of the characters from the current position to the end of the file.
<code>readline([size])</code>	Reads <i>size</i> characters from the current line in a file and returns them as a string. If <i>size</i> is not specified, the method returns all of the characters from the current position to the end of the line.
<code>readlines()</code>	Reads all of the lines in a file and returns them as elements in a list.
<code>write(output)</code>	Writes the string <i>output</i> to a file.
<code>writelines(output)</code>	Writes the strings in the list <i>output</i> to a file.

7.4 Čuvanje kompleksnih podataka u fajlu

- Tekst fajlovi su pogodni zbog svoje jednostavnosti: oni se mogu obrađivati pomoću bilo kog tekst editora; vrlo često su podaci koje treba predstaviti dovoljno jednostavni da se mogu čuvati u tekst fajlu

- Primer jednostavnih podataka: lista količine padavina u mm u gradovima u Srbiji 2014 (godina sa poplavama), može biti smeštena u fajlu padavine_2014.txt u obliku:

```
Beograd      1103
Kragujevac   979
Valjevo      1333
S. Palanka   1042
Novi Sad     821
Niš          956
```

- Sa druge strane, tekst fajlovi imaju ograničenu fleksibilnost u smislu da se u njima mogu čuvati samo nizovi karaktera; ako korisnik želi da u fajlu sačuva neke kompleksnije podatke (kako će kasnije biti pokazano), tekst fajlovi nisu pogodni

- Ideja koja se prva nameće je da se dati kompleksni podatak na neki način pretvori u nizove karaktera i da se tako snimi kao tekst fajl; to međutim nije pogodno jer se između ostalog javlja problem kako rekonstruisati originalni (izvorni) oblik podatka posle očitavanja takvog fajla
- U Pythonu se kompleksni podaci mogu snimati u fajl i očitavati iz njega pomoću samo jedne linije koda; čak se i neka jednostavna baza podataka može na taj način sačuvati u jednom fajlu; za to je prvo potrebno da se, umesto sa tekst fajlovima, radi sa binarnim fajlovima

Binarni fajl

- *Binarni fajl* ("binary file", ekstenzija u opštem slučaju mu je .bin) je, formalno, svaki fajl koji nije tekst fajl, to jest svaki fajl koji služi da predstavi sve one podatke koji nisu jednostavni tekst (povorka karaktera): to su na primer formatirani tekst (boja slova, veličina, font...), ali i fotografije, zvučni zapisi, izvršni fajlovi (ekstenzija .exe) i slično
- Binarni fajl je stoga onaj fajl čiji sadržaj mora biti interpretiran od strane nekog programa (ili procesora), koji unapred zna po kojim pravilima su podaci u fajlu formatirani; ako neki program koji ta pravila ne zna pokuša da očita binarni fajl (ili ako korisnik otvori taj fajl u editoru), videće samo gomilu neupotrebljivih simbola
- Svaki binarni fajl ima svoje zaglavlje ("header"), koje sadrži identifikaciju fajla, to jest informaciju o pravilima po kojima treba interpretirati podatke koji se u fajlu sadrže
- Na primer, u jednom .gif fajlu koji predstavlja neku sliku, prvi bajtovi u zaglavlju predstavljaju tekst GIF87a; tako svi programi pomoću kojih se pokušava otvaranje fajla

znaju o kakvom se fajlu radi, pa tako on postaje koristan samo onim programima koji su namenjeni da otvaraju .gif fajlove

- U ovom poglavlju, koristiće se binarni fajlovi sa ekstenzijom .dat; ova ekstenzija označava generičke fajlove sa podacima; svaki program može da formira svoj fajl sa tom ekstenzijom prema svojim specifičnim pravilima, tako da neki drugi program koji ta pravila ne zna ne može da fajl očita
- Zbog navedene osobine, binarni fajlovi nisu namenjeni da se otvaraju "ručno", to jest pomoću tekst editora; da bi otvorio takav fajl, korisnik mora da zna koji program treba da koristi ("Open with..." opcija u Windows-u)

Kompleksni podaci

- U binarne fajlove će se upisivati, kao što je već navedeno, kompleksni podaci; da bi se ovaj pojam definisao, potrebno je prvo definisati (suprotni) pojam jednostavnog podatka
- *Jednostavni podatak* je onaj podatak koji se predstavlja samo jednom promenljivom; tu spadaju sve strukture podataka koje su dosad uvedene: broj, string, n-torka, lista ili rečnik
- Definicija: pod *kompleksnim podatkom* ovde će se smatrati svaki skup čiji su elementi jednostavni podaci koje zajedno treba čuvati u jednom fajlu (jer su na primer logički povezani); svaki od tih jednostavnih podataka (to jest elemenata pomenutog skupa) nazivamo *objekat*
- Primer: jedan kompleksni podatak opisuje profil Petra Petrovića na FB; to je skup koji se sastoji od sledećih pet elemenata, to jest jednostavnih podataka (jedan string, dva cela broja, dve liste); svaki od pet podataka predstavlja jedan objekat:

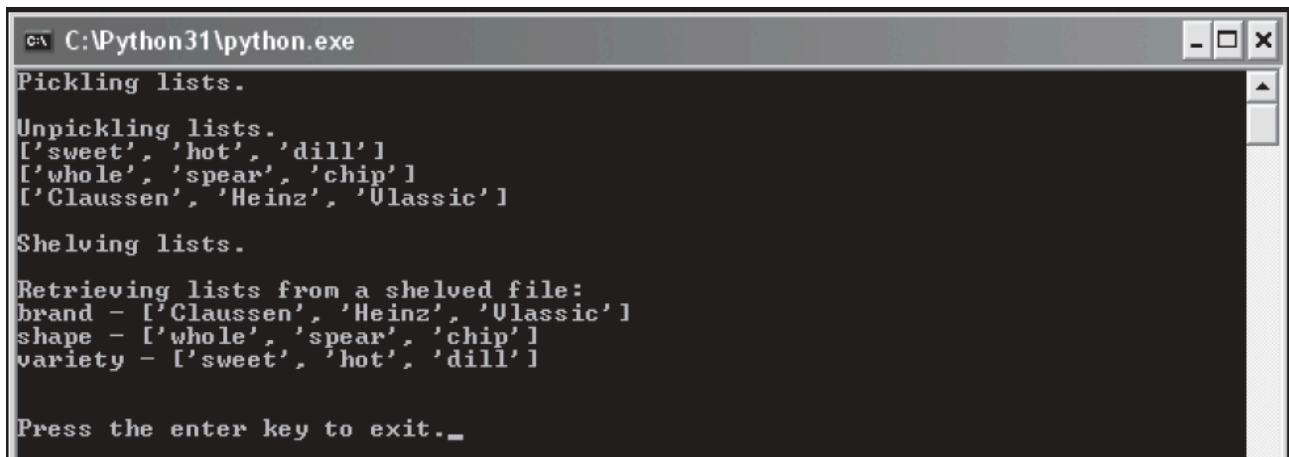
```
ID = 365891
ime_prezime = "Petar Petrovic"
god_rodj = 1995
prijatelji = ["Ivan Ivanovic", "Jovan Jovanovic"]
poseceni_gradovi = ["Rim", "Prag"]
```

- Ako treba sačuvati ceo kompleksni podatak u nekom fajlu, onda se u fajl upisuje objekat po objekat: u gornjem primeru, svi podaci će se smestiti u binarni fajl koji na primer nosi ime 365891.dat (jedininstveno ime prema jedinstvenom ID)
- U trivijalnom slučaju, kompleksni podatak može činiti samo jedan jednostavan podatak (broj članova skupa je jednak 1), što znači da taj kompleksni podatak sadrži samo jedan objekat, koji se takođe može upisati u binarni fajl

Program Pickle It

- *Piklovanje* ("pickling" = kišelj enje, stavljanje u turšiju (?!); glagol: piklovati = "to pickle"; imenica: "pickle" = kiseli krastavac; koristi se i srpski prevod "serijalizovanje" kao generički pojam, ali ga izbegavamo) je postupak pomoću koga se neki kompleksni podatak iz svog originalnog oblika pretvori (pikluje) u neki pogodan (piklovani) oblik, koji se snima u binarni fajl
- Kada se kasnije otvori taj binarni fajl, i preuzme piklovani kompleksni podatak, on se pomoću obrnutog postupka koji se naziva *depiklovanje* ("unpickling") vraća u svoj originalni oblik
- Program Pickle It pokazaće kako se kompleksni podatak koga čine tri liste (koji tako čine njegove objekte), čiji su elementi stringovi:
 - pikluje, pa zatim u piklovanom obliku snimi u binarni fajl
 - učitava iz binarnog fajla i zatim depikluje u originalni oblik

- Ovo će se uraditi na dva načina, kao što prikazuje Slika 7.4: prvo, tako što će se objekti upisati i očitavati sekvencijalno, i zatim isto to ali slučajno ("random")



```

C:\Python31\python.exe
Pickling lists.
Unpickling lists.
['sweet', 'hot', 'dill']
['whole', 'spear', 'chip']
['Claussen', 'Heinz', 'Ulassic']
Shelving lists.
Retrieving lists from a shelved file:
brand - ['Claussen', 'Heinz', 'Ulassic']
shape - ['whole', 'spear', 'chip']
variety - ['sweet', 'hot', 'dill']
Press the enter key to exit._

```

- Odgovarajući kod, pickle_it.py biće predstavljen deo po deo

Piklovanje podataka i njihovo upisivanje u fajl

- Prvo što se radi je učitavanje/uvoženje dva nova modula:

```

# Pickle It
# Demonstrates pickling and shelving data

import pickle, shelve

```

- Modul pickle služi da se kompleksni podaci pikluju i sekvencijalno smeštaju u binarni fajl, i iz njega sekvencijalno očitavaju; modul shelve obavlja isti zadatak, izuzev što je pristup podacima slučajan (proizvoljan), po principu rečnika
- Piklovanje je jednostavno: piklovaní objekti se redom (sekvencijalno) upisuju u binarni fajl; njihovo očitavanje će takođe biti sekvencijalno
- Prvo, u programu se kreiraju objekti koji čine kompleksni podatak; u ovom slučaju, to su tri liste:

```
print("Pickling lists.")
variety = ["sweet", "hot", "dill"]
shape = ["whole", "spear", "chip"]
brand = ["Claussen", "Heinz", "Vlassic"]
```

- Zatim, otvara/kreira se novi binarni fajl koji će služiti za smeštanje kompleksnog podatka:

```
f = open("pickles1.dat", "wb")
```

Kao mod pristupa, zadato je "wb", što označava upis u binarni fajl

- Dozvoljeni modovi pristupa dati su u Tabeli 7.3:

TABLE 7.3 SELECTED BINARY FILE ACCESS MODES

Mode	Description
"rb"	Read from a binary file. If the file doesn't exist, Python will complain with an error.
"wb"	Write to a binary file. If the file exists, its contents are overwritten. If the file doesn't exist, it's created.
"ab"	Append a binary file. If the file exists, new data is appended to it. If the file doesn't exist, it's created.
"rb+"	Read from and write to a binary file. If the file doesn't exist, Python will complain with an error.
"wb+"	Write to and read from a binary file. If the file exists, its contents are overwritten. If the file doesn't exist, it's created.
"ab+"	Append and read from a binary file. If the file exists, new data is appended to it. If the file doesn't exist, it's created.

- Zatim, pomoću funkcije `pickle.dump()`, tri liste (objekti kompleksnog podatka) se pikluju i sekvencijalno upisuju u fajl `pickles1.dat`, posle čega se fajl zatvori:

```
pickle.dump(variety, f)
pickle.dump(shape, f)
pickle.dump(brand, f)
f.close()
```

Ova funkcija znači zahteva dva argumenta: objekat koji se upisuje, i fajl u koga se upisuje

- Kao rezultat, u binarnom fajlu ostaće sačuvana tri objekta kompleksnog podatka (u ovom slučaju, tri liste), svaki sa svojim sadržajem, sekvencijalno jedan za drugim

- Na koji način Python čuva podatke u binarnom fajlu, korisnik ne zna, niti mu je to od važnosti: jedino što je važno je da rekonstrukcija originalnog podatka bude verna

Očitavanje piklovanog podatka iz fajla i njegovo depiklovanje

- Pošto se fajl otvori sa modom pristupa "rb", tri liste (objekti) se sekvencijalno očitavaju iz fajla pickles1.dat i depikluju pomoću funkcije pickle.load():

```
print("\nUnpickling lists.")
f = open("pickles1.dat", "rb")
variety = pickle.load(f)
shape = pickle.load(f)
brand = pickle.load(f)
```

Ova funkcija zahteva jedan argument, a to je fajl iz koga se očitava sledeći objekat

- Preciznije:
 - program očitava prvi piklovani objekat iz fajla
 - depikluje ga, čime generiše listu ["sweet", "hot", "dill"]
 - dodeljuje listu promenljivoj variety
 - zatim isto radi sa drugim pa sa trećim piklovanim objektom

Ovo se odvija sekvencijalno, to jest po redu, objekat po objekat

- Na kraju, tri dobijene promenljive se ispisuju na ekranu, i fajl se zatvara:

```
print(variety)
print(shape)
print(brand)
f.close()
```

- Tabela 4.7 prikazuje dve pomenute funkcije:

TABLE 7.4 **SELECTED PICKLE FUNCTIONS**

Function	Description
<code>dump(object, file, [,bin])</code>	Writes pickled version of object to file. If bin is True, object is written in binary format. If bin is False, object is written in less efficient, but more human-readable, text format. The default value of bin is equal to False.
<code>load(file)</code>	Unpickles and returns the next pickled object in file.

Upotreba police za upisivanje podataka

- Pomoću funkcija `dump()` i `load()` modula `pickle`, pokazano je kako se podaci pikluju i upisuju, to jest očitavaju i depikluju, sekvencijalno; kao analogiju, možemo zamisliti i koncept (uzane) tegle sa kiselim krastavcima, u koju se oni moraju redom stavljati i iz nje redom vaditi

- Novi koncept koji se uvodi je koncept police za knjige ("shelf"): na nju se knjige stavljaju tako da se svakoj od njih može pristupiti nezavisno od drugih; uz to, knjiga se na polici pronade tako što se njen naziv pročita na koricama

- Tako se uvodi modul `shelve`, koji omogućava slučajan ("random") pristup objektima po principu rečnika, to jest prema ključu

- Prvo, kreira se promenljiva s koja predstavlja policu:

```
print("\nShelving lists.")
s = shelve.open("pickles2.dat")
```

- Funkcija `shelve.open()` otvara/kreira novi binarni fajl `pickles2.dat`, koji se zatim dodeljuje promenljivoj `s` (koja igra ulogu police); ova promenljiva će funkcionisati kao rečnik, to jest omogućiće pretraživanje celog svog sadržaja pomoću ključa

- Funkcija `shelve.open()` zahteva samo jedan argument, a to je ime fajla; može joj se opciono zadati i mod pristupa (po defaultu to je "c")
- Modovi pristupa funkcije `shelve.open()` dati su u Tabeli 7.5

TABLE 7.5 SHELVES ACCESS MODES

Mode	Description
"c"	Open a file for reading or writing. If the file doesn't exist, it's created.
"n"	Create a new file for reading or writing. If the file exists, its contents are overwritten.
"r"	Read from a file. If the file doesn't exist, Python will complain with an error.
"w"	Write to a file. If the file doesn't exist, Python will complain with an error.

- Zatim se tri liste dodaju na policu:

```
s["variety"] = ["sweet", "hot", "dill"]
s["shape"] = ["whole", "spear", "chip"]
s["brand"] = ["Claussen", "Heinz", "Vlassic"]
```

- Na levoj strani se u uglastim zagradama navodi ime objekta (pod navodnicima, kao string), a na desnoj strani njegova vrednosti

- Polica znači funkcioniše kao rečnik:

- ključ je ime objekta, a vrednost je njegova vrednost, to jest u ovom slučaju ključ "variety" je uparen sa vrednošću ["sweet", "hot", "dill"], i slično za druge dve stavke

- pretraživanje će se obavljati pomoću ključa; po pravilu ključ mora biti string (kojim se označava ime objekta)

- Na kraju, poziva se metoda `sync()` kako bi se ažurirao sadržaj fajla prema novoj vrednosti promenljive koja predstavlja policu

Upotreba police za očitavanje podataka

- Pošto polica funkcioniše kao rečnik, objektima se može pristupati slučajno, tako što se zada vrednost ključa

- U primeru, objekti se preuzimaju redom obrnutim od onog kojim su upisivani, kako bi se pokazalo da redosled ovde ne igra ulogu (posle čega se fajl zatvara):

```
print("\nRetrieving lists from a shelved file:")
print("brand -", s["brand"])
print("shape -", s["shape"])
print("variety -", s["variety"])
s.close()
```

7.5 Obrada izuzetaka

- Kada Python naiđe na liniju koda koju nije u stanju da izvrši zbog neke greške (ili nekonzistentnosti), dešava se događaj koji se naziva *izuzetak* ("exception"); po difoltu, izuzetak izaziva prekid izvršenje programa i ispisivanje poruke o grešci

- Primer za izuzetak:

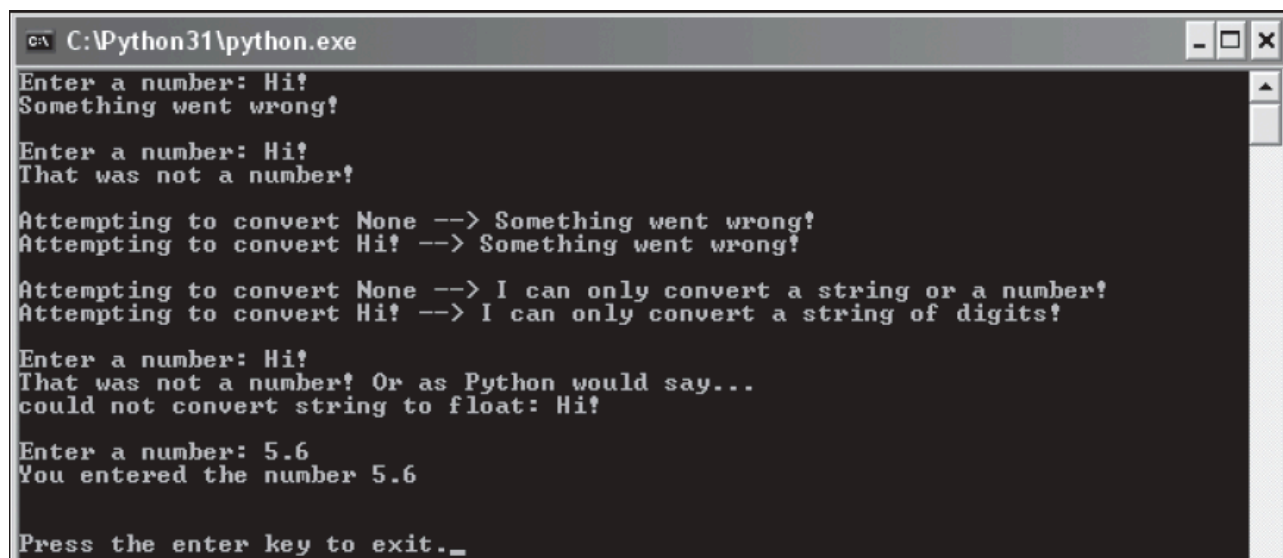
```
>>> num = float("Hi!")
Traceback (most recent call last):
File "<pyshell#1>", line 1, in <module>
num = float("Hi!")
ValueError: could not convert string to float: Hi!
```

Pri pokušaju da se string konvertuje u float, dolazi do izuzetka, zbog čega se izvršenje programa obustavlja, i javlja se poruka o grešci

- Python ima tu funkcionalnost da obrađuje ("intercept" -- presretanje ili "catch") izuzetak, tako što omogućava reakciju u slučaju njegovog nastanka, tako da se spreči prekid izvršenja programa; ako ništa drugo, umesto "brutalnog" prekida izvršenja, programer može da obezbedi da prekid bude elegantniji, na primer sa jasnije ispisanom porukom

Program Handle It

- Program Handle It obrađuje izuzetke nastale zbog nepravilnih podataka koje korisnik unosi, posle čega i sam namerno generiše neke izuzetke; ali umesto prekida izvršenja, program se u svim slučajevima izvrši do kraja, jer obrađuje nastale izuzetke -- Slika 7.5



```
C:\Python31\python.exe
Enter a number: Hi?
Something went wrong!

Enter a number: Hi?
That was not a number!

Attempting to convert None --> Something went wrong!
Attempting to convert Hi? --> Something went wrong!

Attempting to convert None --> I can only convert a string or a number!
Attempting to convert Hi? --> I can only convert a string of digits!

Enter a number: Hi?
That was not a number! Or as Python would say...
could not convert string to float: Hi!

Enter a number: 5.6
You entered the number 5.6

Press the enter key to exit._
```

- Odgovarajući kod, handle_it.py, biće predstavljen deo po deo

Upotreba naredbe try i odredbe except

- Najbolji način da se obradi izuzetak je pomoću naredbe try i odredbe except:
 - pomoću try, označava se deo koda koji potencijalno može da generiše izuzetak (gde postoji rizik da se izuzetak desi = "rizični" deo koda)
 - pomoću except, označava se deo koda koji se izvršava u slučaju da se desi izuzetak (to jest, deo koda koji obrađuje izuzetak = kod za obradu greške)
- U datom primeru, program traži od korisnika da unese broj, a zatim taj broj hoće da konvertuje u float; koriste se try i except da bi se predvidela mogućnost da će korisnik umesto broja uneti tekst (jer se smatra da rizik da se to desi postoji), što bi dovelo do izuzetka, to jest obrađuje se taj izuzetak:

```
# Handle It
# Demonstrates handling exceptions

# try/except
try:
    num = float(input("Enter a number: "))
except:
    print("Something went wrong!")
```

- Znači:

- try označava liniju koja je predviđena da se izvrši
- ako ne dođe do izuzetka (to jest ako korisnik unese broj, kao što je traženo), linija označena sa except se preskače i izvršenje nastavlja
- ako dođe do izuzetka (tako što korisnik unese tekst), umesto poruke o grešci i prekida izvršenja ispisuje se poruka na ekranu da nešto nije u redu

Obrada različitih tipova izuzetaka

- Različiti tipovi greške dovode do različitih tipova izuzetaka; na primer, pokušaj da se uneti tekst string konvertuje pomoću float() dovodi do izuzetka ValueError, jer uneti karakteri nisu cifre

- Tabela 7.6 daje pregled najznačajnijih tipova izuzetaka

Exception Type	Description
IOError	Raised when an I/O operation fails, such as when an attempt is made to open a nonexistent file in read mode.
IndexError	Raised when a sequence is indexed with a number of a nonexistent element.
KeyError	Raised when a dictionary key is not found.
NameError	Raised when a name (of a variable or function, for example) is not found.
SyntaxError	Raised when a syntax error is encountered.
TypeError	Raised when a built-in operation or function is applied to an object of inappropriate type.
ValueError	Raised when a built-in operation or function receives an argument that has the right type but an inappropriate value.
ZeroDivisionError	Raised when the second argument of a division or modulo operation is zero.

- Odredba except omogućava da se obrađuju različiti tipovi izuzetaka: posle except treba samo navesti tip izuzetka; tako se naznačava da u slučaju tog izuzetka treba odraditi odgovarajući deo koda

- U primeru, odgovarajuće linije koda su:

```
# specifying exception type
try:
    num = float(input("\nEnter a number: "))
except ValueError:
    print("That was not a number!")
```

- Znači, ako se unese string, desiće se izuzetak ValueError i ispisati odgovarajuća poruka da nije unet broj; ako se desi neki drugi tip izuzetka (što u ovom konkretnom slučaju nije moguće), izvršenje će se prekinuti i ispisati poruka o grešci

- Dobra programerska praksa nalaže da se predvidi svaki izuzetak koji može da nastane, i da se svaki posebno obradi; na primer, prilikom svake interakcije programa sa okruženjem treba predvideti šta sve može "poći naopako"; na primer, čak i ako programer veruje da fajl koga program očitava postoji, treba da predvidi i mogućnost da to nije slučaj

- Ako programer ne zna koji tip izuzetka će generisati određena greška, najbolje je da sam generiše taj izuzetak i pročita poruku o grešci koju je dobio -- u njoj piše tip

- Primer -- deljenje nulom daje:

```
>>> 1/0
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    1/0
ZeroDivisionError: int division or modulo by zero
```

pa je znači tip izuzetka ZeroDivisionError

Obrada više tipova izuzetaka

- Jedna linija koda može, u zavisnosti od okolnosti, generisati više tipova izuzetaka; svaki tip se može obrađivati

- U nekim slučajevima, više tipova se može obraditi na isti način, pomoću jedne except odredbe posle koje se navode svi tipovi u zagradi:

```
# handle multiple exception types
print()
for value in (None, "Hi!"):
    try:
        print("Attempting to convert", value, "-->", end=" ")
        print(float(value))
    except (TypeError, ValueError):
        print("Something went wrong!")
```

- Naime, ovde se pokušava konverzija dve vrednosti pomoću float(), i obe dovode do greške to jest izuzetka:

- float(None) generiše TypeError jer funkcija može da konvertuje samo stringove i brojeve
- float("Hi!") generiše ValueError, jer su karakteri u stringu pogrešna vrednost (nisu cifre)

Oba tipa izuzetka se obrađuju na isti način, porukom da nešto nije u redu

- Sa druge strane, svaki tip izuzetka se može obrađivati posebno, pomoću više except odredbi koje slede posle jedne try naredbe; to je ono što se uvek preporučuje:

```
print()
for value in (None, "Hi!"):
    try:
        print("Attempting to convert", value, "-->", end=" ")
        print(float(value))
    except TypeError:
        print("I can only convert a string or a number!")
    except ValueError:
        print("I can only convert a string of digits!")
```

Svakom izuzetku odgovara po jedna poruka

Upotreba argumenta izuzetka

- Kada se desi izuzetak, on obično ima i odgovarajući argument, koji predstavlja njegov malo precizniji opis (objašnjenje); to je tekst koji se u okviru poruke o grešci ispisuje posle tipa greške

- U primeru sa početka sekcije, kada je generisana greška ValueError, argument je bio:

- string "could not convert string to float:"

- na koga je nadovezan uneti string ("Hi!")

- Argumentu izuzetka se pristupa tako što se posle tipa izuzetka navede prvo ključna reč as, pa onda ime promenljive kojoj će se dodeliti vrednost argumenta izuzetka:

tip_izuzetka as ime_promenljive

- Odgovarajuće linije u primeru su:

```
# get an exception's argument
try:
    num = float(input("\nEnter a number: "))
except ValueError as e:
    print("That was not a number! Or as Python would say...")
    print(e)
```

- Ovde je vrednost argumenta dodeljena promenljivoj e, koja se zatim ispisuje posle poruke koju ispisuje programer

Dodavanje odredbe else

- Posle svih odredbi except, može se navesti odredba else -- odgovarajući blok koda se izvršava ako se ne desi nijedan izuzetak:

```
# try/except/else
try:
    num = float(input("\nEnter a number: "))
except ValueError:
    print("That was not a number!")
else:
    print("You entered the number", num)
```

- U navedenom primeru, num se (u poslednjoj liniji) ispisuje na ekranu samo ako dodela iz try bloka nije generisala izuzetak; drugim rečima, to je ona linija na koju se nastavlja izvršenje u slučaju da nema izuzetka

7.6 Povratak na igru Trivia Challenge

- Igra je napravljena tako da se koristi tekst fajl sa pitanjima i odgovorima, koji se može uređivati u editoru sasvim nezavisno od programskog koda; tekst fajl koji se očitava, trivia.txt, mora se nalaziti u istom folderu kao i .py fajl

Organizacija podataka u tekst fajlu

- Pre predstavljanja koda, treba razumeti kako su organizovani podaci u okviru tekst fajla trivia.txt: prva linija fajla je glavni naslov; ostatak fajla se sastoji od blokova, svaki blok čini 8 linija, po jedan blok za svako pitanje; broj blokova je znači jednak broju pitanja, i može biti proizvoljan (pišemo pitanja koliko god hoćemo)

- Generička reprezentacija jednog bloka (koji odgovara jednom pitanju) je:

```
<naslov>
<pitanje>
<odgovor 1>
<odgovor 2>
<odgovor 3>
<odgovor 4>
<tačan odgovor (1-4)>
<objašnjenje>
```

- Početak tekst fajla (glavni naslov plus dva bloka) je:

```

An Episode You Can't Refuse
On the Run With a Mammal
Let's say you turn state's evidence and need to "get /
on the lamb." If you wait too long, what will happen?
You'll end up on the sheep
You'll end up on the cow
You'll end up on the goat
You'll end up on the emu
1
A lamb is just a young sheep.
The Godfather Will Get Down With You Now
Let's say you have an audience with the Godfather of /
Soul. How would it be /smart to address him?
Mr. Richard
Mr. Domino
Mr. Brown
Mr. Checker
3
James Brown is the Godfather of Soul.

```

- Struktura redom po linijama:

- prva linija ("An Episode You Can't Refuse") je glavni naslov ("title")
- naslov ("category") za prvo pitanje ("On the Run With a Mammal")
- tekst prvog pitanja ("Let's say...?")
- četiri ponuđena odgovora
- redni broj tačnog odgovora od 1 do 4 (u ovom slučaju, 1)
- objašnjenje tačnog odgovora ("A lamb is just a young sheep.")

Ista struktura se ponavlja i za sledeća pitanja

- Važno je napomenuti da je u tekstu pitanja uključen i karakter /: to je urađeno da bi se uvela nova linija, jer bi Python u suprotnom ispisao veoma dugačku liniju, bez prelaska u novi red; kao što će kasnije biti pokazano, svaki znak / će biti zamenjen newline karakterom, čime će se prebaciti naredni tekst u novu liniju

Funkcija open_file()

- Prva funkcija u programu je open_file():
- prihvata ime fajla i mod pristupa (oba su stringovi)

- vraća odgovarajući objekat tipa fajl
- koristi try i except da bi se sprečio IOError (na primer, u slučaju kada fajl ne postoji)
- Ako se javi neki problem pri otvaranju fajla trivia.txt, doći će do izuzetka; u tom slučaju, nema smisla nastavljati izvršenje programa: na ekranu se ispisuje odgovarajuća poruka, i poziva se funkcija sys.exit() koja trenutno prekida izvršenje; da bi se ona pozvala, potrebno je da se uveze modul sys

- Odgovarajući deo koda je:

```
# Trivia Challenge
# Trivia game that reads a plain text file

import sys

def open_file(file_name, mode):
    """Open a file."""
    try:
        the_file = open(file_name, mode)
    except IOError as e:
        print("Unable to open the file", file_name, "Ending program.\n", e)
        input("\n\nPress the enter key to exit.")
        sys.exit()
    else:
        return the_file
```

Funkcija next_line()

- Zatim se definiše funkcija next_line(), koja prihvata objekat tipa fajl i očitava narednu liniju teksta iz fajla trivia.txt:

```
def next_line(the_file):
    """Return next line from the trivia file, formatted."""
    line = the_file.readline()
    line = line.replace("/", "\n")
    return line
```

- Prilikom očitavanje linije, mora se odraditi jedno formatiranje: svi karakteri / se menjaju newline karakterima (\n) -- na taj način će Python znati gde da započne novu liniju

Funkcija next_block()

- Funkcija `next_block()` čita ceo blok koji odgovara jednom pitanju:
 - prihvata objekat tipa fajl
 - vraća 4 stringa: `category`, `question`, `correct`, `explanation` (naslov, pitanje, tačan odgovor, objašnjenje) i listu stringova `answers` (ponuđeni odgovori)
- Odgovarajući kod je:

```
def next_block(the_file):
    """Return the next block of data from the trivia file."""
    category = next_line(the_file)

    question = next_line(the_file)

    answers = []
    for i in range(4):
        answers.append(next_line(the_file))

    correct = next_line(the_file)
    if correct:
        correct = correct[0]

    explanation = next_line(the_file)

    return category, question, answers, correct, explanation
```

- Kada se stigne do kraja fajla `trivia.txt`, očitavanje linije vraća prazan string; znači, u tom slučaju `category` postaje prazan string; `category` će se proveravati u `main()` funkciji, pa kada se ustanovi da je taj string prazan, igra se završava

Funkcija `welcome()`

- Ova funkcija želi dobrodošlicu igraču i ispisuje na ekranu glavni naslov (pošto ga prethodno prihvati kao argument):

```
def welcome(title):
    """Welcome the player and get his/her name."""
    print("\t\tWelcome to Trivia Challenge!\n")
    print("\t\t", title, "\n")
```

Pripremne radnje

- Zatim se kreira funkcija main(), koja sadrži glavnu petlju programa: u prvom delu ove funkcije, poziva se funkcija koja otvara fajl, preuzima se glavni naslov (prva linija fajla trivia.txt), poziva se funkcija za dobrodošlicu i rezultat koga je igrač postigao se inicijalno postavlja na 0

- Kod koji odgovara prvom delu funkcije je:

```
def main():
    trivia_file = open_file("trivia.txt", "r")
    title = next_line(trivia_file)
    welcome(title)
    score = 0
```

Postavljanje pitanja igraču

- Zatim se očitava prvi blok (koji odgovara prvom pitanju); zatim se startuje while-petlja, koja će postavljati pitanja sve dok ih ima, to jest sve dok category ne postane prazan string; pitanje se postavlja tako što se ispisuje naslov, zatim tekst pitanja, i redom četiri ponuđena odgovora

- Odgovarajući deo koda je:

```
# get first block
category, question, answers, correct, explanation = next_block(trivia_file)
while category:
    # ask a question
    print(category)
    print(question)
    for i in range(4):
        print("\t", i + 1, "-", answers[i])
```

Preuzimanje odgovora od igrača

- Zatim se od igrača traži da unese svoj odgovor:

```
# get answer
answer = input("What's your answer?: ")
```

Provera tačnosti odgovora

- Zatim se igračev odgovor poredi sa tačnim odgovorom: ako se poklapaju, igraču se čestita i njegov rezultat povećava za jedan; ako ne, igrač se obaveštava da je promašio; u oba slučaja, ispisuje se objašnjenje zašto je tačan odgovor tačan; na kraju se ispiše trenutni rezultat

- Odgovarajući deo koda je:

```
# check answer
if answer == correct:
    print("\nRight!", end=" ")
    score += 1
else:
    print("\nWrong.", end=" ")
print(explanation)
print("Score:", score, "\n\n")
```

Prelaz na naredno pitanje

- Zatim se poziva funkcija next_block() koja preuzima blok stringova koji odgovaraju narednom pitanju; ako nema više pitanja, category će biti prazan string i to će biti znak da se izvršenje while-petlje prekine:

```
# get next block
category, question, answers, correct, explanation = next_block(trivia_file)
```

Kraj igre

- Pošto se završi izvršenje while-petlje, fajl trivia.txt se zatvara i rezultat ispisuje na ekranu:

```
trivia_file.close()

print("That was the last question!")
print("You're final score is", score)
```

Pokretanje funkcije main()

- Pretposlednja linija koda pokreće igru:

```
main()  
input("\n\nPress the enter key to exit.")
```